الجمهورية العربية السورية
وزارة التعليم العالي والبحث العلمي
الجامعة السورية الخاصة

الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

Automated Security Platform

Senior-2 Project Report – Prepared in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Informatics Engineering, Specializing in System Security and Computer Networks .

Prepared by

Abdulmalek Nawaf Salameh

Supervised by

Dr. Wassim Al-Juneidi - Eng. Amjad Al-Safadi

June – 2026

SUPERVISOR CERTIFICATION

I certify that the preparation of this project entitled **Automated Security Platform**, prepared by **Abdulmalek Nawaf Salameh**, was made under my supervision at Faculty of Artificial Intelligence Engineering in partial fulfillment of the Requirements for the Degree of Bachelor of Science in Informatics Engineering, **Specializing in System Security and Computer Networks** .

Name : Signature: Date:

الملخص :

في ظلّ التطور المتسارع للتهديدات السيبرانية وتزايد حجم البيانات الأمنية، تبرز الحاجة إلى حلول مبتكرة تعزّز قدرات أنظمة الكشف والاستجابة. تواجه أنظمة إدارة معلومات وأحداث الأمن التقليدية، مثلWazuh ، تحديات في معالجة الكم الكبير من التنبيهات الأمنية والتمييز بين التهديدات الحقيقية والإنذارات الكاذبة، مما يؤدي إلى إجهاد المحللين وارتفاع متوسط زمن الاستجابة للحوادث. واستجابةً لهذا التحدي، يقدّم هذا المشروع منصة أمن سيبراني مؤتمتة مدعومة بالذكاء الاصطناعي، تهدف إلى تعزيز Wazuh من خلال دمج تقنيات التوليد المعزز بالاسترجاع RAG ونماذج اللغة الكبيرة LLMs لتوفير تحليل سياقي ذكي للتنبيهات، إضافة إلى اقتراح وتنفيذ إجراءات استجابة مناسبة عند الحاجة. يتجاوز النظام المقترح حدود القواعد التقليدية الثابتة من خلال فهم أعمق للسياق الأمني، وتقليل الإنذارات الكاذبة، ودعم اتخاذ القرار الأمني بصورة أكثر دقة وموثوقية.

تم تنفيذ النموذج المقترح واختباره ضمن بيئة افتراضية باستخدامVirtualBox ، تضمنت أجهزة تعمل بأنظمة Ubuntuو Windowsمع Wazuh Agents ، وجهاز Parrot Security لمحاكاة الهجمات. شملت المنصة المطوّرة نظام Wazuh لجمع الأحداث وتحليلها، وقاعدة بيانات متجهية Qdrant لبناء قاعدة معرفة أمنية، ونموذج تضمين نصوص BGE لتحويل السجلات والمعرفة إلى متجهات، إضافة إلى نموذج اللغة الكبير Llama3.2 عبر .Ollama اعتمدت منهجية العمل على تنفيذ سيناريوهات هجومية مختلفة، ثم قيام Wazuh بالتقاط السجلات والتنبيهات الناتجة عنها، لتتم معالجتها لاحقًا عبر مرحلتي RAG Detection و RAG Response.يقوم النظام باسترجاع السياق المناسب من قاعدة المعرفة، وتحليل الحدث بلغة طبيعية، وتوليد تنبيهات واستجابات محسّنة تُحقن مرة أخرى في واجهةWazuh ، مع إمكانية تنفيذ بعض الإجراءات مثل حظر عنوان IP أو عزل ملف أو تعطيل مستخدم وفق ضوابط محددة.

Abstract:

In light of the rapid evolution of cyber threats and the growing volume of security data, there is a pressing need for innovative solutions that enhance the capabilities of detection and response systems. Traditional Security Information and Event Management (SIEM) systems, such as Wazuh, face challenges in processing the overwhelming stream of security alerts and distinguishing between real threats and false positives. This results in analyst fatigue and an increase in the average response time to incidents. In response to this challenge, this project proposes an automated, AI-powered cybersecurity platform designed to augment Wazuh by integrating Retrieval-Augmented Generation (RAG) techniques and Large Language Models (LLMs) to provide intelligent contextual analysis of alerts, as well as to recommend and execute suitable response actions when required. The proposed system goes beyond traditional static rules by providing a deeper understanding of the security context, reducing false positives, and supporting more accurate and reliable security decision-making.

The prototype was implemented and tested within a virtualised environment using VirtualBox, comprising Ubuntu and Windows machines running Wazuh Agents, as well as a Parrot Security machine for attack simulation. The developed platform includes Wazuh for event collection and analysis, a Qdrant vector database for building a security knowledge base, a BGE text embedding model for converting logs and knowledge into vector representations, and the Llama3.2 large language model via Ollama. The methodology involved simulating various attack scenarios, with Wazuh capturing the resulting logs and alerts, which are then processed through two stages: RAG Detection and RAG Response. The system retrieves relevant context from the knowledge base, analyses the incident in natural language, and generates enhanced alerts and response outputs that are injected back into the Wazuh interface, with the ability to perform selected actions such as blocking an IP address, quarantining a file, or disabling a user under controlled conditions.

Table of Contents

Table of Tables

Table of Figures

1. Introduction

1.1 Introduction

In light of the rapid evolution of the global digital environment, organizations have become entirely dependent on their technological infrastructure, making cybersecurity not merely a supporting function, but a fundamental pillar for business continuity and the protection of organizational assets. With this growing dependence, cyber threats have increased in both scale and complexity, evolving from simple individual attacks into sophisticated and systematic campaigns targeting critical infrastructure and the theft of sensitive data. This transformation in the threat landscape necessitates continuous efforts to enhance traditional defense mechanisms, which often rely on static signature databases and may lack effectiveness against unprecedented threats such as Zero-Day Threats and Advanced Persistent Threats (APTs)[1] .

To address these challenges, Security Information and Event Management (SIEM) systems have emerged, such as Wazuh, which combines the functions of Security Information and Event Management (SIEM) with Endpoint Detection and Response (EDR). These systems play a pivotal role in collecting and analyzing vast amounts of security data from multiple sources and generating the alerts necessary to address potential incidents. However, the effectiveness of such systems remains constrained by the human capacity to process the continuous stream of alerts. Analysts in Security Operations Centers (SOCs) face an overwhelming volume of notifications, leading to the phenomenon of alert fatigue, in which genuine alerts may be overlooked or response actions delayed due to the large number of false positives [1]. Statistics indicate that the Mean Time to Respond (MTTR) remains high in many organizations, allowing attackers to remain within networks for extended periods before being detected and contained.

In this context, Artificial Intelligence (AI) emerges as a transformative solution to this challenge. In recent years, Machine Learning techniques have been integrated into SIEM systems to improve detection accuracy and reduce false positives. The most recent and influential advancement lies in the emergence of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) techniques. These technologies are not limited to detection processes or statistical analysis; rather, they provide the capability to understand complex security contexts, interpret alerts in natural language, and generate customized and detailed response plans based on rich and continuously updated knowledge bases.[2]

The integration of these capabilities into an automated security platform represents a paradigm shift from the model of "automated assistance" to that of "intelligent response." This research seeks to bridge this gap by developing a system based on RAG and LAGMs that integrates directly with Wazuh. The proposed system is distinguished by its ability not only to provide recommendations, but also to automate remediation and response processes, thereby ensuring that preventive actions are executed within seconds rather than hours or days. Consequently, the Security Operations Center can be transformed into a proactive and effective defensive system [3]. This transformation constitutes the core of the present study, which aims to deliver a practical and implementable solution that enhances security efficiency while reducing the cognitive and operational burden on security analysts

.

1.2 Research Problem

Despite the significant progress achieved by Security Information and Event Management and endpoint security systems such as Wazuh in threat detection and the aggregation of security data, a fundamental gap remains between the capabilities of automated detection and the need for intelligent automated response and remediation. The core problem lies in the fact that most current response mechanisms, including Wazuh's Active Response feature, rely on fixed and pre-defined rules, which lack the flexibility required to address complex scenarios or evolving threats that demand deep contextual understanding [4]. This deficiency in contextual intelligence results either in manual and slow response processes, thereby increasing the Mean Time to Respond (MTTR), or in automated responses that are rigid and ineffective when confronted with emerging threats.

The scientific challenge extends beyond merely applying Artificial Intelligence for detection purposes; rather, it concerns how preliminary security alerts can be transformed into reliable and automatically executable remediation actions [5]. This process requires the advanced integration of generative Artificial Intelligence techniques, particularly Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG), in order to enable the system to achieve several complementary objectives. First, it must be capable of interpreting complex security events and correlating them with threat intelligence sources such as the MITRE ATT&CK framework. Second, it should generate a customized response plan for each individual case. Third, it must

execute this plan in a secure and reliable manner through Application Programming Interfaces (APIs) or the automation tools integrated into Wazuh.

Failure to achieve such intelligent automation leaves Security Operations Centers in a persistent state of delayed reaction to threats, thereby exposing organizations to significant financial and operational risks [6]. Accordingly, this research seeks to provide a systematic solution aimed at bridging this gap by transforming Wazuh from a conventional detection system into a fully automated response and remediation system capable of handling advanced and complex threats with both flexibility and intelligence.

1.3 Project Objectives

This project seeks to develop an Automated Security Platform that represents the next generation of cybersecurity solutions through the integrated combination of the detection and response capabilities (EDR/SIEM) provided by Wazuh with Generative Artificial Intelligence technologies, specifically Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG). This integration is intended to bring about a fundamental shift in the way security systems address threats, moving from the stage of manual analysis to a stage of autonomous understanding and intelligent automated response.

To achieve this overarching objective, the research focuses on accomplishing a set of specific scientific objectives, as follows:

1. **Developing an Intelligent Copilot System Based on RAG Technology**:

   This component aims to design and implement a Copilot system that integrates directly with alerts generated by the Wazuh platform and relies on Retrieval-Augmented Generation (RAG) to produce security incident alerts and accurate, reliable response recommendations. The system enhances its decision-making process by referencing a domain-specific security knowledge base in the form of a vector database, thereby ensuring that its recommendations are grounded in trustworthy knowledge sources and aligned with the security context of the incident.

2. **Developing an Integrated Automated Response and Remediation Mechanism**:

This objective involves the development of an intelligent automation module capable of transforming the recommendations generated by the Copilot into practically executable remediation actions, such as isolating an infected host, blocking suspicious communications, or deleting malicious files. These actions are carried out through the Active Response feature integrated into Wazuh, while incorporating a Verification and Audit Mechanism to ensure execution accuracy and maintain system integrity throughout the automation process.

3. **Establishing a Vector-Based Security Knowledge Base**:

This objective focuses on building a vector-based database that leverages vector indexing techniques and contains Incident Response Playbooks, Cyber Threat Intelligence (CTI) information, and Security Best Practices. This knowledge base serves as the primary reference source for supporting the RAG system, enabling it to generate recommendations that are context-aware and grounded in precise security references.

4. **Building an interactive graphical interface to suggest responses**.

This objective focuses on designing and developing an interactive graphical user interface (GUI) that enables security analysts to view alerts, review AI-generated response recommendations, and monitor the status of remediation actions in real time. The interface is intended to provide a clear and user-friendly environment through which analysts can interact with the Copilot system, validate suggested actions when necessary, and obtain contextual information related to each security incident. By improving visibility, usability, and decision support, the interface enhances the overall efficiency of incident handling and facilitates seamless collaboration between human analysts and the automated security platform.

5. **Evaluating the Performance and Operational Efficiency of the Developed Platform**:

This component aims to conduct a rigorous experimental evaluation of the platform's performance within a lab environment that simulates realistic attack scenarios. Key performance indicators, such as Mean Time to Respond (MTTR) and the level of security analyst efficiency before and after the implementation of the system, will be measured in order to assess the

platform's effectiveness in reducing response time and improving the quality of security-related decision-making in comparison with conventional systems.

1.4 Literature Review

The need for automation in the field of cybersecurity is not a recent development; rather, it is the result of a long evolutionary trajectory imposed by the growing challenges faced by Security Operations Centers (SOCs). This concept has evolved through three principal stages, which together have formed the historical framework for the development of detection and response systems.

In the first stage, that is, prior to 2024, the need for Security Information and Event Management (SIEM) systems and Endpoint Detection and Response (EDR) systems emerged as a result of the rapid growth in security log volumes and the impracticality of analyzing them manually. The emergence of systems such as Wazuh represented a pivotal turning point in the aggregation and analysis of security data and in providing comprehensive visibility into network activities. However, this progress also revealed new challenges, most notably the phenomenon of alert fatigue, which resulted from high rates of false positives and excessive reliance on static rules. As these challenges intensified, research began to shift toward the use of Machine Learning techniques to improve the accuracy of anomaly detection within SIEM systems, marking the first step toward the application of Artificial Intelligence in this field.[12]

The second stage, which coincided with 2024, witnessed the more explicit integration of Artificial Intelligence into cybersecurity systems, with research efforts primarily focused on reducing false positives and improving analytical accuracy. During the first half of that year, studies concentrated on applying traditional Machine Learning algorithms to detection processes within SIEM and EDR systems [5]. In the second half, however, research began to explore Large Language Models (LLMs) as assistive tools for analysts in Security Operations Centers (SOCs). These models were employed to analyze security incident logs and generate preliminary response plans, thereby accelerating decision-making and improving the quality of recommendations [4]. Subsequent studies also examined the potential use of LLMs in the planning and review of incident response strategies [9], while others investigated their role in supporting threat modeling frameworks and transforming Wazuh rules into more flexible and dynamic formats [10]. Nevertheless, the scope of improvement

remained largely confined to the detection stage and did not effectively extend to the response stage, as the process of determining and executing the appropriate remediation action continued to depend on manual human intervention.

In the third stage, which was characterized by the rise of Generative Artificial Intelligence in 2025, a fundamental shift occurred toward the concept of intelligent automation. Recent studies have demonstrated the importance of integrating Retrieval-Augmented Generation (RAG) with threat intelligence frameworks such as MITRE ATT&CK in order to enhance the efficiency of Security Operations Centers (SOCs) [1]. As this direction continued to develop, researchers found that relying solely on the general knowledge embedded in LLMs was insufficient, which necessitated combining these models with RAG to generate incident responses that are both customized and contextually relevant [6]. This integration led to the emergence of advanced models of Autonomous Incident Response systems capable of executing precise remediation actions based on reliable knowledge-driven recommendations [3]. Other studies also addressed the enhancement of RAG capabilities through Knowledge Graphs [7], as well as the exploration of Agentic AI applications within SOC environments to pave the way toward comprehensive process automation [8]. In addition, practical implementations have emerged in the form of integrating LLMs with Wazuh and automating response tasks through platforms such as Jira [11], alongside the use of lightweight language models for incident response planning and execution.[9]

The above discussion may be summarized as follows:

Table 1.1 : Literature Review

Research	Authors	Findings	Relationship to the Project	Reference
Combating Alert Fatigue: AI-Enhanced SIEM Framework for Optimized Incident Response	W. Olabiyi	The study demonstrated that traditional Machine Learning techniques are effective in anomaly detection; however, they lack the capability to interpret context and	It represents the pre-LLM stage and highlights the need for more intelligent solutions with stronger contextual understanding, thereby justifying the transition to	[12]

		generate complex .response actions	Large Language Models .(LLMs)	
Using LLMs as Incident Prevention Copilots in Cloud Infrastructure	S.P. Veluru; M.K. Manchala	The study provides a practical framework for employing Large Language Models (LLMs) as an assistive tool for cyber defenders, with a particular emphasis on .preventive aspects	The study provides a framework for employing LLMs as a Copilot tool within a live environment, thereby supporting the concept of an automated assistant in the .proposed project	[4]
Employing LLMs for Incident Response Planning and Review	Sam Hays; Jules White	The study demonstrated that LLMs are capable of analyzing and updating incident response documentation with high efficiency, thereby reducing the level of .human effort required	The study provides the theoretical foundation for the use of LLMs in generating and updating response playbooks and operational guidelines, which constitutes an essential component of the knowledge .base	[5]
Advancing Security Operations Centers: Modern Use Cases, MITRE ATT&CK Integration, and Coverage Optimization	Salman Ghani Virk; Jawaid Iqbal; Atif Ali; Ali Rashid Mahmud; Imran Rashid; Tariq Hanif	The study confirmed that integrating Artificial Intelligence with the MITRE ATT&CK framework constitutes the most effective approach to improving the key performance indicators of Security Operations Centers .(SOCs)	The study confirms the need for intelligent automation and provides evaluation metrics, such as Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR), which will be used to assess the success of .the proposed platform	[1]

IRCopilot: Automated Incident Response with Large Language Models	Xihuan Lin; Jie Zhang; Gelei Deng; Tianzhe Liu; Xiaolong Liu; Changcai Yang; Tianwei Zhang; Qing Guo; Riqing Chen	The study introduced the IRCopilot framework, which simulates the human decision-making process in incident response using LLMs, thereby enabling comprehensive .automation	The study provides an advanced model for automating the full phases of incident response, which serves as the theoretical framework from which the .project draws inspiration	[3]
Advancing Autonomous Incident Response: Leveraging LLMs and Cyber Threat Intelligence	Amine Tellache; Abdelaziz Amara Korba; Amdjed Mokhtari; Horea Moldovan; Yacine Ghamri-Doudane	The study demonstrated that integrating LLMs with Cyber Threat Intelligence (CTI) significantly enhances the system's ability to make accurate and timely autonomous response .decisions	Enriching the RAG knowledge base with Cyber Threat Intelligence (CTI) data in order to enhance the accuracy of recommendations, which constitutes a primary objective in the development of the vector-based .knowledge base	[6]
CyberAlly: Leveraging LLMs and Knowledge Graphs to Empower Cyber Defenders	Minjune Kim; Jeff Wang; Kristen Moore; Diksha Goel	The study demonstrated that the use of Knowledge Graphs in conjunction with LLMs provides a deeper and more interconnected security context, thereby enhancing response .capabilities	The study provides a vision for deepening security context and overcoming the limitations of conventional RAG through the use of .Knowledge Graphs	[7]

Autonomous Agentic AI Architectures for Optimizing Security Operations Centers (SOC) KPIs	Miroslav Stefanov et al	The study presented architectural models for intelligent agents capable of operating autonomously in order to improve the efficiency of Security Operations Centers (SOCs) and enhance their key performance indicators .(KPIs)	It supports the transition from a Copilot model, as an assistive tool, to an Agentic AI model capable of autonomous .incident response	[8]
Incident Response Planning Using a Lightweight Large Language Model with Reduced Hallucination	Kim Hammar; Tansu Alpcan; Emil C. Lupu	The study demonstrated the feasibility of using lightweight LLMs for efficient response planning, thereby reducing infrastructure .requirements	The study provides practical solutions to the problem of computational resource requirements, particularly the need for GPUs demanded by large-scale models, thereby supporting the feasibility and practical applicability of the .proposed project	[9]
A Large Language Model-Supported Threat Modeling Framework for Transportation Cyber-Physical Systems	M. Sabbir Salek; Mashrur Chowdhury; Muhaimin Bin Munir; Yuchen Cai; Mohammad Imtiaz Hasan; Jean-Michel Tine; Latifur	The study proposed a framework that enables LLMs to analyze threats and generate specialized defensive rules, thereby opening the way for the dynamic generation of .Wazuh rules	The study highlights the potential of using LLMs to dynamically generate Wazuh rules, thereby strengthening the automation dimension of .the proposed project	[10]

	Khan; Mizanur Rahman			
Leveraging LLM Integration with Wazuh and Jira for Automated Cyberattack Detection and Incident Response	P.P. Naraduhita; M.T.E. Bumbungan	The study presented a practical model illustrating how Wazuh can be integrated with LLMs to improve detection and analysis processes, thereby supporting the technical feasibility of the proposed .project	An applied study that focuses on the practical integration of Wazuh with LLMs, thereby providing a practical model for the implementation of the .proposed project	[11]

This research represents the natural extension of these scholarly developments, as it seeks to move beyond the stage of "response assistance" toward "fully automated response and remediation." It draws upon the latest advances in LLMs and RAG to transform Wazuh into a security platform capable of contextual analysis, decision-making, and autonomous execution of remediation actions in an intelligent manner. In doing so, it aims to achieve a qualitative transformation in the concept of security incident management.

1.5 Challenges

Developing an automated security platform that relies on Generative Artificial Intelligence, particularly LLMs and RAG, and integrates with live systems such as Wazuh constitutes a multidimensional challenge. These challenges are not limited to technical aspects alone, but also extend to security controls and performance requirements in real-world production environments. The success of this type of project requires an integrated approach that balances technological innovation with practical guarantees of reliability, security, and sustainability.

First: **The Problem of Hallucination and Trust in Large Language Models** (**Hallucination and Trust**)

Despite the remarkable generative capabilities of Large Language Models (LLMs), they remain susceptible to the phenomenon of hallucination, which refers to the production of outputs that appear logical and linguistically coherent yet lack a factual basis or valid supporting evidence. This phenomenon poses a serious risk in cybersecurity environments, as an incorrect recommendation generated by the system may lead to the disruption of critical systems, the deletion of legitimate files, or the creation of new unintended vulnerabilities.

Addressing this problem requires the establishment of a rigorous validation layer within the RAG architecture to ensure that all response recommendations are based exclusively on trusted data sources derived from the vector-based security knowledge base. In addition, an extra Verification Layer should be implemented to examine the retrieved sources before allowing the model to generate its final recommendation, thereby enhancing the level of trust in the resulting decisions and reducing operational risks.[8]

Second: **Latency and Real-Time Performance**

The project aims to minimize the Mean Time to Respond (MTTR) to the greatest extent possible, which requires the automated system to deliver near real-time performance. However, the operation of LLMs, together with retrieval processes from vector databases, may introduce latency that can adversely affect the effectiveness of automation.

Addressing this challenge requires optimizing the entire processing pipeline, from the moment an alert is generated in Wazuh to the execution of the remediation action. This includes the use of high-efficiency retrieval algorithms within the RAG framework, as well as performance optimization techniques such as quantization and pruning, in addition to reliance on Graphics Processing Units (GPUs) or specialized accelerators to ensure near-instantaneous processing without compromising the quality of the response.[3]

Third: **Scalability and Computational Constraints**

Large Language Models (LLMs) consume substantial computational resources during both training and inference processes, which constitutes a significant obstacle for organizations with limited resources or those lacking robust cloud infrastructure.

Overcoming this obstacle requires the selection of lightweight LLMs specifically designed to operate in resource-constrained environments while maintaining an acceptable level of accuracy in generated recommendations. In addition, the system should be designed to be scalable so that it can accommodate the expected growth in the volume of alerts and security data without negatively affecting overall performance or causing excessive resource consumption.[9]

Fourth: **Verification of Automated Actions and the Avoidance of Side Effects**

When executing automated remediation commands, such as isolating devices, disabling communications, or deleting files, there arises a critical need for a post-action verification mechanism to ensure that the action has been carried out successfully and has not produced harmful side effects. In the absence of such verification, there is a risk of disrupting critical services or causing otherwise healthy systems to become unavailable.

This challenge may be addressed by establishing a feedback loop between the automation module and the Wazuh system in order to confirm outcomes and verify the integrity of the executed actions. This loop should include a post-action verification process that assesses the effectiveness of the remediation measure and confirms the elimination of the malicious behavior without adversely affecting the system's core functions.[11]

Fifth: **Resistance to Model Manipulation and Advanced Adversarial Attacks**

Generative models constitute a potential target for adversarial attacks, whereby attackers may manipulate inputs or engineer security logs in ways that deceive the model and lead to misleading outputs or incorrect response decisions.

Addressing this challenge requires the integration of advanced robustness measures, such as detecting abnormal patterns in input data and adopting multi-layered verification policies to ensure the reliability of system responses. Furthermore, continuous monitoring mechanisms should be implemented to detect any attempts to manipulate either the vector-based knowledge base or the LLM itself, thereby ensuring that the system cannot be exploited as an offensive tool in its own right.

Addressing these challenges requires a balanced design approach that combines innovation in the application of Generative Artificial Intelligence with rigor in ensuring security, reliability, and performance. This balance represents the core scientific value of the project, as it seeks to achieve

intelligent automation without compromising the fundamental principles of protection and trust within the cybersecurity environment.

1.6 Practical Applications

The proposed automated security platform, which integrates Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) with Wazuh Active Response, possesses broad practical applications that extend beyond the scope of traditional Security Operations Centers (SOCs).
Its ability to understand complex security contexts, generate dynamic responses, and automatically execute remediation actions makes it a transformative tool for strengthening cybersecurity across organizations and various critical sectors.

The following are five major domains and practical application scenarios that illustrate the added value the platform can provide:

1. Managed Security Service Providers (MSSPs)

Managed Security Service Providers (MSSPs) are among the primary beneficiaries of this system, given the nature of their work, which requires the management of diverse security environments for a large number of clients while maintaining compliance with defined service-level commitments.

- Reducing Alert Fatigue:

  Managed security centers face a substantial challenge in handling hundreds of thousands of alerts on a daily basis. The automated platform enables the initial analysis and automatic classification of incidents, thereby reducing the burden associated with processing low- and medium-severity alerts. This, in turn, allows human analysts to concentrate exclusively on critical cases.

- Standardizing Response Procedures:

  Given the diversity of client environments, RAG can retrieve client-specific playbooks and security policies, thereby ensuring that each response remains aligned with applicable regulatory requirements and contractual obligations. In this way, consistency in incident response can be maintained even under conditions of full automation.

- Reducing Mean Time to Respond (MTTR):
  Thanks to the capability of Wazuh Active Response to immediately isolate infected devices or block malicious IP addresses, the platform contributes to reducing the time required to contain incidents, which constitutes one of the key performance indicators in MSSP environments.

2. Financial Institutions and Banks

Financial institutions and banks are regarded as primary targets of cyberattacks and are subject to stringent regulatory oversight that requires an immediate and well-documented response to security incidents.

- Automating Responses to Fraud Incidents:
  The system can analyze suspicious transaction patterns detected by fraud detection systems, and then employ an LLM to determine the appropriate course of action, whether that involves freezing an account, notifying the customer, or reporting the incident to regulatory authorities. The required action is then executed automatically through Application Programming Interfaces (APIs) or through Wazuh to isolate the suspected devices.

- Regulatory Compliance:
  RAG retrieves the precise regulatory requirements relevant to the incident, such as PCI DSS standards or central bank regulations, and incorporates them into the automatically generated report produced during the response process.
  This ensures that all actions undertaken are properly documented and legally compliant, thereby enhancing both regulatory compliance and institutional transparency.

3. Critical Infrastructure and the Energy Sector

Protecting critical infrastructure—such as power plants, water networks, and transportation systems—is one of the most significant challenges in cybersecurity, as any error in automation may lead to catastrophic consequences that directly affect public safety.

- Operational Technology (OT) Security:

  The platform can be integrated with Operational Technology (OT) security systems in order to monitor industrial control protocols (ICS/SCADA).

  The LLM is used to analyze the complex alerts generated by these systems, which conventional SIEM solutions are often unable to interpret accurately. It then produces precise and conservative response recommendations, thereby supporting more reliable decision-making in OT security environments.

- Constrained Response:

  In such sensitive environments, automation is designed to be constrained and strictly controlled. For example, the role of Wazuh Active Response may be limited to isolating a non-critical system or issuing an immediate high-priority alert, while the role of the LLM remains confined to providing recommendations directed to the human operator without direct execution.

4. Government and Defense Agencies

These institutions are exposed to complex and advanced attacks, particularly Advanced Persistent Threats (APTs), which require an intelligent response capable of comprehending the full operational context.

- Advanced Persistent Threat (APT) Analysis:

  The platform employs an LLM to analyze Cyber Threat Intelligence (CTI) reports and correlate them with the Tactics, Techniques, and Procedures (TTPs) of the attack within the MITRE ATT&CK framework.

  Subsequently, RAG retrieves the most appropriate response procedures based on the type of threat identified, thereby ensuring an adaptive and effective response.

- Automated Forensic Investigation:

  The platform is capable of collecting evidence from multiple sources—such as event logs, system files, and process memory—in order to automatically generate a comprehensive incident narrative.

This contributes to accelerating the forensic investigation process and reducing the time required to determine the scope of the attack and its operational impact.

5. **Healthcare Sector**

The healthcare sector is regarded as one of the most sensitive sectors due to the nature of the data it handles, which includes Protected Health Information (PHI), in addition to the critical importance of maintaining the continuity of medical services.

- Protection of Patient Data (PHI Protection):

  The platform can monitor Electronic Health Records (EHR) systems in order to detect unauthorized access attempts or abnormal data exfiltration behaviors.

  The LLM evaluates the severity of the incident in accordance with healthcare regulatory requirements, such as HIPAA, and then directs Wazuh to execute the appropriate protective measures, such as isolating the device or terminating the suspicious session.

- Response to Ransomware Attacks:

  Given the sensitivity of this environment, the system can respond to ransomware attacks by immediately isolating infected devices, retrieving the appropriate recovery plan from the RAG-based knowledge base, and guiding IT teams to initiate restoration procedures in a coordinated and precise manner.

2. Chapter One: Theoretical Study

2.1 Comprehensive Overview of Intrusion Detection and Response

2.1.1 Intrusion Detection

Intrusion Detection is the process of monitoring and analyzing events occurring within a computer system or network in order to identify indications of potential security incidents, namely violations of security policies or imminent threats to them. Intrusion Response, on the other hand, focuses on taking rapid actions once threats are detected in order to stop them or minimize their impact.

The detection process is often carried out through automated systems known as Intrusion Detection Systems (IDSs), which collect and analyze activity and network logs. Response actions, however, may be performed manually or automated through Intrusion Prevention Systems (IPSs), which possess the capability to stop threats as they are being detected. Modern systems combine both detection and prevention capabilities in what is known as Intrusion Detection and Prevention Systems (IDPSs), thereby achieving both functions simultaneously.[13]

2.1.2 Historical Overview of the Evolution of Intrusion Detection Systems

The concept of **intrusion detection** emerged in the early 1980s as a response to the growing need to protect computer systems. Dorothy Denning is credited with presenting the first mathematical model for intrusion detection in 1987, in which she proposed a framework based on analyzing system logs to detect abnormal behavior by users and processes [14]. This model, known as the **Denning Model**, served as a starting point for the development of intrusion detection systems based on monitoring statistical patterns of usage behavior. It was also preceded by the pioneering work of James Anderson in 1980, who produced a report on intrusion monitoring, thereby establishing the idea that monitoring event logs could reveal hidden attacks. During the late 1980s and 1990s, research interest in this field increased, driven by major incidents such as the Morris Worm in 1988, which highlighted the need for tools capable of detecting intrusions in real time.

The 1990s witnessed significant developments in intrusion detection technologies. Various commercial and academic intrusion detection systems emerged and were classified into two main categories: **signature-based detection systems** and **anomaly-based detection systems**.

The first type relied on predefined knowledge bases of known attacks, similar to the concept of antivirus signatures. It achieved high accuracy in detecting known intrusions but was unable to identify entirely new attacks. The second type relied on building behavioral profiles of users and systems and then detecting any significant deviation from those profiles [14]. This approach enabled the detection of unknown attacks, such as **Zero–Day Attacks**, by identifying unusual behavior. However, it also suffered from a high rate of false positives due to the diversity of legitimate usage patterns. During this period, the concept of **hybrid detection systems** also emerged, combining both approaches to reduce their limitations and improve detection rates.

From a practical perspective, the widespread adoption of intrusion detection solutions in organizations began toward the end of the 1990s. Leading companies introduced software and hardware solutions for both **network–based** and **host–based** intrusion detection. For example, the open-source **SNORT** system, released in 1998, relied on simple signatures to inspect network traffic and later became a cornerstone of many security practices. In parallel, **Host–Based Intrusion Detection Systems** (**HIDSs**) were developed to monitor system logs, configuration files, and file integrity in order to detect suspicious changes on individual devices.

By the early twenty-first century, cyberattacks had become more complex and faster in execution, creating a need for automated response capabilities in addition to detection. As a result, **Intrusion Prevention Systems** (**IPSs**) emerged as an advanced generation of security systems capable not only of issuing alerts when intrusions are detected, but also of taking immediate actions, such as blocking specific network traffic or terminating malicious processes. Technically, many of these systems were developed by extending intrusion detection systems with the ability to enforce proactive policies, such as dropping suspicious network packets. For instance, in 2003, Enterasys introduced the first commercial IPS device that combined firewall and IDS functions to perform automated actions. Companies such as Cisco and IBM also integrated detection and prevention technologies into unified security solutions.

In a related context, the concept of **Security Operations Centers** (**SOCs**) became increasingly prominent in large organizations, and intrusion detection and response systems became the backbone of these centers. SOC analysis teams relied heavily on IDS/IPS outputs to monitor infrastructure

around the clock. At the same time, new challenges emerged in managing the alerts generated by these systems, as security teams began facing large volumes of daily alerts. This led to the phenomenon of **alert fatigue**, which became evident in the late 2000s, where important alerts could be ignored or response actions delayed due to being overwhelmed by a large number of false or low-priority alerts. Statistics from that period indicated that the **Mean Time to Respond** (**MTTR**) remained high in many organizations due to alert accumulation, giving attackers more time to move freely before being detected. Consequently, by the late 2000s and early 2010s, attention shifted toward improving the intelligence of detection systems to reduce inaccurate alerts, as well as enabling automated alert triage processes to prioritize incidents.

In the late 2010s, with advances in **Machine Learning** technologies, a new phase began in the history of intrusion detection. Machine Learning methods were incorporated into some security products to enable deeper data analysis and detect more complex patterns that manual rules could not identify. These developments paved the way for what is now known as **next-generation intrusion detection systems**, in which algorithms leverage statistical models and deep learning to achieve more accurate and flexible detection.

In addition, the historical evolution of intrusion detection was accompanied by a shift in the incident response model. Although most research efforts over previous decades focused primarily on detection techniques, the last decade has witnessed growing interest in **automated intrusion response systems**. Researchers recognized that detection alone, without rapid response, is no longer sufficient in the face of modern attacks that spread quickly [15]. Accordingly, response lifecycle frameworks were developed, including steps such as selecting the appropriate response action, executing it, and evaluating its impact. This research movement toward response automation represents one of the key features of the modern development of **Intrusion Detection and Response**, where early detection is integrated with immediate containment actions to reduce damage.

Finally, it is important to distinguish between the **expert system model** and the **Machine Learning model**. In the early stages, many IDSs were built as expert systems that relied on the knowledge of security specialists encoded into predefined rules. However, as data volumes increased

and manually formulating all detection rules became increasingly difficult, the model gradually shifted toward learning algorithms capable of deriving detection rules from historical data. This leads to modern developments in which the core models for intrusion detection now include artificial neural networks and complex models capable of extracting attack characteristics with limited human intervention in rule definition.

2.1.3 Classifications of Intrusion Detection Systems

Intrusion detection and response systems can be classified according to multiple criteria that provide a clearer understanding of their functions and areas of application.

The most common classifications are presented below:

1. **Based on the Monitoring Location and Scope**:

Network-Based Intrusion Detection System (**NIDS**): A NIDS monitors network traffic passing through central nodes such as routers and switches, or through independent sensors. It can cover multiple devices and subnets simultaneously and is particularly suitable for detecting attacks that propagate through the network, such as protocol-based attacks, session hijacking, and similar threats. For example, a NIDS device may be deployed at the organization's Internet gateway to detect incoming attacks at the enterprise level.

Host-Based Intrusion Detection System (**HIDS**): A HIDS operates on a specific device, such as a server or personal computer, and monitors events related only to that device, including system logs, CPU usage, system calls, and similar activities. It is suitable for detecting abnormal behavior occurring directly on the host, which may not be visible at the network-traffic level, such as the local execution of malware from a USB drive. In many cases, a HIDS is installed as an agent on sensitive servers.

2. **Based on the Detection Methodology**:

Signature-Based IDS: These systems rely on a predefined knowledge base of known attacks. Each signature represents a specific packet pattern or behavior that indicates an intrusion, such as a particular byte sequence in a payload associated with a virus, or a sequence of connections matching a known attack. Such systems are highly effective in detecting attacks that already exist

in their databases, offering high accuracy and a low rate of false positives for those specific attacks. However, they are unable to detect previously unknown attacks.

Anomaly–Based IDS: These systems establish a baseline of normal behavior through extended monitoring of legitimate activity and the application of statistical or intelligent learning algorithms. During operation, any significant deviation from this baseline, such as a substantial increase in a user's data traffic volume or a process consuming unusual system resources, is considered an indicator of a potential intrusion. These systems are distinguished by their ability to detect unusual activities; however, they often generate a higher rate of false positives than signature-based systems, particularly in dynamic environments or when they are not properly tuned.

Hybrid IDS: These systems combine the two aforementioned approaches in order to benefit from the advantages of both. For example, a hybrid system may use signatures to rapidly detect known attacks, while simultaneously forwarding the remaining traffic to an anomaly detection engine for analysis in order to identify any suspicious behavior .

Figure 2.1 illustrates these classifications.

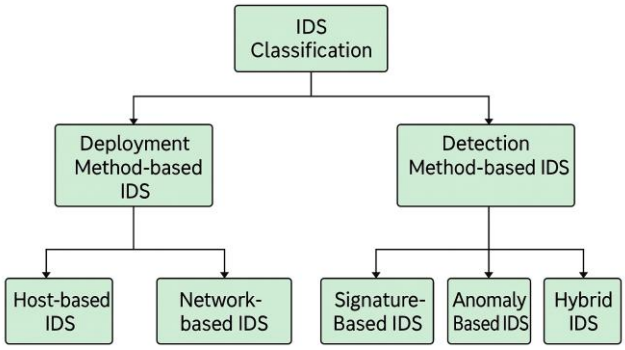


Figure 2.1: Classifications of Intrusion Detection Systems.

2.1.4 Core Operations

Intrusion detection and response systems operate through a set of core processes that ensure effective monitoring and action execution. The typical workflow of these systems can be summarized in the following stages:

1. **Data Collection**:

This is the first and most fundamental step, in which the system captures raw security events from their original sources. In the case of NIDS, this involves intercepting network packets and decoding protocols to extract information such as IP headers, HTTP headers, and other relevant fields. In the case of HIDS, it includes reading system logs, such as Windows Event Logs or Linux log files, monitoring system files and changes made to them, and possibly intercepting system calls at the kernel level. Some systems also collect data from additional sources, such as firewalls or application servers, particularly in distributed environments.

2. **Normalization and Data Transformation**:

After data collection, the diverse data formats are standardized into a unified structure within the system. This is necessary because multiple sources, such as different devices and operating systems, generate logs in various formats. The normalization module translates log fields into a unified internal data model. For example, the format of a login event may differ between Windows and Linux; therefore, it is normalized into a unified field, such as "AUTH Success=True/False", that can be used internally by the system. This process later facilitates the correlation of similar events originating from different sources.[16]

3. **Analysis and Detection**:

This stage represents the core of the system, where patterns and rules are matched in the case of signature-based detection, statistical probabilities of anomalies are calculated in the case of behavior-based detection, or trained Machine Learning models are executed. In modern systems, multiple layers of analysis may be employed. For example, the analysis may begin with rapid matching against known signatures, using structured rules such as regular expressions to inspect packets or messages. Events that do not match known signatures may then be passed to a deeper analysis layer that applies anomaly detection algorithms. During this stage, if an attack pattern condition is met or the anomaly score exceeds a defined threshold, internal alerts are generated to indicate the detection of a potential incident.

4. **Correlation and Logical Linking of Events**:

In advanced systems, the correlation module links separate alerts and events in order to determine whether they form parts of a larger attack. It also helps reduce false positives by verifying whether the reported events support one another within the narrative of a real attack. Technically, correlation relies on logical rules or intelligent algorithms that identify causal or temporal relationships between events.[13]

5. **Alerting and Notification**:

Once an intrusion pattern or security incident has been confirmed after analysis and correlation, the system generates an alert. The alert typically includes a brief description of the incident, such as "SQL Injection attempt on Server X", a suggested severity level—low, medium, high, or critical—based on predefined policies or the strength of the match, as well as contextual data such as the source and destination IP addresses, the affected username, the timestamp, and the unique identifiers of the related events.[13]

6. **Response**

At this stage, countermeasures are executed. If the system includes IPS capabilities or is integrated with control platforms, it may automatically initiate actions to disrupt the attack or limit its impact.[13]

7. **Logging and Reporting**:

The system continuously records all events and actions processed through it, whether or not they are classified as attacks. These records are stored in a database or structured files for later reference .[16]

2.1.5 Challenges

Despite the significant progress achieved in intrusion detection and response technologies, several fundamental challenges continue to limit the full effectiveness of these systems in real-world environments:

1. **False Positives and False Negatives Alerts**:

   Achieving a balance between sensitivity and accuracy is one of the most difficult challenges. Excessive reliance on rigid static rules may lead to a large number of false positives, where any unfamiliar event may be classified as an attack. This results in alert fatigue and negatively affects the response to genuine alerts. Conversely, attempting to significantly reduce false positives by lowering system sensitivity may cause actual attacks to be missed, resulting in false negatives, particularly in the case of innovative or slow-moving attacks that conceal themselves within normal activities.[12]

2. **Lack of Contextual Intelligence**:

   Many traditional detection systems operate in isolation from the broader context. For example, a system may detect and generate an alert for an unusual connection originating from a specific server, without being aware that a new legitimate application has recently been installed on that server, which may explain the observed behavior.[26]

3. **Scalability and Handling Big Data**:

   In modern networks, enormous volumes of event logs and network data are generated every second. Storing and analyzing this data in real time represents a major technical challenge.[26]

4. **Encryption Challenge**:

   The increasing reliance on protocols such as HTTPS and TLS conceals packet contents, thereby hindering in-depth data analysis.[26]

5. **Lack of Integration and Interoperability Among Multiple Tools**:

   In a typical enterprise environment, several different security systems may be deployed, such as firewalls, network-based and host-based IDSs, SIEM platforms, antivirus solutions, and others. These tools often operate in isolation, with limited or no automated information exchange between them. This lack of integration reduces the effectiveness of incident response.[26]

2.1.6 Recent Developments

The field of Intrusion Detection and Response has witnessed significant qualitative developments in recent years, driven by two major technological shifts: the rise of Big Data and Artificial Intelligence, and the emergence of advanced intelligence models such as Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG). These modern technologies offer promising solutions to many of the traditional challenges associated with IDS systems, and they have begun to be integrated, both experimentally and practically, into advanced cybersecurity solutions.

Deep Learning and Enhanced Detection: With the availability of massive volumes of attack data and security logs, it has become possible to train Deep Learning models, such as multilayer neural networks, to improve detection capabilities. Deep Learning algorithms enable the extraction of complex features from data that experts may not be able to define manually with ease. This has contributed to improving the detection rate of new attacks and reducing false positives [17].

The Perspective of "Cognitive Detection" Using LLMs: One of the most significant recent developments is the introduction of Large Language Models (LLMs) into the field. These models, such as GPT-4 and others, have demonstrated unprecedented capabilities in understanding and generating natural language. Researchers have begun exploring the potential of integrating LLMs into intrusion detection systems to transform them into what may be referred to as Cognitive IDSs. The core idea is that LLMs can effectively process unstructured data, such as textual log messages or written intrusion commands, and can also interpret the context of an attack in natural language. For example, a large language model can process an extensive event log report, extract the sequence of suspicious activities, and present it as an understandable summary for the analyst, rather than merely displaying numbers and symbols [18].

Retrieval-Augmented Generation (**RAG**): RAG is considered a complementary technique to the use of LLMs, as it enables the language model to access an external knowledge base during the analysis and generation process. In the context of IDS, this means that the language model can be linked to an information repository containing up-to-date threat data, such as CVE vulnerability databases or Cyber Threat Intelligence reports, allowing it to retrieve information relevant to the event currently being analyzed. In this way, when the model detects a particular behavior, it can rely

on external knowledge to determine whether that behavior is associated with a known vulnerability or attack campaign. This improves analytical accuracy and contextual understanding while reducing the risk of model hallucination, which refers to the tendency of language models to sometimes generate incorrect conclusions based on their internal knowledge [2].

The Role of LLMs in Incident Response: In addition to detection, another advanced use of LLMs has emerged in the areas of response planning and the formulation of incident-handling procedures. For example, a recent study conducted in 2024 showed that the use of Large Language Models such as ChatGPT can significantly enhance the development and review of incident response plans.[5]

2.1.7 Best Practices

- **Multi-Layered Deployment** (**Defense in Depth**): Reliance should not be placed on a single detection tool alone. Best practices require the use of multiple layers of detection systems deployed across different locations.
- **Periodic Tuning and Accurate Initial Calibration**: Sufficient time should be allocated during the system deployment phase to tune its sensitivity and reduce false positives.
- **Enabling Contextual Correlation and Integration with SIEM**: To achieve better results, it is essential to integrate the detection system with a centralized SIEM platform that aggregates its outputs with those of other monitoring tools.[13]
- **Clear and Up-to-Date Incident Response Plans**: Detecting an incident is not sufficient on its own; security teams must also know what actions to take once an incident is detected. Therefore, developing a comprehensive Incident Response Plan and continuously updating it in accordance with emerging threats is considered one of the most important best practices.
- **Periodic Testing**: To ensure the readiness of IDS systems, it is recommended to conduct regular penetration testing simulations and security exercises.
- **Log Retention and Post-Incident Analysis**: Even if an attack appears to have been successfully mitigated, it remains important to retain all incident-related data and analyze it afterward.

2.1.8 Conclusion

In conclusion, this section demonstrates that Intrusion Detection and Response represents a vital and central domain within modern cybersecurity. It has evolved over decades of research and practical implementation to include a wide range of technologies and methodologies that work together to protect information systems.

Despite this progress, the success of Intrusion Detection and Response systems remains dependent on the adoption of strict operational methodologies and best practices, including continuous tuning, tool integration, staff training, and the utilization of lessons learned from previous incidents. Only through a combination of advanced technology and informed management can the required level of protection against intrusions be achieved in the face of escalating cyber threats.

2.2 Comprehensive Overview of Artificial Intelligence

2.2.1 Concept of Artificial Intelligence

Artificial Intelligence is a scientific and technical field that aims to develop systems and software capable of simulating human intelligence and performing tasks that typically require cognitive abilities associated with humans, such as learning, reasoning, and problem-solving. In other words, Artificial Intelligence can be defined as the design of intelligent machines and programs that are able to think, understand, and make decisions in a manner similar to the human mind. This includes enabling machines to understand natural language, recognize patterns in data, and learn from previous experiences in order to improve their future performance.[19]

2.2.2 Historical Overview of Artificial Intelligence

Artificial Intelligence has undergone historical development through successive stages from the mid-twentieth century to the present. The early beginnings date back to the 1950s, when initial research in the field began. In 1950, Alan Turing introduced his well-known test for measuring machine intelligence through a machine's ability to simulate human conversation. In 1956, the Dartmouth Conference was held under the leadership of John McCarthy, Marvin Minsky, and others, marking the official birth of the field of Artificial Intelligence. During the late 1950s, pioneering programs were developed, such as the Logic Theorist, which was designed to solve mathematical problems. In addition, specialized programming languages emerged, most notably LISP, created by McCarthy

in 1958, which became the dominant language for programming Artificial Intelligence applications for several decades.[19]

The Following Decades (**1960s–1970s**): Optimism increased during the 1960s with the emergence of more sophisticated programs, such as the General Problem Solver, as well as early models of simple neural networks, such as the Perceptron. However, by the 1970s, the limitations of these early systems became apparent, leading to a period known as the "AI Winter," during which funding and enthusiasm declined due to the failure to meet overly ambitious expectations. Nevertheless, the 1970s also witnessed important successes, particularly the spread of expert systems based on knowledge rules for solving specific problems. These systems proved effective in certain domains. For example, the MYCIN system, developed in the mid-1970s, used expert rules to diagnose and recommend treatments for blood diseases, paving the way for the adoption of expert systems in medicine and engineering.

Revival and Machine Learning (**1980s–1990s**): In the 1980s, interest in Artificial Intelligence re-emerged due to the commercial success of expert systems and the investment of companies in their development. The concept of Machine Learning also became more prominent as a branch of AI focused on developing algorithms that enable computers to learn patterns from data and predict outcomes. Methods such as multilayer neural networks and backpropagation appeared in the late 1980s, reviving research in neural networks. One of the most notable milestones was the victory of IBM's Deep Blue computer over world chess champion Garry Kasparov in 1997, which demonstrated the advanced potential of Artificial Intelligence in performing complex tasks.

The Launch of the Twenty-First Century (**2000s**): Artificial Intelligence entered the new millennium with remarkable advances in computing power and the availability of massive amounts of data, creating an ideal environment for the later growth of Deep Learning techniques. During the first decade of the century, commercial AI applications emerged in fields such as online recommendation systems, search engines, and big data analytics. Intelligent robots capable of interacting with their environment and making simple decisions also continued to develop.

The Deep Learning and Large Models Revolution: The year 2012 marked a major turning point with the success of deep neural network algorithms in the ImageNet image recognition competition, which demonstrated the power of Deep Learning. Multi-layered neural networks were developed with the ability to process images and texts with unprecedented efficiency. This was followed by the introduction of the Transformer architecture in 2017, which revolutionized Natural Language Processing (NLP) due to its ability to understand word context in a parallel and efficient manner. Large Language Models later relied on this architecture to achieve major advances in language understanding and generation. In 2018, Google introduced BERT, one of the first large-scale bidirectional models for language understanding. It was later followed by GPT-3, developed by OpenAI in 2020, with an unprecedented scale of 175 billion parameters, enabling it to generate coherent texts and perform various tasks without task-specific training. GPT-3 and its successor, GPT-4 in 2023, demonstrated remarkable capabilities in producing human-like text, bringing these models to the forefront of both public and academic interest. This stage has become known as the era of Large Language Models (LLMs), during which institutions have increasingly competed to build larger, more extensively trained models with stronger contextual understanding .[20]

The Latest Generation: **Retrieval-Augmented Generation** (**2020**): Despite the remarkable capabilities of Large Language Models, it became evident that they suffer from certain limitations, such as the restriction of their factual knowledge after a specific date and the possibility of fabricating information, a phenomenon known as hallucination. Therefore, researchers moved toward integrating information retrieval techniques with the language generation process, leading to what is known as Retrieval-Augmented Generation (RAG). In this new approach, which emerged around 2020, the language model is supplied with external information from a retrievable knowledge base during the process of generating responses. One of the pioneering examples of this approach is the RAG model developed by Lewis et al. (2020), which retrieves relevant passages from Wikipedia through semantic search and then incorporates them into the process of answering knowledge-based questions. This method enabled the model to access updated and accurate knowledge beyond its own internal "memory," thereby improving performance and providing greater credibility compared with standalone models. By 2023–2024, various applications based on Retrieval-Augmented Generation began to emerge, enabling language models to access institutional

knowledge repositories or databases during conversations. This helped overcome information limitations and provide sources for generated answers. This modern phase represents a culmination of the historical development of Artificial Intelligence, as it combines the power of large generative models with the ability to retrieve information and reason in real time.[21]

2.2.3 Importance in Cybersecurity

Artificial Intelligence technologies in general, and Large Language Models in particular, have become strategically important in the field of cybersecurity. They provide unprecedented capabilities for analyzing the massive and diverse volumes of data generated within the digital infrastructure of organizations, thereby helping security teams detect threats and respond to them more effectively. Modern cyberattacks are characterized by complexity and rapid evolution, making traditional tools based on static rules insufficient to keep pace with the constantly changing techniques of attackers [22]. In this context, Artificial Intelligence emerges as an adaptive solution. For example, Machine Learning models can analyze activity and network logs to detect anomalous patterns that may indicate a new attack, even if such patterns were not predefined. Similarly, Deep Learning algorithms enable the development of intrusion detection systems capable of identifying malware or intrusion attempts based on data patterns, rather than relying solely on previously known signatures.

As for Large Language Models (LLMs), they have demonstrated their ability to understand natural language context and generate intelligent responses, which has opened the door to their use in advanced cybersecurity applications. These models are capable of rapidly analyzing large volumes of security threat reports and vulnerability bulletins at a speed that exceeds human capabilities, while automatically extracting the most important information from them. They can also correlate fragmented pieces of information with one another. For example, an LLM can link a current security incident to a known attack pattern or to information contained in a previous intelligence report. Researchers indicate that LLMs provide a new approach to cyber defense due to their capabilities in analyzing complex patterns, predicting threats, and supporting responders in real time.[22]

Their importance is also evident in security incident response, where language models can be employed to generate procedural steps for handling a specific incident or to suggest best practices in real time. For example, recent research has shown that the use of models such as ChatGPT can

enhance the development and review of incident response plans by assisting analysts in preparing comprehensive plans and identifying gaps in documentation [5]. Thus, large models contribute to improving organizational readiness for dealing with cyberattacks through well-structured prior planning and AI-supported recommended actions.

Moreover, Large Language Models have been found to possess the capability to perform digital forensic analysis and understand system logs during incident investigations. Security analysts have already begun adopting LLM-based assistants to analyze log files, extract significant events, and interpret them in natural language, thereby reducing both time and effort. A field study involving security analysts in a Security Operations Center (SOC) indicated that the majority used the language model as an immediate tool for building understanding and context during investigations, without delegating sensitive decisions to it [23]. This demonstrates that Artificial Intelligence has become an intelligent assistant to the human analyst, enhancing their efficiency in understanding attacks and making timely decisions.

On the other hand, Retrieval-Augmented Generation (RAG) is of particular importance in cybersecurity because it helps bridge the knowledge gap inherent in generative models. By using RAG, the model can retrieve updated or sensitive information from a security database while analyzing a given threat, thereby providing an answer based on the latest available threat intelligence rather than relying solely on what it learned during training. This approach increases the reliability of the model's outputs and supports the provision of reference-based evidence for each analysis, which is critically important in a field where decisions must be based on accurate and up-to-date information.[24]

In summary, Artificial Intelligence has become a cornerstone of cybersecurity. It not only supports the automation of many demanding tasks, such as network monitoring and alert analysis, but also provides proactive analytical capabilities that help detect and anticipate unknown threats before they escalate. As the digital environment becomes increasingly complex, organizations are becoming more reliant on AI-supported solutions to strengthen their cyber defenses and build more adaptive and effective digital immune systems.

2.2.4 Types of Artificial Intelligence

Under the umbrella of Artificial Intelligence, there are multiple types and classifications that can be categorized from different perspectives. Among the most important classifications are those based on the scope of capability and the technical methodology.

1. **Based on the Scope of Capability**: **Types of Artificial Intelligence in Terms of Strength**

- **Narrow or Weak Artificial Intelligence**: This is the most widely used type of Artificial Intelligence today. It is specialized in a specific domain or a particular task. Such systems are intelligent only within their limited scope, such as a chess-playing program or a movie recommendation system. Most current AI applications fall under this category.
- **Artificial General Intelligence** (**AGI**): This refers to an Artificial Intelligence system with comprehensive cognitive capabilities comparable to general human intelligence. In other words, it would be able to understand and perform any intellectual task with a level of competence similar to that of a human. This type remains theoretical and has not yet been fully achieved in practice. It is also known as Strong Artificial Intelligence.
- **Artificial Superintelligence** (**ASI**): This is a future-oriented concept that assumes Artificial Intelligence will surpass human cognitive capabilities across all domains. This level remains more of a philosophical and ethical subject of debate than a current technical reality.

2. **Based on Methodologies and Technologies**:

- **Rule-Based Systems**: These are among the earliest types of Artificial Intelligence systems, in which the system operates according to manually programmed logical rules, typically in the form of "if–then" statements. This category includes expert systems, which were widely used during the 1980s. Such systems contain a knowledge base composed of IF–THEN rules that represent human expertise in a specific domain. Rule-based systems perform well in limited domains where the rules can be clearly defined, but they struggle with complexity or with exceptions that have not been explicitly programmed.
- **Machine Learning**: These are systems that learn from data. Instead of being programmed with fixed rules, a mathematical model is trained on large datasets in order to discover patterns

and relationships on its own. Machine Learning is further divided into several main categories:

  - **Supervised Learning**: In this approach, the system is provided with pre-labeled data, consisting of inputs and their correct outputs, so that it can learn the relationship between them. An example is training a spam email classifier using messages whose classifications are already known.
  - **Unsupervised Learning**: In this approach, the system is left to infer the underlying structures within unlabeled data on its own. An example is a clustering algorithm that identifies groups of similar customers within a company.
  - **Reinforcement Learning**: In this approach, an intelligent agent learns through trial and error within an interactive environment. It receives rewards or penalties based on its actions, such as training a robot to walk by reinforcing correct movements.
- **Deep Learning**: Deep Learning is a branch of Machine Learning based on multi-layer artificial neural networks. It is characterized by its ability to automatically extract features from raw data through numerous hidden layers that perform nonlinear operations. Deep Learning has proven highly effective in tasks such as computer vision, including image classification, and natural language processing, including machine translation. It also lies at the core of modern Large Language Models, which are deep neural networks trained on massive volumes of textual data.
- **Fuzzy Logic**: Fuzzy Logic is an approach that enables systems to handle uncertainty through multi-valued logic rather than strict binary logic.
- **Evolutionary Algorithms and Swarm Intelligence**: These include genetic algorithms inspired by natural selection, swarm intelligence inspired by the behavior of ants, birds, and other collective organisms, among others. These methodologies rely on finding solutions to problems by simulating natural and evolutionary processes.

All of these types collectively contribute to forming the comprehensive landscape of Artificial Intelligence. For example, an intelligent system may combine manually designed rules with a Machine Learning model, or it may use an evolutionary algorithm to optimize the weights of a neural network, as in neural network tuning. Large Language Models, on the other hand, represent an advanced form of supervised Deep Learning, as they learn from massive textual datasets in a largely

unsupervised manner and are then refined through instruction tuning and Reinforcement Learning from Human Feedback (RLHF), enabling them to follow instructions effectively.

2.2.5 Core Components of Artificial Intelligence

Artificial Intelligence systems, despite their diversity, rely on a set of fundamental components that they share to varying degrees. The most important of these components include:

Data: Data is the fuel that drives Artificial Intelligence systems. An intelligent system learns patterns by analyzing the training data available to it. The more abundant, clean, and diverse the data is, the more accurate and reliable the model's performance becomes. Data may include images, texts, numerical values, videos, and other forms, depending on the application domain. For example, a Large Language Model such as GPT-4 was trained on hundreds of billions of words collected from sources such as the Internet, encyclopedias, and books. Similarly, security SIEM systems, which will be discussed in the following section, rely primarily on collecting event logs from various devices as a fundamental source of data.

Algorithms and Models: Algorithms and models represent the core component responsible for processing data and extracting meaningful results. A mathematical model or algorithm defines the method by which data is handled, analyzed, and transformed into outputs.

Computational Processing Power: Artificial Intelligence models, particularly deep models, require substantial computational capacity to perform millions or even billions of mathematical operations. Therefore, hardware is considered a fundamental component of AI systems. The development of Graphics Processing Units (GPUs) has played a major role in accelerating training and inference processes in neural networks.

Knowledge or Knowledge Base: Some Artificial Intelligence systems include an explicitly stored knowledge base, particularly expert systems and question-answering systems. This knowledge base may consist of a set of logical rules or a network of facts and relationships, as in semantic knowledge bases.[21]

Interaction Interface: For Artificial Intelligence to perform its role effectively, there must be an interface that connects it either to the user or to the system within which it operates. This interface may take the form of a Graphical User Interface (GUI) that displays the system's recommendations and analysis results, or an Application Programming Interface (API) that enables other systems to send data to the intelligent system and receive outputs from it.

Evaluation and Feedback Mechanisms: To ensure the effectiveness of Artificial Intelligence, performance evaluation processes and feedback mechanisms are incorporated to improve system outputs. During development, data is typically divided into training and evaluation sets in order to test the model's performance. Some systems also continue to learn during operation through Online Learning, where they benefit from human feedback or subsequent outcomes to enhance their performance over time.

Figure 2.2 illustrates these components.

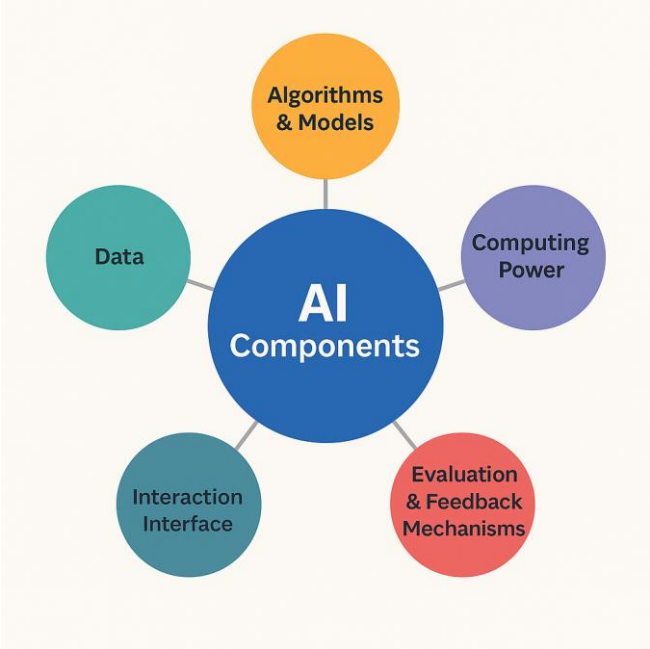


Figure 2.2: Components of Artificial Intelligence.

2.2.6 Real-World Applications of Artificial Intelligence

Artificial Intelligence, with its various technologies, has witnessed widespread adoption in real-world applications that affect people's daily lives and transform numerous industries. The following are some of the most prominent real-world application domains:

Intelligent Personal Assistants: Applications such as Apple's Siri, Google Assistant, and Amazon's Alexa have become iconic examples of the application of Artificial Intelligence in natural language understanding and voice-based interaction. These systems use speech recognition models to convert spoken language into text, followed by Natural Language Processing (NLP) algorithms to analyze the user's intent and generate an appropriate response, and finally text-to-speech technologies to convert the response into audible speech.

Social Media and Entertainment: Artificial Intelligence algorithms are employed in social media platforms to rank content and display posts that are most relevant to each user.

Healthcare: Artificial Intelligence is used in disease diagnosis through the analysis of X-ray images and medical scans using pattern recognition algorithms.

Financial Sector: Banks and financial institutions rely on Artificial Intelligence systems for the early detection of fraud by identifying anomalous financial activities, such as unusual account transactions, using classification and anomaly detection techniques.

Industry and the Internet of Things (**IoT**): Smart factories use AI-based control and prediction systems for predictive maintenance, enabling the anticipation of machine failures before they occur through the analysis of sensor data. Intelligent robots are also employed in production lines to perform precise tasks that require vision capabilities and adaptation to environmental changes.

2.2.7 Ethical Implications

Despite the significant benefits of Artificial Intelligence, it also raises a range of ethical and societal challenges that require careful study and responsible management. The most prominent ethical concerns can be summarized as follows:

Fairness and Bias: Artificial Intelligence systems rely on data to learn and make decisions. If the data contains human biases or an unbalanced representation of social groups, the model may reproduce these biases in its outputs.[25]

Privacy and Data Protection: Artificial Intelligence applications collect vast amounts of personal information in order to infer patterns and predict behavior. This raises concerns regarding how such data is used and who has the right to access it.[25]

Accountability: When an important decision is made by an Artificial Intelligence system, such as approving a loan application or diagnosing a disease, a critical question arises: who bears responsibility if the decision is incorrect or unfairly harms an individual? Is it the company that developed the system, the user who relied on it, or the system itself, which is not a legal entity? The lack of clarity in this regard creates what is known as the accountability gap.[25]

Impact on Society and Humanity in General: There are broader ethical and philosophical debates concerning the future of humanity in the presence of Artificial Intelligence that may eventually become general or superintelligent. Some thinkers are concerned about the potential loss of human control over highly intelligent systems that may become difficult to stop if they begin to act in undesirable ways. Questions are also raised regarding consciousness and identity: could a highly advanced machine possess consciousness or rights that must be taken into consideration? These questions extend beyond the technical dimension and reach deep ethical issues concerning the relationship between humans and machines.[25]

2.2.8 Conclusion

In conclusion, Artificial Intelligence is a rapidly evolving and constantly changing field, with future prospects that appear broad and rich with both opportunities and challenges. Continuous advancements in Deep Learning capabilities and the emergence of hybrid approaches that combine learning and reasoning, such as RAG, indicate a clear shift toward smarter and more reliable systems. However, there remains a critical need to deal with these technologies wisely and responsibly in order to ensure their alignment with human interests. In the following sections, the discussion will

examine in greater depth one of the important practical applications of Artificial Intelligence in cybersecurity, namely Security Information and Event Management (SIEM) systems, while exploring the role of Artificial Intelligence, including Large Language Models, in developing these platforms and enhancing their capabilities.

2.3 Comprehensive Overview of Security Information and Event Management (SIEM) Systems

2.3.1 Definition

Security Information and Event Management systems, commonly abbreviated as SIEM systems, are specialized software solutions that combine two main processes in the field of information protection: Security Information Management (SIM) and Security Event Management (SEM) [16]. The role of a SIEM system can be summarized as collecting security data from multiple sources, such as server logs, network devices, traffic flows, and various security applications, and then analyzing this data in real time or near real time in order to detect events or conditions that may constitute a security threat to the organization.

In simple terms, a SIEM system functions as a vigilant monitoring mechanism that continuously observes network activities and an organization's technical infrastructure in search of any indicators that may suggest a security breach or a violation of an established policy. When such indicators are detected, the system generates alerts for security teams to notify them of a potential issue. Moreover, SIEM facilitates post-incident investigation by enabling analysts to review the sequence of events across multiple systems in order to trace the attacker's actions and their impact.

2.3.2 Role of SIEM in Security

A SIEM system plays a central role in the information security infrastructure of any organization. This role can be summarized through several key points:

Centralized Monitoring: Before the emergence of SIEM systems, security teams relied on separate tools to monitor different parts of the network, such as one tool for operating system logs and another for firewall logs. With SIEM, however, all of these logs can be collected within a single centralized platform.

Early Incident Detection: Through correlation and pattern analysis, a SIEM system can detect complex attacks that occur in fragmented stages across multiple systems. For example, an attacker may perform several failed login attempts on multiple servers, eventually succeed in gaining access, and then execute a suspicious command. A SIEM system is capable of correlating the repeated login attempts across different servers with the command executed afterward, interpreting them as an interconnected sequence that may indicate a password-guessing attack or privilege escalation, and generating an alert accordingly.

Incident Response and Management: When a security incident occurs, the SIEM system serves as a primary tool for analysts to understand what happened. It provides digital forensic investigation capabilities through logs and enables the chronological tracking of events. For example, analysts can search within the SIEM for all actions associated with a specific IP address or user during a defined time period.

Compliance and Regulatory Adherence: Many organizations are subject to compliance requirements related to security laws and standards, such as the PCI DSS standard for protecting payment card data or the GDPR regulations for personal data protection. SIEM reports help demonstrate compliance with such standards by providing detailed and structured records of all activities occurring within the network.[16]

Proactive Improvement of the Security Posture: The role of SIEM is not limited to detecting incidents after they occur; it also contributes to incident prevention by providing security teams with visibility into vulnerabilities and unusual activities. For example, if SIEM detects a significant increase in scanning attempts targeting a specific server, the security team can take proactive measures, such as strengthening monitoring of that server or patching a potential vulnerability. Similarly, by reviewing periodic SIEM reports, organizations can identify attack trends and the most common types of threats they face, thereby improving their security policies accordingly.

2.3.3 Historical Overview of SIEM Systems

The concept of combining **Security Information Management** (**SIM**) and **Security Event Management** (**SEM**) within a single platform emerged in the early 2000s. Prior to 2005, the concepts of SIM and SEM existed separately:

Security Information Management (**SIM**) **Systems**: These systems emerged in the late 1990s and early 2000s as tools for the long-term storage of event logs within a centralized repository, with the ability to perform historical analysis and generate periodic reports [16]. They focused more on archiving and reporting than on real-time monitoring. Their primary role was to help organizations maintain comprehensive records of activities for the purposes of regulatory compliance and subsequent forensic investigations.

Security Event Management (**SEM**) **Systems**: These systems also emerged during the same period, but with a different role. They were concerned with the real-time aggregation of events and the immediate generation of alerts when suspicious activity occurred. In other words, SEM was designed to handle the current stream of events by filtering active logs and comparing them against known rules and patterns in order to generate instant security alerts. SEM systems were characterized by capabilities such as normalization and simple real-time correlation to identify security issues requiring urgent intervention. In this sense, SEM can be described as an early warning system.[16]

By the mid-2000s, security experts recognized that integrating these two functions would provide significant value. At this point, the research firm Gartner introduced the term "SIEM" in 2005 to unify these two concepts. Gartner described SIEM at the time as an integrated solution that performs the functions of SIM, including archiving and historical analysis, in addition to the functions of SEM, such as real-time monitoring and alerting. Thus, SIEM combines the strengths of both approaches: comprehensive historical visibility and real-time response. This development led to the emergence of a new industry and triggered competition among technology companies to develop SIEM products.[16]

The Second Half of the 2000s: The first SIEM products began to emerge from security companies such as ArcSight, IBM through the Q1 Labs QRadar platform, and others. These early systems can be referred to as the first generation of SIEM (SIEM 1.0). They were characterized as log-centric

systems and relied heavily on fixed knowledge-based correlation rules to detect threats. They also depended on traditional relational databases for data storage, which imposed performance limitations as data volumes increased. This generation faced difficulties in processing massive streams of events in real time while retaining a sufficient level of detail. For example, correlation rules often focused on linking events based on IP addresses or ports. However, in a modern environment with mobile users and dynamic IP addresses, such correlations become limited in usefulness. Despite these limitations, SIEM 1.0 represented a significant technological shift at the time, as it provided, for the first time, the ability to observe an emerging threat across multiple layers of the digital infrastructure .[16]

Late 2000s–Early 2010s: The first generation of SIEM faced several challenges as threats became more complex and advanced attacks emerged, requiring more dynamic and context-aware analysis. In addition, the rise of Big Data led to a massive increase in the volume of logs that needed to be stored and analyzed, exposing the limitations of relying solely on relational databases. As a result, the second generation of SIEM, known as SIEM 2.0, began to develop. This generation can be described as more intelligent and context-aware. During the 2010s, SIEM 2.0 introduced several improvements, including the following:[16]

Integration of Behavioral and Contextual Analytics: In this approach, the system does not merely match individual events against static rules. Instead, it examines the broader context of the event, such as who the user is, the nature of the device involved, and the related previous activities, in order to determine the level of risk associated with the event.[16]

Full Tracking of the Incident Lifecycle: SIEM 2.0 came to include not only incident detection, but also the management of the incident from beginning to end. This includes documenting the different phases of the incident and the response steps, as well as linking events related to a single incident into one sequence, sometimes referred to as a case or an incident within the system. The ability to execute automated actions as part of the response process was also added.[16]

Automated Reports and Recommendations: SIEM systems evolved to provide analysts with recommendations on how to handle specific incidents, based on embedded knowledge or even

through integration with external knowledge bases. They also generate summarized reports for senior management regarding the overall security posture and the effectiveness of protection efforts .[16]

Big Data Technologies: To overcome the performance limitations of the first generation, the second generation adopted more advanced storage and processing infrastructures. NoSQL databases, distributed search engines, and parallel processing technologies were used to accelerate the analysis of massive volumes of logs. Some systems also began relying on in-memory storage or column-oriented databases, which enabled significantly faster analysis compared with the previous reliance on traditional SQL databases. Concepts of distributed computing were also introduced, allowing SIEM systems to distribute processing workloads across multiple server nodes.[16]

Integration of External Threat Intelligence Sources: This capability was not common in the early stages of SIEM development. Modern SIEM systems have become capable of importing threat feeds from external intelligence sources, such as known malicious IP addresses and new malware signatures, and incorporating them into the analysis engine. This has provided broader visibility that extends beyond the organization's internal data to include the latest global threat intelligence, thereby enhancing the ability of SIEM systems to identify activities associated with known attack campaigns.[16]

It can be said that the modern history of SIEM represents a transition from a mere log management and storage tool into an advanced security analytics platform supported by Artificial Intelligence. However, this evolution has not ended there. As will be discussed in the section on future prospects, entirely new technologies—such as Machine Learning and Large Language Models—have entered the field, pushing the boundaries of what SIEM systems can achieve in the coming years.

2.3.4 Core Concepts

SIEM systems rely on several key concepts that form the theoretical and technical foundation of their capabilities. These concepts include:

- **Log**: A log is the most basic data unit handled by a SIEM system. Logs are messages or events generated by different technological systems to document what is occurring within them. A single log may represent a security event, such as a login attempt, a change in system configuration, or an alert generated by antivirus software. A SIEM system is distinguished by its ability to process massive volumes of logs arriving at high speed from diverse sources.
- **Aggregation**: Due to the enormous number of logs, a SIEM system often performs an aggregation process in which a large number of similar logs are merged into a single summarized event. For example, if a user attempts to log in unsuccessfully 100 consecutive times, the system may display this as one aggregated event indicating 100 failed login attempts, rather than presenting 100 separate log entries.[16]
- **Normalization**: Normalization is the process of converting different log formats into a unified format within the system. Since each system, such as Windows, Linux, Cisco routers, firewalls, and others, generates logs in a different format, the SIEM performs parsing and extracts important fields, such as the source IP address, username, event type, and other relevant attributes. It then represents them using shared standard fields. In this way, the system can interpret "User Login Failed" from a Windows server and "Authentication Failure" from a network router as two events belonging to the same general category, namely login failure, regardless of the differences in the original message text.[16]
- **Correlation**: Correlation is considered the cornerstone of SIEM systems. It refers to the logical linking of two or more events in order to derive a security-related conclusion. Correlation is performed either according to rules defined by security administrators or through Machine Learning algorithms in advanced systems. An example of a correlation rule would be: "If more than five failed login attempts occur for a specific user within ten minutes, followed by a successful login by the same user from an unfamiliar device, then generate an account compromise alert." In this case, multiple events are linked through temporal and logical conditions. Correlation helps detect sequences that may appear normal when each event is viewed individually, but become dangerous when the information is analyzed collectively.[16]
- **Context**: Introducing contextual information makes analysis more accurate and intelligent. Context refers to additional information about the elements mentioned in logs. For example, asset lists may contain details about each device, such as its type, location, and level of

importance, while user lists may include users' roles and privileges. When a SIEM system knows that a particular server is sensitive because it contains a financial database, for instance, it will assign higher importance to events associated with that server. Similarly, if a certain user account is known to be a highly privileged administrator account, a failed login attempt targeting that account will be considered more concerning than a failed login attempt involving an ordinary employee account. Enriching events with contextual information helps reduce noise and highlight critical issues.[16]

- **Baseline and Anomalous Behavior**: Many modern SIEM systems adopt the concept of learning a baseline for network and user behavior, and then detecting any significant deviation from it. For example, the system may learn that a user named Ahmad usually connects from the company office between 9:00 a.m. and 5:00 p.m. If a VPN session for Ahmad appears from another country at midnight, the SIEM system identifies this as anomalous activity. This concept requires the application of User and Entity Behavior Analytics (UEBA), which has become integrated into modern generations of SIEM systems. It represents a shift from static rules toward statistical modeling of normal behavior, followed by measuring the degree of anomaly. This helps detect insider threats or compromises that evade known signature-based patterns.
- **Alerting and Escalation**: When the conditions of a correlation rule are met, or when suspicious behavior exceeds a defined threshold, the SIEM system generates a security alert. Each alert is accompanied by a severity level, such as high, medium, or low, based on predefined configurations or contextual analysis. Escalation policies are often defined so that, when a high-severity alert is generated, the incident response team is notified immediately and certain response actions may be activated. Alerts represent the primary outputs that SOC analysts deal with on a daily basis, and the effectiveness of their work depends heavily on the quality of these alerts, particularly whether they are truly significant and not false positives.
- **Incident Management**: Incident management involves tracking the status of each incident after an alert has been generated. A SIEM system, or the systems integrated with it, provides an interface for recording the analyst's investigation notes, remediation actions, and final closure of incidents after they have been resolved. This process creates a useful historical record that can support future analysis, reporting, and continuous improvement.

2.3.5 Components of SIEM Systems

A SIEM system, as an integrated software solution, consists of several core components that work together in harmony to perform its intended functions. The main components and the typical architectural structure of a SIEM system can be detailed as follows:

Data Collectors/Agents: These are software components or services responsible for extracting logs from different systems and sending them to the SIEM platform. They may be built into the source systems themselves, such as the ability of Linux servers to send logs through the Syslog protocol, or they may take the form of dedicated client software installed on the servers and applications that need to be monitored. The main function of these agents is to capture important events as soon as they occur and transmit them in real time or near real time to the central SIEM engine. In some cases, a SIEM system may operate without dedicated agents and instead rely on standard logging protocols, such as Syslog, SNMP, and Windows Event Forwarding.

Transmission Channel: The agents transmit data over the network to the SIEM servers. This channel must be both secure and reliable. Therefore, the transmitted data is often encrypted and protected to ensure that attackers cannot intercept, modify, or manipulate it during transmission.

Ingestion and Processing Engine: When logs arrive at the central SIEM system, they are received by the ingestion component. At this stage, the data is first normalized, as previously explained, by converting it into a unified format [16]. Filtering policies may also be applied, such as ignoring certain low-value events to save storage space, along with aggregation processes that merge repetitive events. Afterward, the normalized events are passed to the correlation and analysis engine. This engine represents the core intelligence of the SIEM system, as it applies correlation rules defined by analysts and may also perform behavioral analytics or anomaly detection algorithms if supported. Any event or sequence of events that matches an alert condition results in the generation of an alert record, which is then forwarded to the alert management component. In modern systems, this engine may be distributed across multiple nodes to process events in parallel, due to the massive volume of data involved.

Data Storage: A SIEM system needs to store massive volumes of data for extended periods, sometimes for one year or more due to regulatory requirements. Therefore, it includes a storage component capable of preserving filtered and selected data. In first-generation SIEM systems, this storage component was often based on a relational database. Modern SIEM systems, however, commonly rely on a combination of Big Data technologies, such as Apache Hadoop, Elasticsearch, and other distributed storage and search platforms.[16]

Alert and Incident Management Module: This component receives the alerts generated by the analytical engine and presents them to analysts in the form of security event lists. It typically supports capabilities such as grouping related alerts into a single incident, assigning the incident to a specific analyst, and recording the incident status, such as under investigation, resolved, false positive, and so on. It also allows analysts to add comments and document the remediation steps taken.

User Interface (**Dashboard**/**UI**): The user interface is the layer through which security analysts and administrators interact with the SIEM system. It is usually a web-based interface that enables users to view dashboards containing statistical summaries, such as the number of alerts by type, the most targeted assets, and suspicious activities by geographic source. In addition, the interface provides detailed views for reviewing alerts and their associated logs. It also allows users to create custom queries to search the log archive, such as searching for all login events related to a specific user within a defined time period, and to generate customized reports in different formats.

Administration and Configuration: This component enables system administrators to configure and manage the SIEM system. This includes adding new data sources, such as defining a new device whose logs need to be collected, modifying correlation rules, configuring data retention schedules, and managing the permissions of users who access the system. Administration also includes monitoring the health of the SIEM system itself, such as verifying whether data is being received smoothly, detecting any delay or loss in logs, and notifying administrators in the event of a technical issue.

Integrations: A SIEM system often includes integration modules with other systems. These may include integration with Identity Management Systems to retrieve user information and permissions, integration with antivirus platforms to receive alerts directly from them, or integration with firewalls to automatically send IP-blocking commands as part of incident response.

Figure 2.3 illustrates these components.

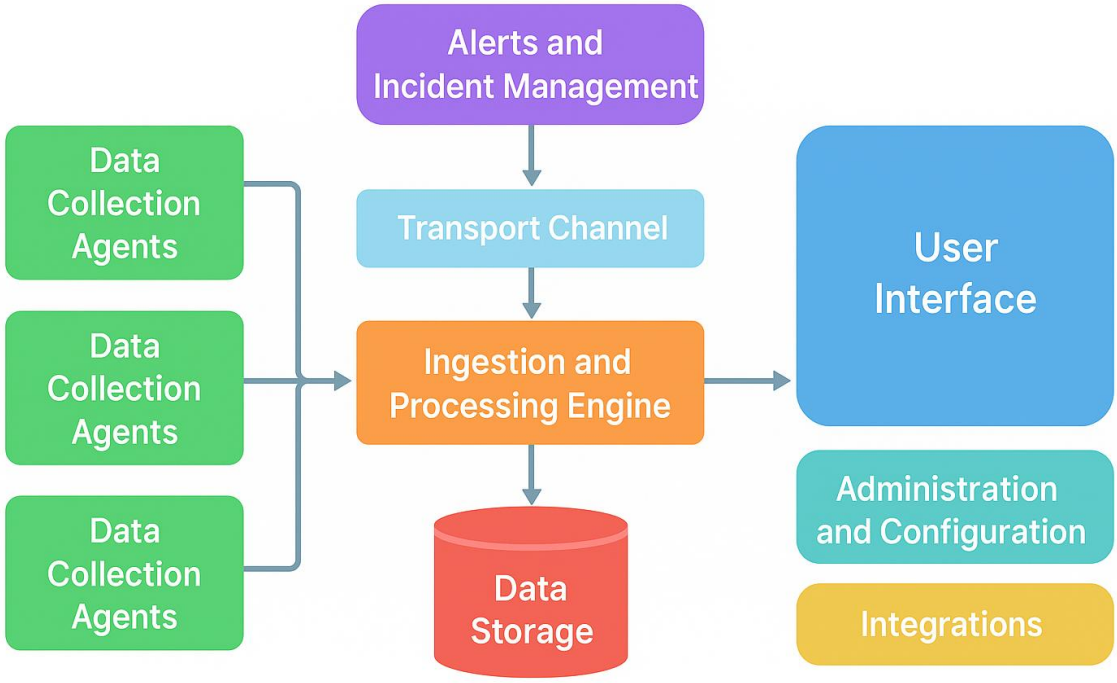


Figure 2.3 : Components of SIEM Systems.

2.3.6 Future Prospects of SIEM Systems

As the threat landscape continues to evolve and technical infrastructures become increasingly complex, experts are turning their attention toward the next generation of SIEM systems, which are expected to be more intelligent and more reliant on modern Artificial Intelligence technologies, including Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG). The following are some future trends and the ways in which they may shape the development of SIEM systems in the coming years:

AI-Driven Analytics: Machine Learning and Deep Learning algorithms are expected to be integrated more deeply into the SIEM analysis engine to detect hidden and complex threats. This

trend has already begun through the adoption of User and Entity Behavior Analytics (UEBA), as previously mentioned. However, the future will witness greater maturity in these algorithms and their ability to learn continuously from the environment. As a result, SIEM systems will become capable of building predictive models that anticipate where the next attack may occur based on event patterns. They may also assign a "compromise probability score" to different assets within the network, helping security teams focus their preventive efforts more effectively.

Integration of Large Language Models into SIEM Platforms: Imagine an Artificial Intelligence assistant integrated into a SIEM platform, where users can ask questions in natural language and the system converts them into complex data queries before returning the answer. This capability may soon become achievable through the integration of Large Language Models (LLMs), such as GPT-4 and similar models, into SIEM interfaces. In this case, even a junior security analyst could ask the system, for example: "Show me all suspicious activities related to Server X during the past week," and receive an intelligent summary instead of writing complex manual queries. This capability may even extend to analytical questions such as: "What is the cause of this alert?" In such a scenario, the language model would examine the event and its related logs, then provide a clear textual explanation based on established security rules and knowledge.[5]

Expansion of Integrated Data Sources: Future SIEM systems are expected to process a broader range of data sources, including activity data from cloud services, data from Operational Technology (OT) networks and industrial control systems, and even information from social media platforms or the open web when relevant to the organization, such as detecting leaked corporate data on the dark web. This diversification of sources will generate massive volumes of security-related data, thereby increasing the need for advanced Artificial Intelligence techniques to synthesize and interpret it effectively. In this context, Retrieval-Augmented Generation (RAG) may play an important role. Instead of storing and processing all of these diverse data sources locally, a SIEM system may rely on a language model capable of querying external sources in real time when needed [24]. For example, during an incident investigation, the SIEM system may automatically retrieve information from a global threat intelligence database through RAG in order to determine whether a suspicious IP address involved in the incident is already known to be associated with malicious activity.

Toward the Concept of an Autonomous Security Operations Center (Autonomous SOC): This trend involves minimizing human intervention in routine tasks, allowing SIEM systems, when integrated with SOAR platforms and Artificial Intelligence, to automatically handle incidents in many cases. Several steps contribute to achieving this goal, including the adoption of intelligent agent capabilities, where smart data-collection agents can execute local response actions when they detect dangerous activity, such as temporarily isolating a device from the network.[5]

Reducing False Positives through Improved Intelligence: One of the greatest current challenges facing security teams is the high volume of false or low-priority alerts. The future may offer solutions to this problem through Artificial Intelligence systems capable of reviewing alerts before presenting them to human analysts. For example, a Large Language Model could examine an alert log along with the context of preceding and subsequent events, and then provide a recommendation on whether the alert requires immediate investigation or can be safely deprioritized. The system may even automatically close alerts that it determines, with a high level of confidence, to be insignificant, similar to the judgment of an experienced analyst. This can be achieved by learning from previous analyst decisions through feedback-based model improvement. As a result, routine workload is reduced, allowing analysts to focus their efforts on truly important incidents.

2.3.7 Conclusion

Maximizing the benefits of SIEM systems, both in the present and in the future, requires a comprehensive approach. From a technical perspective, this involves deploying, tuning, and continuously updating these systems to keep pace with the latest threats. From a human perspective, it requires training analysts to use them effectively and integrating them efficiently into daily operational procedures. A SIEM system is not merely a technical product; rather, it is part of a broader security governance ecosystem within organizations. With the increasing integration of Artificial Intelligence into this field, it has become more important than ever to combine human expertise with machine capabilities in order to achieve a more secure and resilient digital environment capable of withstanding cyberattacks.

3. Chapter Two: Work Environment

3.1 General Introduction

Selecting the appropriate technical tools and work environment is a fundamental factor in the success of any advanced project that relies on Artificial Intelligence, particularly in a sensitive field such as cybersecurity. This chapter presents the components of the work environment prepared for the project, justifying the selected programming language, virtualization platform, operating systems, libraries, and Artificial Intelligence algorithms adopted in the development of the proposed solution.

3.2 Python Programming Language

The programming language represents one of the most important pillars of the work environment, as it is the tool through which the system logic is expressed and the required algorithms are implemented. In this project, Python was selected as the primary programming language. This choice is based on several fundamental reasons related to Python's suitability for this type of project:

- **Ease of Learning and Clear Structure**: Python is characterized by a simple and readable syntax, which makes the development process faster and less prone to errors.
- **Widespread Use in Cybersecurity and Data Analysis**: Python is widely used in the fields of data analysis and cybersecurity tool development. Many security testing and auditing tools, as well as system log analysis processes commonly used in SIEM platforms, support Python or provide APIs compatible with it. This widespread adoption ensures the availability of a strong developer community and a rich ecosystem of ready-made solutions for common technical challenges.
- **Strong Support for Artificial Intelligence and Machine Learning Libraries**: Python provides a large number of specialized libraries for Artificial Intelligence and Machine Learning, such as PyTorch, TensorFlow, scikit-learn, and others. Since this project integrates LLM and RAG technologies, Python offers ready-to-use libraries for working with language models, processing textual data, and building vector-based knowledge bases with ease.
- **Integration with Wazuh and Its Interfaces**: Wazuh provides REST APIs that can be accessed over the network to retrieve security data, including alerts and logs, from the server. In this context, Python plays an important role due to the availability of high-level HTTP

libraries, such as requests, which facilitate communication with the Wazuh APIs and enable the retrieval of information for further processing.

Despite these advantages, it should be noted that Python is slower in execution, in terms of processing speed, compared with some languages such as C++ or even Java, especially when handling large volumes of data. It may also consume more memory when running large models or performing computationally intensive operations. Nevertheless, within the context of this project, these limitations did not constitute a major obstacle. The development priorities were rapid implementation, ease of experimentation, and strong support for Artificial Intelligence libraries—areas in which Python clearly outperforms many other programming languages.

For comparison, languages such as Java were not selected for this project because the objective was not to build a high-performance commercial production system, but rather to develop a prototype quickly and flexibly in order to test the proposed concept. Therefore, Python was chosen as the most suitable language for conducting the experiments and building the intended integration between the Wazuh platform and Artificial Intelligence technologies.

To install Python and pip, and to download the required libraries, the following commands are used:

```
i77@i77-server:~$ sudo apt install python3.10
i77@i77-server:~$ sudo apt install python3-pip
```

3.3 Setting Up the Virtual Environment Using VirtualBox

In implementing the project, we relied on building a Virtual Lab that simulates a small-scale real network environment in which the Wazuh platform and its surrounding components operate, including agents installed on target systems and an attacker machine. To achieve this, VirtualBox was used to create and run multiple Virtual Machines (VMs) on the same host computer. VirtualBox enables the simulation of devices running different operating systems and allows communication

between them according to predefined network configurations. This provided a closed and secure laboratory environment for conducting experiments without affecting a real network or being exposed to external risks.

Internal Network Connection (**NAT Network**): VirtualBox was configured using a dedicated internal NAT Network that connects all virtual machines participating in the project. **The Network Address Translation** (**NAT**) mode provides an isolated network through which the virtual machines can communicate with one another, while also allowing them to share the host machine's Internet connection when needed for downloading updates or packages. The virtual network was named and configured to assign each machine a fixed internal IP address within a defined range, such as **10.0.2.x**. This ensures that the Wazuh server and the other machines can communicate directly. This configuration allowed us to simulate a realistic network environment, where the client machine, or agent, sends its data to the Wazuh server through an internal IP address, while the attacker machine can also direct malicious traffic toward other devices within the same network. The NAT Network configuration provides an important balance in this context: it completely isolates the lab from the real production network to ensure safety, while still allowing free internal communication between the virtual machines as if they were part of a single local network.

Components of the Virtual Environment: The architecture we prepared consisted of four main virtual machines, each assigned a specific role within the overall system, as follows:

1. **Wazuh Manager Server**: This is a virtual machine running Ubuntu Server 2024. The Wazuh platform, version 4.14.0, was installed on this server to function as the main SIEM server, responsible for receiving log data from agents, correlating events, and generating alerts. Appropriate resources, such as RAM and virtual processors, were allocated to this server to ensure that Wazuh operates efficiently, particularly since log indexing and analysis processes can be resource-intensive.

2. **Windows 7 Operating System** (**Wazuh Agent**): This is a secondary virtual machine running Windows 7 Ultimate 32-bit, on which the Wazuh Agent for Windows was

installed. The purpose of including a relatively old Windows platform is to simulate the presence of an organizational workstation or server operating within the monitored network. Although Windows 7 is an outdated operating system, it still represents a useful model for systems that may be exposed to attacks, such as malware targeting legacy vulnerabilities or attacks exploiting services and ports such as SMB. The Wazuh Agent installed on this machine monitors Windows system logs, such as Event Logs, detects any suspicious or rule-violating activity, and continuously sends this data to the Wazuh Manager Server for analysis and alert generation.

3. **Ubuntu Desktop Operating System** (**Wazuh Agent**): The third virtual machine runs Ubuntu Desktop 2024 with the Wazuh Agent for Linux installed. This machine represents another workstation or server within the network, but operating in a Linux desktop environment. The purpose of including this diversity, represented by both Windows and Linux systems, is to verify the ability of the Wazuh platform to handle multiple operating systems simultaneously, which is one of Wazuh's advantages, as it supports agents across different platforms. The Wazuh Agent on Ubuntu collects system logs, such as authentication logs related to login activities and general system logs. It also enables features such as File Integrity Monitoring (FIM) and process execution monitoring. All collected data is then sent to the central Wazuh Manager Server.

4. **Attacker Machine** (**Attack Machine – Parrot OS**): The fourth virtual machine was dedicated to functioning as an attack platform used by the testing team to simulate attacker behavior. Parrot Security OS 2024 was selected for this machine. Parrot is a Linux distribution specialized in security and offensive operations, similar to the well-known Kali Linux distribution. However, it is generally more lightweight and places additional emphasis on privacy and encryption tools alongside penetration testing utilities. Parrot was preferred over Kali due to its lower resource consumption and greater stability in virtualized environments. This makes it suitable for academic and research purposes, where a flexible and customizable environment is required without excessive consumption of the host machine's resources.

5. **Host Machine – Windows 11**: In addition to the virtual machines, the host machine running Windows 11 was used outside the VirtualBox environment to run the Ollama tool and the llama3.2:latest large language model. This decision was made in order to take advantage of the GPU available on the host machine, since running large language models inside a virtual machine may result in noticeable response delays or high resource consumption. Accordingly, the virtual machines were responsible for simulating the network environment, attacks, and log collection, while the host machine was responsible for running the large language model and processing analysis requests received from the RAG system through the Ollama interface.

Once these components were configured and operated together, a semi-integrated environment was established. This environment consisted of a central server responsible for receiving data, represented by the Wazuh Manager, two monitored hosts, represented by the Wazuh Agent on both Windows and Linux, an attacker machine used to execute attack scenarios, and a host machine responsible for running the Large Language Model in order to utilize the available GPU capabilities. As shown in **Table 3.1**, these components were integrated through the virtual network, as previously explained.

Table 3.1 : Description of the Environment Components

IP Address	Operating System	Role	Device
10.0.2.7	Ubuntu Server 22.04 LTS (64-bit)	The central intelligence of the platform, responsible for receiving and analyzing logs from the agents.	Wazuh manager
10.0.2.10	Windows 7 Ultimate (32-bit)	Represents a targeted endpoint on the network and sends logs to the Wazuh Manager.	Windows Agent
10.0.2.15	Ubuntu Desktop 22.04 LTS (64-bit)	Represents a targeted endpoint on the network and sends logs to the Wazuh Manager.	Ubuntu Agent
10.0.2.9	Parrot Security OS	A machine used to launch attacks against network endpoints in order to test the platform's effectiveness in detecting intrusions.	Attacker
192.168.56.1	Windows 11	Responsible for running the Large Language Model (LLM) and processing analysis requests received from the RAG system through the Ollama interface.	Ollama Host

3.4 Libraries and Artificial Intelligence Technologies Used

The following section presents the main libraries and technologies used in the development of the proposed solution, along with the role of each component:

- **sentence_transformers Library**: This is an open-source Python library that enables the easy use of text embedding models. It was used to load the **BAAI/bge-small-en-v1.5** model and convert security-related texts, whether knowledge base passages or security logs received from **Wazuh**, into numerical vectors that represent the semantic meaning of the text. These vectors form the foundation of the vector search process within the **Qdrant database**.

- **qdrant_client Library**: This is the official library for interacting with the **Qdrant vector database**. It was used to create collections, insert vectors into the database, and perform searches for the most similar passages to the security log being analyzed. This library enables vector queries to be sent to Qdrant and retrieves the most semantically similar results, which represents a fundamental step in the operation of the **RAG** system.

- **Data Handling Libraries**: The json and orjson libraries were used to process data in JSON format, which is a fundamental format for Wazuh logs and model outputs. The **orjson** library provides higher performance in JSON reading and writing operations compared with the standard library, which is beneficial when handling large numbers of logs. In addition, **pandas** and **openpyxl** were used to read tabular files and convert their contents into processable data when preparing knowledge sources.

- **Text Processing and Validation Libraries**: The re library was used to search within texts using regular expressions and to extract important fields from logs. The **ipaddress** library was also used to validate IP addresses and handle them in a structured manner. Furthermore, **hashlib** was used to generate a unique fingerprint for each log, preventing the same log from

being analyzed more than once within a specific time period, thereby reducing repetition and noise in the system.

- **System and File Management Libraries**: The **os**, **pathlib**, **shutil**, **subprocess**, and **sys** libraries were used to interact with the file system, execute external commands when necessary, manage file paths, and run supporting processes. The **argparse** library was also used to facilitate running scripts from the command line and passing configuration options to them. In addition, **typing** and **future** were used to improve code readability and support modern data type definitions.

- **Communication and Integration Libraries**: The **requests** library was used to communicate with **HTTP** interfaces, whether when interacting with **Ollama** or with certain system services. The socket library was also used to send final alerts to Wazuh through the TCP protocol. The importance of socket lies in its ability to inject the alert generated by the RAG system directly into Wazuh, allowing it to be captured by custom rules and displayed within the graphical interface. In addition, **flask** and **flask_cors** were used to build simple service interfaces when needed, enabling some system functions to be provided as APIs that can be called by other components.

- **Organization and Performance Libraries**: The collections library was used to organize data within suitable structures, such as lists and default dictionaries. The **concurrent**.**futures** library was used to execute certain operations in parallel when required, which helps improve performance when processing multiple logs or knowledge passages. The **datetime** and **time** libraries were also used for time-related operations, including recording event times and managing waiting and monitoring intervals.

After explaining the role of each library within the system architecture, we now move to the steps required to install the previous packages and libraries using the pip package manager and the appropriate system tools:

```
i77@i77-server:~$ sudo pip install sentence-transformers
i77@i77-server:~$ sudo pip install qdrant-client nltk requests
i77@i77-server:~$ sudo pip install pypdf openpyxl
i77@i77-server:~$ sudo pip install Flask
i77@i77-server:~$ sudo pip install pandas pdfminer.six
i77@i77-server:~$ python3 - << 'EOF'
>> import ntlk
>> ntlk.download('punkt')
>> ntlk.download('punkt_tab')
>> ntlk.download('averaged_perceptron_tagger')
>> print("all done")
>> EOF
```

3.5 Wazuh Platform as an Open-Source SIEM System

Wazuh is an open-source Security Information and Event Management (SIEM) system that combines traditional SIEM capabilities with Endpoint Detection and Response (EDR) functions. Wazuh was specifically selected after examining several alternatives in the SIEM and security solutions market, for several fundamental reasons:

- **Open Source and Free**: Unlike well-known commercial platforms, such as Splunk, which require costly licenses, Wazuh is released under an open-source license. This makes it a low-cost and highly customizable solution.
- **Flexibility and Scalability**: Wazuh is characterized by a distributed architecture that enables it to scale efficiently for processing large volumes of data. Multiple Wazuh servers and tens or even hundreds of agents can be deployed with minimal operational limitations. Studies indicate that Wazuh has gained wide recognition for its flexibility, scalability, and strong threat detection capabilities. These features made it a suitable choice for this project, which may require processing a large volume of security logs, particularly when simulating diverse and repeated attack scenarios.
- **Easy Integration with the Operational Environment**: Wazuh is designed to be easily deployed within Security Operations Centers (SOCs). It integrates effectively with various

operational environments and is compatible with other log management systems, databases, and analysis tools. It also provides a rich RESTful API that enables external systems or custom applications to retrieve information from it and send commands to it. This seamless integration allows Wazuh to be used as a foundation and leveraged as a ready-made infrastructure instead of building a detection system from scratch.

- **Integrated Detection and Response Capabilities**: Wazuh provides a wide range of detection rules based on known threat signatures and patterns, such as detecting repeated failed login attempts. In addition, it includes the Active Response feature, which enables the execution of automated actions when a specific alert condition is met, such as temporarily blocking the source IP address of an attack.

Table 3.2 : Comparison of Different SIEM Platforms

Evaluation	Technical Analysis and Limitations	Platform
It was excluded due to the high level of customization required to make it function as a fully integrated security platform, which would consume additional time and resources.	It requires significant effort in configuration and detailed tuning, and it lacks direct built-in security detection capabilities compared with specialized solutions.	Elastic Stack (ELK)
It was excluded due to its limited functionality, which may not meet the complex security requirements of the project.	It offers limited features and a narrower scope of coverage compared with modern solutions such as Wazuh or commercial platforms such as Splunk.	OSSIM (Open-Source Security Information Management)
It was excluded because it is primarily a log management tool rather than a comprehensive security solution for threat detection and incident response.	It primarily focuses on log aggregation and search capabilities, and lacks advanced security analytics and automated incident response capabilities.	Graylog
It represents the most balanced and efficient solution, as it satisfies the project requirements without the complexity associated with ELK or the functional limitations found in OSSIM.	It combines the advantages of being open source, similar to ELK, with a strong security-focused design, similar to Splunk, while providing ready-to-use detection and response capabilities.	Wazuh

To install Wazuh Manager, the following commands will be used:

```
i77@i77-server:~$  sudo curl -Ls
https://github.com/wazuh/wazuh/archive/v4.14.0.tar.gz | tar zx

i77@i77-server:~$ sudo cd wazuh-4.14.0

i77@i77-server:~$ sudo ./install.sh

1- What kind of installation do you want (manager, agent, local, hybrid, or
help)? manager
```

As for installing the Wazuh Agent, this can be done easily through the Wazuh Manager web interface. First, the user accesses the Wazuh web dashboard, then navigates to Agent Management and selects Deploy New Agent. Through this interface, the user can specify the type of operating system on which the Wazuh Agent will be installed, as well as the required information related to the Wazuh Manager. After completing these fields, the platform generates customized installation commands for the target system. These commands can then be executed on the selected endpoint to install and register the agent with the Wazuh Manager, as illustrated in Figure 3.1 and Figure 3.2.

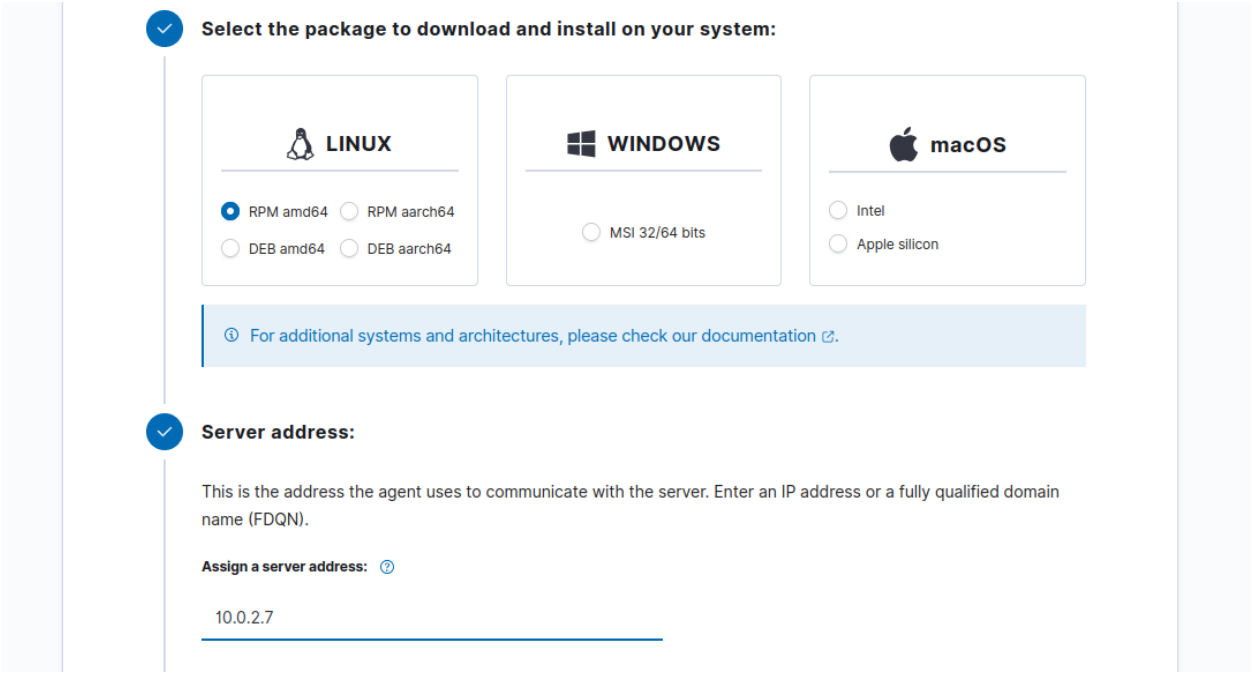


Figure 3.1 : Selecting the Type of System on Which the Wazuh Agent Will Be Installed.

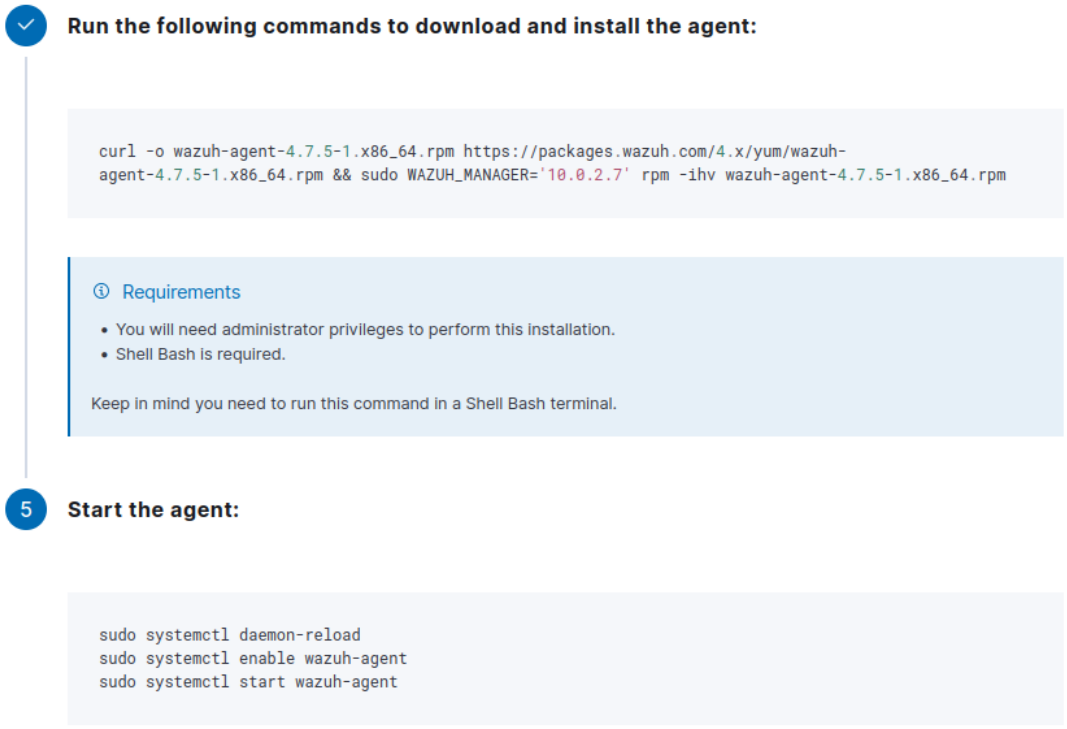


Figure 3.2 : Customized Commands for Installing the Wazuh Agent on the Target System.

3.6 Qdrant Vector Database and Its Role in Supporting the RAG Mechanism

The Retrieval-Augmented Generation (RAG) methodology relies on the existence of a knowledge base that the system can consult when analyzing any security event. This knowledge base is not traditional; that is, it is not a relational database or merely a collection of text files. Rather, it is a Vector Database that stores high-dimensional numerical representations of important text segments and information, enabling searches based on geometric similarity measures between vectors.

Why Qdrant Specifically? Among the various available vector database solutions, Qdrant stands out as an open-source, high-performance, production-ready platform specialized in vector similarity search. Qdrant provides a robust infrastructure for handling vectors and enables search operations based on dense vector search, which is precisely what is required in semantic search applications and RAG systems. Unlike traditional text search engines that rely on keyword matching, Qdrant allows

results to be retrieved based on the proximity of vectors within the mathematical space, meaning that retrieval is based on semantic similarity rather than exact word matching alone.

From a practical perspective, Qdrant is characterized by ease of integration and deployment. It provides a modern API and can be easily run locally within the testing environment without relying on external cloud services. This is particularly important in a security project, as logs and alerts may contain sensitive information that should not be sent to external services. Qdrant is also scalable, as it can efficiently handle thousands or even millions of vectors, while supporting features such as metadata filtering and data updates when required.

The specific commands for installing the vector database, Qdrant, are as follows:

```
i77@i77-server:~$ sudo wget
https://github.com/qdrant/qdrant/releases/download/v1.15.5/qdrant_1.15.5-
1_amd64.deb

i77@i77-server:~$ sudo chmod +x qdrant_1.15.5-1_amd64.deb

i77@i77-server:~$ sudo apt install ./qdrant_1.15.5-1_amd64.deb
```

After installation, it can be observed that the database is running successfully, as illustrated in Figure 3.3.

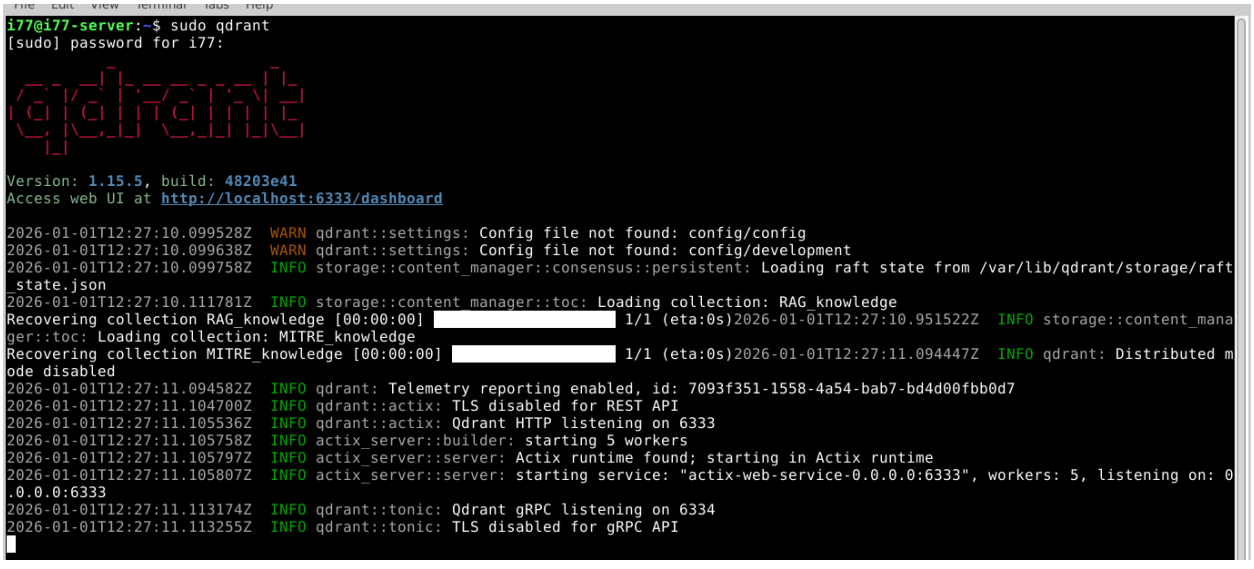


Figure 3. 3 : Verification of the Successful Operation of the Qdrant Database.

The Role of Qdrant in Our Platform: Qdrant was configured to contain a security knowledge base built on the MITRE ATT&CK framework as the primary data source in the current version of the project. MITRE ATT&CK is a well-known framework that documents the Tactics, Techniques, and Procedures (TTPs) used by attackers and provides a structured description of adversary behaviors across different stages of an attack. MITRE techniques and related information were inserted into the knowledge base as small text segments, then converted into vectors using the embedding model and stored inside Qdrant.

For example, if an alert indicates repeated login attempts on the RDP service, vector search can retrieve the most semantically relevant MITRE entries, such as Brute Force or Valid Accounts techniques. Similarly, if a log indicates the creation of a suspicious new service on Windows, the system can associate this behavior with techniques such as Create or Modify System Process. In this way, the model does not interpret the log in isolation; instead, it relies on structured security knowledge that helps it understand the meaning of the event and correlate it with known attack behavior.

In this version of the project, the **MITRE ATT&CK** framework was used as the primary and sole source of the knowledge base. This decision was made to reduce noise caused by multiple sources and to ensure that retrieval is performed from a clear security reference directly related to attack techniques. In the future, the knowledge base can be expanded by adding other sources, such as incident response guides or security governance frameworks. However, the current version focuses on MITRE in order to control result quality and improve the accuracy of correlating logs with attack tactics.

3.7 BAAI/bge-small-en-v1.5 Model and the Llama 3.2 Large Language Model via Ollama

Our intelligent system primarily relies on two Artificial Intelligence model components: the Embedding Model, which converts texts into vectors, and the Large Language Model (LLM), which is responsible for analyzing linguistic context and generating recommendations. In this project, the BAAI/bge-small-en-v1.5 model was selected as the embedding model, while a model from the

LLaMA family, referred to here as Llama 3.2, was selected as the Large Language Model used to generate outputs. The Ollama tool was used to run and manage the latter model locally.

3.7.1 BAAI/bge-small-en-v1.5 Embedding Model

This model is a compact version of the BGE (Beijing Academy of Artificial Intelligence – BGE) family of models, which are designed to produce high-quality text embeddings. The small version is specifically trained on English texts to generate vectors that represent the underlying semantic meaning of sentences, while maintaining a smaller size and fewer parameters compared with larger versions such as bge-large.

The small version was selected due to the limited computational resources available in the project environment. The smaller model requires less memory and generates embeddings more quickly, allowing a large number of text segments to be processed within a shorter time. Despite its smaller size, the performance of bge-small-en-v1.5 is sufficient for achieving the project objectives, as it is capable of capturing the main semantic similarities between security-related texts. The model was downloaded from the Hugging Face repository and executed through the sentence-transformers library. It was used to convert texts such as "Failed SSH login from 192.168.1.5" or "Multiple suspicious file modifications detected" into numerical vectors with a fixed number of dimensions, specifically 384 dimensions in this model. Each vector represents the position of the text within a semantic space, allowing it to be compared with other vectors, such as the vector of a sentence like "an attacker is attempting to guess a password," in order to extract a similarity score.

In addition, the fact that the model is open source and freely available for use made its integration into the project smooth and free from licensing restrictions or dependence on external services.

The specific commands for installing the embedding model are as follows:

```
i77@i77-server:~$ sudo apt install git-lfs -y
i77@i77-server:~$ git lfs install
i77@i77-server:~$ git clone https://huggingface.co/BAAI/bge-small-en-v1.5
```

3.7.2 Llama 3.2 Large Language Model and Its Execution via Ollama

LLaMA models, developed by Meta, have become among the most prominent open-source language models in terms of performance since their release. In this project, a model from the LLaMA family, referred to as Llama 3.2, was selected to serve as the analytical and generative engine of the platform. This model, with its relatively large size of 3 billion parameters, is capable of understanding complex contextual textual inputs and generating coherent and logically structured natural-language outputs. The main reasons for selecting Llama 3.2 specifically include the following:

- **High Performance in Analytical Tasks**: LLaMA models have demonstrated strong capabilities in logical reasoning and in providing detailed answers to questions, especially in the newer optimized versions. In this project, a model is required that can deconstruct a complex security event, which may include one or more logs and technical information, correlate it with the retrieved knowledge, such as response procedures from the knowledge base, and then generate an appropriate recommendation. LLaMA provides this capability due to its extensive training and deep understanding of language.
- **Ability to Run Locally Without External Connectivity**: As an open-source model, LLaMA can be operated on private servers without the need to call cloud-based APIs. Given the sensitivity of cybersecurity data and the project's objective of keeping processing local within the testing environment, using an open model such as LLaMA is an ideal choice. We did not want to send descriptions of security incidents to a cloud-based model such as GPT-4, for example, due to privacy concerns, the need to preserve information confidentiality, and cost considerations. With LLaMA, the model was able to run entirely on our local machine.

To run the model easily and efficiently, we used Ollama. Ollama is an open-source platform designed to simplify the local hosting and execution of Large Language Models, while providing an API that enables communication with the model through the HTTP protocol. In practice, the LLaMA 3.2 model was downloaded and loaded into the Ollama engine, which handled the management of runtime resources, such as loading the model into memory and utilizing CPU cores or the graphics accelerator when available. Ollama provides both a command-line interface and a

lightweight HTTP server, through which prompts can be sent to the model and generative responses can be received.

Running the Model on the Windows 11 Host Machine: Ollama and the llama3.2:latest model were executed on the host machine running Windows 11, outside the VirtualBox environment, in order to take advantage of the available GPU resources. This choice helped improve response speed compared with running the model inside a resource-limited virtual machine. The system scripts communicate with Ollama through its local interface, where the security log and the contextual information retrieved from Qdrant are sent to the model. The generated output is then received, processed, and converted into an alert that can be injected into Wazuh.

After installation, we asked the model about its capabilities in analyzing security logs in order to verify that the model was functioning correctly, as illustrated in Figure 3.4.

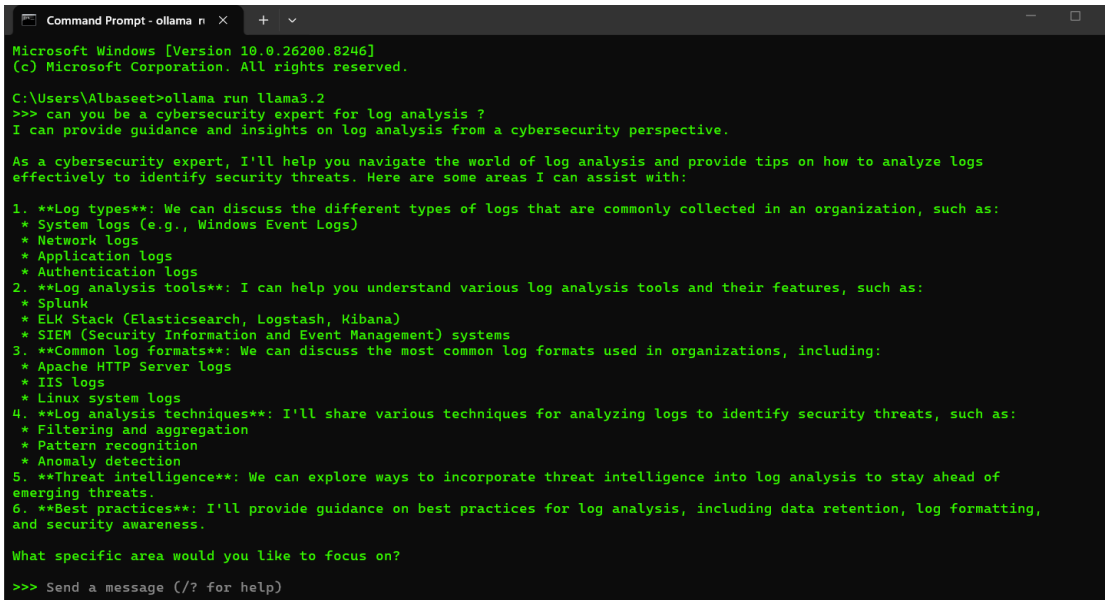


Figure 3.4 : Result of Querying the llama3.2 Model.

In summary, the combination of BAAI-bge and LLaMA served as the intelligent core of the system. The former converts information into a numerical representation that can be semantically processed and compared, while the latter leverages its deep language understanding and the retrieved knowledge context to generate meaningful outputs for the human user.

4. Chapter Three: Proposed Solution

4.1 Introduction

After completing the study of the theoretical concepts related to intrusion detection systems, Security Information and Event Management systems, and generative Artificial Intelligence technologies, this chapter moves on to present the practical proposed solution developed within this project. The solution aims to build an automated security platform capable of exceeding the limitations of traditional SIEM systems by integrating the Wazuh platform with Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), vector databases, and Active Response mechanisms. This integration is intended to address a fundamental problem in cybersecurity environments: traditional detection systems are often capable of generating alerts, yet they do not always possess sufficient capability to understand deep security context, accurately distinguish between genuine threats and false positives, or automatically and systematically recommend an appropriate response action.

The proposed solution is distinguished by the fact that it does not replace Wazuh, but rather enhances it with an Artificial Intelligence layer operating above it. Wazuh remains responsible for log collection, detection rule execution, agent management, and alert visualization through the graphical interface. Meanwhile, the RAG layer adds interpretive and contextual capabilities based on a security knowledge base built from MITRE ATT&CK and response sources, stored within the Qdrant vector database. As a result, the system becomes capable of analyzing a security event not only based on a static rule, but also by relying on relevant security knowledge and the linguistic reasoning provided by the Llama 3.2 model through Ollama.

4.2 Text Chunking Mechanism

The process of segmenting texts into small chunks is a fundamental step in preparing the knowledge base for the RAG-based system. Its objective is to divide large documents and files into limited-size text segments that can be processed efficiently by the embedding model, while also ensuring the retrieval of specific and relevant sections when needed, rather than retrieving entire long texts.

4.2.1 Text Chunking Mechanism for the Detection Stage

The process of building the Detection knowledge base begins with the **chunk_for_detection.py** script. The function of this script is to convert **MITRE ATT&CK** files in Excel format into a structured JSONL file named **attack_cards.jsonl**. The resulting file serves as an important intermediary between the original knowledge files and the Embedding stage. Instead of passing complete Excel files directly to the embedding model, the script extracts the relevant information and transforms it into relatively short, semantically focused cards.

The script includes several helper functions, such as **clean_text**, which is used to clean text and remove unnecessary spaces; **split_listish**, which splits fields that contain list-like values, such as platforms or tactics; and **infer_event_families**, which infers the type of security event family. The **infer_event_families** function classifies the text into categories such as **auth** when the text includes terms related to authentication, login, or passwords; **web** when it contains web-related indicators such as HTTP, PHP, or Web Shell; **process** when it refers to processes or commands such as PowerShell, bash, or cmd; **file** when the text is related to file creation or modification; and **network** when it is associated with connections, DNS, or ports. It also includes other categories such as **policy_denied** and **system_error**. These categories are highly important later because they are used as metadata fields inside Qdrant, allowing the search process to be narrowed to the event type most relevant to the log being analyzed.

The script also includes the **infer_platforms_from_text** function, which attempts to infer the operating system or environment associated with the text, such as Linux, Windows, macOS, Containers, Cloud, or Network. In addition, the **infer_tags** function adds semantic tags such as **linux_auth**, **webshell**, **file_activity**, **process_activity**, **network_activity**, and **privilege_escalation**. These tags are not used merely for documentation; they also help improve retrieval within the Qdrant database through the payload indexes that are created later.
See **Figure 4.1**.

```python
def infer_tags(text, attack_id="", name="", doc_type=""):
    t = f"{attack_id} {name} {text}".lower()
    tags = set()

    if "linux" in t or "auditd" in t or "pam_unix" in t or "sudo" in t or "sshd" in t or "apparmor" in t:
        tags.add("linux")

    if re.search(r'auth|authentication|login|logon|password|credential|pam|sshd|sudo|su\b', t):
        tags.add("auth")
        tags.add("linux_auth" if "linux" in t or "pam_unix" in t or "sudo" in t or "sshd" in t else "general_auth")

    if re.search(r'sudo|su\b|privilege escalation|elevate|root', t):
        tags.add("sudo_su")
        tags.add("privilege_escalation")

    if re.search(r'local interactive|tty|pts/|sudo:auth|su:session', t):
        tags.add("local_auth")

    if re.search(r'web shell|upload\.php|public-facing|web server|server software component', t):
        tags.add("webshell")

    if re.search(r'apparmor|snap\.', t):
        tags.add("apparmor_policy")

    if re.search(r'file creation|file modification|open, write|/var/www/|php|shell script', t):
        tags.add("file_activity")

    if re.search(r'process creation|command execution|execve|powershell|bash|cmd', t):
        tags.add("process_activity")

    if re.search(r'network connection|c2|beacon|socket|dns|proxy', t):
        tags.add("network_activity")

    if doc_type == "technique":
        tags.add("attack_technique")
    elif doc_type == "data_component":
        tags.add("telemetry")
    elif doc_type == "detection_strategy":
        tags.add("detection_logic")

    return sorted(tags)
```

Figure 4.1 : Adding Semantic Tags.

The script generates three main types of cards. The first type is Technique Cards, which extract the technique ID, technique name, description, platforms, tactics, detection notes, and procedure examples. The second type is Data Component Cards, which link each technique to the data sources or log types that can be used to detect it. The third type is Detection Strategy Cards, which combine the detection strategy, related analytics, and suitable log sources. At the end of execution, the script removes duplicates and writes the resulting cards into the **attack_cards.jsonl** file, where each line represents an independent card in JSON format.

The importance of this mechanism lies in the fact that it preserves the relationship between the attack technique and its detection sources, rather than randomly splitting the text. Each knowledge card in the Detection stage carries a clear security meaning: it is either a technique, a data component, or a detection strategy. This improves the quality of retrieval later, because when a security log is converted into a vector and searched within Qdrant, the system does not retrieve general text fragments; rather, it retrieves structured cards directly related to the detection domain.

4.2.2 Text Chunking Mechanism for the Response Stage

The Response knowledge-building mechanism differs from the Detection mechanism in terms of its objective. In the Detection stage, the system requires knowledge that helps interpret the event and correlate it with a specific technique or detection strategy. In the Response stage, however, the system requires knowledge that supports the selection of an appropriate containment or remediation action. Therefore, a dedicated script was developed for this stage, named **chunk_for_response.py**. This script reads several **MITRE ATT&CK** files, including techniques, mitigations, relationships, detection strategies, and data components. It then produces a file named **response_knowledge.jsonl**, and it can also upload the knowledge directly to Qdrant within the **RAG_response_knowledge** collection.

The script includes an important function called **infer_response_guidance**, which is responsible for inferring the recommended actions based on the characteristics of each technique. This function begins by adding general actions that are applicable to most incidents, such as verifying the alert context, comparing the event with endpoint logs, and preserving evidence before executing any action that may alter the system state. After that, the function analyzes the technique name, its description, the associated tactics, and the platforms on which it operates. If the technique is related to initial access, credential access, or privilege escalation, the function adds recommendations related to reviewing account activity, verifying successful logins following failed attempts, and examining privilege memberships. If indicators such as brute force, password, or credential activity are detected, it adds containment actions such as enabling account lockout or blocking a remote source when a valid IP address is available.

The function also considers the targeted platform. If the technique is associated with Linux, it adds recommendations such as reviewing **auth.log** or **journalctl**, examining **sudo**, **su**, **ssh**, and **PAM**, and checking **systemd** services, cron jobs, and command history. If the technique is associated with Windows, it adds recommendations such as reviewing Security, System, PowerShell, and Sysmon logs, as well as examining services, scheduled tasks, and Run registry keys. Similarly, if the text indicates a service-related behavior, the function adds a containment action that involves stopping and disabling the service after collecting its information. If the text includes indicators such as PowerShell or AMSI, it suggests isolating the host or quarantining the malicious script after evidence

collection. If indicators such as Web Shell, PHP files, or paths such as /**var**/**www** appear, it recommends isolating the web application or quarantining the malicious file and collecting web server logs.

After that, the **build_playbook_text** function converts the document into a concise playbook-like text, containing the technique description, recommended actions, containment procedures, and the evidence that should be collected. As a result, the Response knowledge base becomes not merely a description of the threat, but a procedural knowledge base that helps the language model select an appropriate action. This point is particularly important because response selection cannot be based solely on the name of the attack; rather, it must take into account the targeted platform, incident parameters, severity level, and the safety of automated execution. See **Figure 4.2**.

```
response_guidance = infer_response_guidance(technique)

doc = {
    "id": f"response_playbook::{attack_id}",
    "doc_type": "response_playbook",
    "attack_id": attack_id,
    "title": clean_text(technique.get("name")),
    "description": clean_text(technique.get("description")),
    "attack_url": clean_text(technique.get("url")),
    "platforms": platform_list,
    "tactics": tactic_list,
    "is_sub_technique": bool(technique.get("is sub-technique")),
    "parent_technique": clean_text(technique.get("sub-technique of")),
    "mitigation_ids": sorted(set(mitigation_ids)),
    "mitigation_names": sorted(set(mitigation_names)),
    "mitigation_descriptions": list(dict.fromkeys(mitigation_descriptions))[:10],
    "detection_strategy_ids": sorted(set(detection_ids)),
    "detection_strategy_names": sorted(set(detection_names)),
    "procedure_examples": list(dict.fromkeys(technique_to_examples.get(attack_id, [])))[:8],
    "recommended_actions": response_guidance["recommended_actions"],
    "containment_actions": response_guidance["containment_actions"],
    "evidence_to_collect": response_guidance["evidence_to_collect"],
    "auto_executable": False,
    "requires_approval": True,
    "destructive": False,
    "target_platforms": platform_list,
    "source_trace": {
        "technique_sheet": "techniques",
        "associated_mitigations_sheet": "associated mitigations",
        "associated_detection_sheet": "associated detection strategies",
        "procedure_examples_sheet": "procedure examples",
    },
}
doc["text"] = build_playbook_text(doc)
docs.append(doc)
```

Figure 4.2 : Response Document Structure.

4.3 Embedding Mechanism

After preparing the knowledge cards for both the Detection and Response stages, the Embedding stage begins, as shown in **Figure 4.3** .

```
i77@i77-server:~/AS-Platform/services$ sudo python3 chunk_for_detection.py
[+] Wrote 1488 cards to /home/i77/AS-Platform/attack_cards.jsonl
[+] Breakdown:
    - technique: 691
    - data_component: 106
    - detection_strategy: 691
i77@i77-server:~/AS-Platform/services$ sudo python3 chunk_for_response.py
[+] Built 1532 response knowledge documents.
[+] Sources: techniques=691 | mitigations=44 | detection_strategies=691 | data_components=106
[+] Useful relationship rows observed for future enrichment: 19431
[+] Wrote 1532 response docs to /home/i77/AS-Platform/response_knowledge.jsonl
[+] Breakdown:
    - detection_response_map: 691
    - evidence_collection_guide: 106
    - mitigation_mapping: 44
    - response_playbook: 691
i77@i77-server:~/AS-Platform/services$
```

Figure 4.3 : Splitting Files into Detection and Response Cards

For this purpose, an embedding model trained for Natural Language Processing (NLP) is used to generate high-dimensional vectors that preserve the underlying semantic meanings of texts. In the proposed solution, the open-source **BAAI/bge-small-en-v1.5 model**, developed by the Beijing Academy of Artificial Intelligence, was selected to generate these vectors. This model is characterized by its ability to produce effective representations of English texts with high efficiency. Its larger variants, such as bge-large, have demonstrated notable performance compared with commercial models in terms of speed and accuracy, while maintaining lower operational cost. The use of an open-source model also provides greater flexibility and reliability, without the limitations of cloud-based services or per-request usage costs.

The embedding process begins by loading the SentenceTransformer model from the local path **/home/i77/AS-Platform/models/bge-small-en-v1.5**. This model is responsible for converting text into a numerical vector that represents the semantic meaning of the text. After loading the model, the vector dimensionality is obtained using get_sentence_embedding_dimension, since Qdrant requires the vector size before creating the collection. The system then connects to the **Qdrant database** running on **localhost** at port **6333**.

Cosine Similarity: Cosine similarity is a metric used to determine the degree of similarity between the directions of two vectors in a multidimensional space, regardless of their magnitudes. It is calculated as the cosine of the angle between the two vectors. In the context of RAG, two vectors with a high cosine similarity score, with a value close to 1, indicate that they represent the same

semantic concept, even if they do not share any exact keywords. This represents the core principle of semantic search, which distinguishes RAG from traditional keyword-based search.

In the **Detection stage**, the script uses the **RAG_detection_knowledge** collection. If the collection already exists, it is deleted and recreated to ensure that the stored knowledge matches the latest version of attack_cards.jsonl. The collection is created using VectorParams with Distance.COSINE, since Cosine Similarity is suitable for semantic search over textual representations. After creating the collection, payload indexes are created on the fields used for filtering, such as doc_type, platforms, event_families, and tags. These indexes make the search process faster and more accurate when applying conditions such as document type or event family.

as shown in **Figure 4.4**.

```python
import json
from qdrant_client import QdrantClient
from qdrant_client.http.models import VectorParams, Distance, PointStruct, PayloadSchemaType
from sentence_transformers import SentenceTransformer

JSONL_PATH = "/home/i77/AS-Platform/attack_cards.jsonl"
MODEL_PATH = "/home/i77/AS-Platform/models/bge-small-en-v1.5"
QDRANT_HOST = "localhost"
QDRANT_PORT = 6333
COLLECTION = "RAG_detection_knowledge"

def main():
    print("[+] Loading embedding model...")
    model = SentenceTransformer(MODEL_PATH)
    vector_size = model.get_sentence_embedding_dimension()
    print(f"[+] Embedding dimension: {vector_size}")

    print("[+] Connecting to Qdrant...")
    client = QdrantClient(host=QDRANT_HOST, port=QDRANT_PORT)

    # بدل recreate_collection لأنه deprecated
    if client.collection_exists(COLLECTION):
        print(f"[+] Deleting existing collection: {COLLECTION}")
        client.delete_collection(collection_name=COLLECTION)

    print(f"[+] Creating collection: {COLLECTION}")
    client.create_collection(
        collection_name=COLLECTION,
        vectors_config=VectorParams(
            size=vector_size,
            distance=Distance.COSINE
        )
    )

    for field in ["doc_type", "platforms", "event_families", "tags"]:
        try:
            client.create_payload_index(
                collection_name=COLLECTION,
                field_name=field,
                field_schema=PayloadSchemaType.KEYWORD
            )
            print(f"[+] Indexed payload field: {field}")
        except Exception as e:
            print(f"[WARN] Could not index {field}: {e}")

    points = []
    pid = 1

    print("[+] Reading cards and generating embeddings...")
    with open(JSONL_PATH, "r", encoding="utf-8") as f:
        for line in f:
            card = json.loads(line)
            text = card.get("text", "").strip()
            if not text:
```

Figure 4.4 : Core Operations in the Embedding Program.

The script then reads the JSONL file line by line, where each line represents one knowledge card. The text field is extracted and converted into an embedding using model.encode. After that, a PointStruct object is created containing the identifier, the vector, and the payload. The payload preserves all structured information, such as doc_type, attack_id, name, platforms, tactics, event_families, tags, and the original text itself. These points are uploaded to Qdrant using the upsert operation. In the **Response stage**, a similar logic is applied to the response_knowledge.jsonl file, but the target collection is **RAG_response_knowledge**. As a result, two separate knowledge stores are created: the first for detection analysis and the second for response planning.

as shown in **Figure 4.5** and **Figure 4.6** .

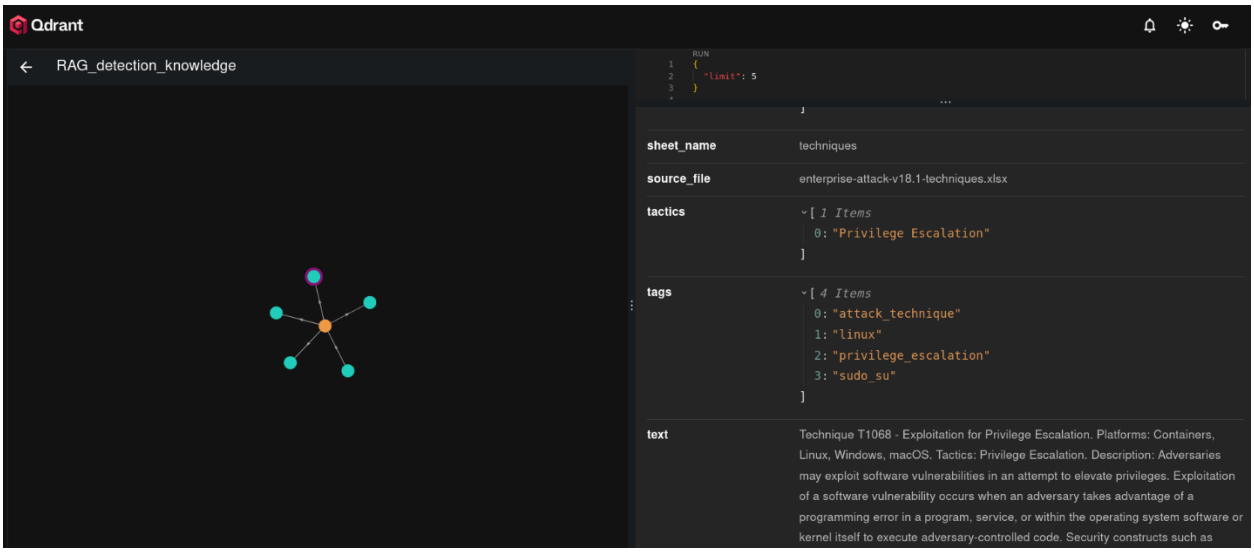


Figure 4.5 : RAG Detection Collection

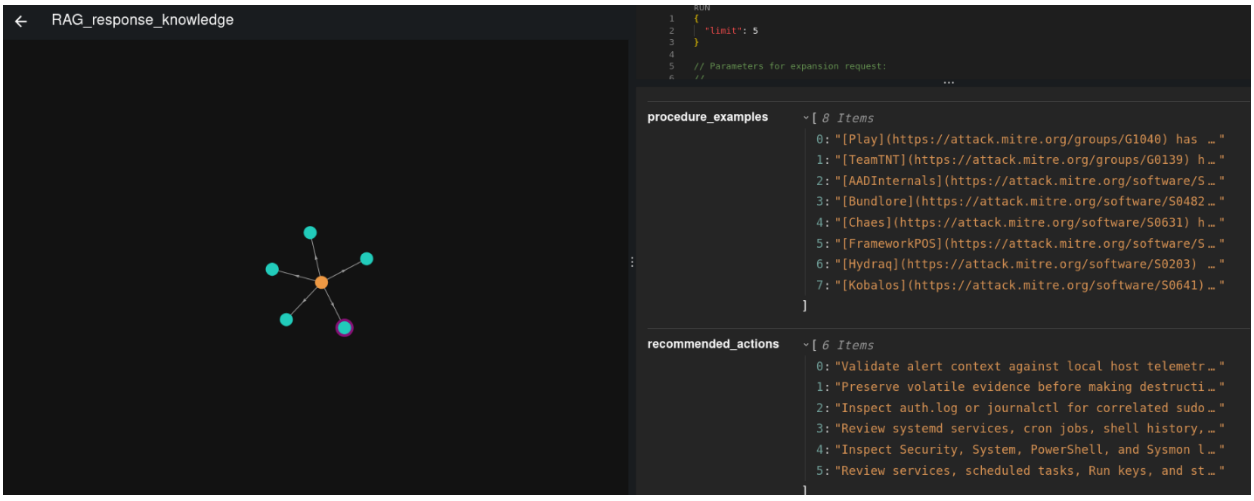


Figure 4.6 : RAG Response Collection.

4.4 Architecture of the Proposed Solution

After preparing the core components of the system, namely the Wazuh platform, the Qdrant vector database, the **BAAI/bge-small-en-v1.5** embedding model, and the **Llama 3.2** Large Language Model operated through Ollama, the general architecture of the proposed solution can be described as an interconnected sequence of stages that begins from the moment an attack is executed and ends with the generation of an intelligent alert, followed by the production of an appropriate response recommendation or action. The proposed system does not function as a replacement for **Wazuh**; rather, it operates as an AI-supported layer above it, benefiting from Wazuh's ability to collect logs and apply detection rules, while adding deeper contextual analysis capabilities through **Retrieval-Augmented Generation** (**RAG**).

Figure 4.7 illustrates the general architecture of the proposed solution, where the system is presented as a set of successive layers. The first layer begins with the Parrot Security attack machine, which is used to execute test attacks against endpoint machines running Windows or Ubuntu. The second layer consists of the targeted endpoints that contain the Wazuh Agent, which is responsible for monitoring local events and sending them to the Wazuh Manager. The third layer represents the Wazuh Manager, which serves as the central point for collecting and analyzing logs according to traditional rules. The fourth layer is the Artificial Intelligence layer, which includes RAG Detection, RAG Response, Qdrant, Llama 3.2 via Ollama, the Flask Control API, the Action Tracking Ledger, and the Active Response Dispatcher.

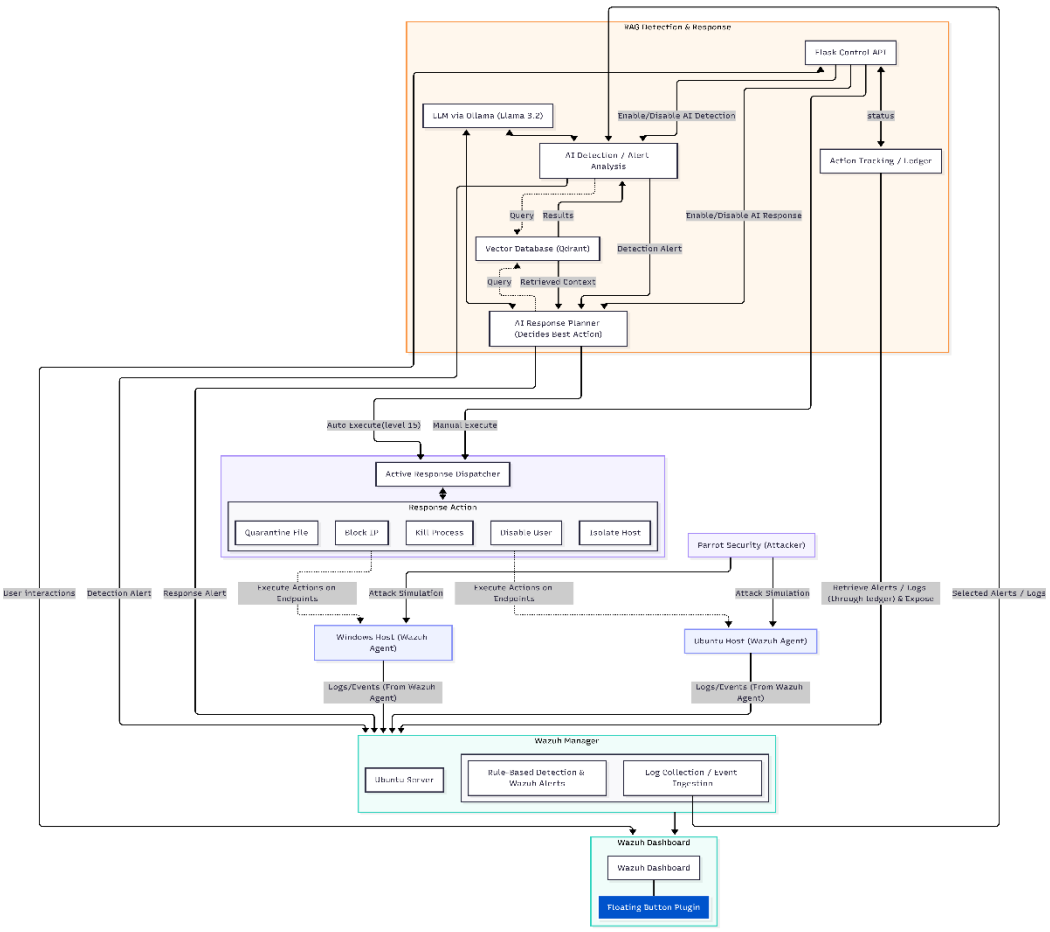


Figure 4.7 : General Architecture of the Proposed Solution

The importance of this diagram lies in the fact that it does not present the components as isolated tools; rather, it illustrates the flow of data between them. The security log begins as a raw event on the endpoint, then moves to the Wazuh Manager, where it is evaluated either through Wazuh's traditional rules or through the RAG Detection layer. After that, the intelligent alert is forwarded to the RAG Response layer to determine the appropriate action.

If the severity level is very high and the safety conditions are satisfied, the action is passed to the Active Response Dispatcher for execution on the appropriate endpoint. In this way, the system evolves from a platform limited to monitoring and alerting into a system capable of analysis, recommendation, and response.

4.4.1 Attack Execution and Initiation of Logging

The practical scenario of the proposed solution begins with the Parrot Security machine, which represents the attacker device within the test environment. This machine is used to simulate various attacks against one of the endpoint devices, whether the target system is running Windows or Ubuntu. These attacks may include, for example, uploading a Web Shell, executing remote commands, performing failed login attempts, scanning for vulnerabilities, or creating a suspicious process or service within the target system.

When the malicious activity occurs, the target operating system generates raw logs related to that activity. In the case of Windows, the events may appear in the Windows Event Log or within application and service logs. In the case of Linux, the events may appear in files such as auth.log, syslog, auditd logs, or application logs. At this stage, the role of the Wazuh Agent becomes essential, as it reads these logs, collects them, and sends them to the Wazuh Manager for centralized analysis.

4.4.2 Log Collection by Wazuh Manager

After logs are collected from the endpoints, the Wazuh Agent sends them to the Wazuh Manager through the agent communication channel. The Wazuh Manager receives, stores, and analyzes these logs according to predefined detection rules. This stage represents the traditional line of defense within the system, as Wazuh relies on XML-based rules to detect known patterns of security events, such as failed login attempts, suspicious command execution, sensitive file creation, or activities associated with vulnerabilities or malware.

When a specific log matches one of the Wazuh rules, the manager generates a traditional alert containing information such as the agent name, agent ID, rule description, severity level, and the groups associated with the alert. However, if the log does not match a clear rule, the event may remain within the archived logs without being converted into a high-severity alert. Therefore, log storage in JSON format was enabled within the archives.json file, as this file represents an important source for the RAG Detection layer. Through this file, the Artificial Intelligence component can analyze certain raw events that may not be detected by traditional rules.

4.4.3 Decision to Forward Events to RAG-Based Intelligent Analysis

After the logs reach the Wazuh Manager, the system proceeds to the decision-making stage, in which it determines whether the event requires intelligent analysis or not. Not all logs are sent to the Artificial Intelligence model, as doing so would lead to excessive resource consumption and increased noise. Therefore, a selection logic was designed within the RAG Detection script to focus only on important or ambiguous events. There are two main cases (illustrates in **Figure 4.8**) in which events are forwarded to RAG Detection:

- The first case is a **high-severity alert**. If Wazuh generates an alert with a level of 10 or higher, the alert or its associated log is sent to the RAG Detection system to verify its significance and reduce the possibility of false positives. In this case, RAG operates in **Validation mode**; that is, its objective is not to detect the event from scratch, but to reassess an important alert generated by Wazuh and confirm its actual context.
- The second case involves a **raw or low-severity** log that does not match a clear detection rule but contains suspicious indicators. In this case, RAG operates in **Hunting mode**, meaning that its objective is to search for potential threats that were not escalated by traditional rules. This may occur because the attack is new, because the log does not match a known signature, or because the event appears simple when viewed in isolation but becomes significant when linked to its temporal and local context.

```python
def follow(file_path):
    if not os.path.exists(file_path):
        print(f"[FATAL] File not found: {file_path}")
        raise SystemExit(1)

    with open(file_path, "r") as file_obj:
        file_obj.seek(0, os.SEEK_END)
        while True:
            line = file_obj.readline()
            if not line:
                time.sleep(0.1)
                continue
            yield line


def should_skip_log(log_entry, full_log_str):
    return (
        "ai_id" in log_entry
        or log_entry.get("data", {}).get("ai_id") == "000_ai"
        or "000_ai" in full_log_str
        or any(noise in full_log_str for noise in INFRASTRUCTURE_NOISE)
    )


def get_trigger_reason(full_log_str, wazuh_level):
    if wazuh_level >= 10:
        return f"High Level Rule ({wazuh_level})"
    if any(keyword in full_log_str for keyword in SUSPICIOUS_KEYWORDS):
        return "Suspicious Keyword Match"
    return "Raw Suspicious Log"
```

Figure 4.8 : Core Functions for the Decision to Forward Events to RAG.

At this stage, the ai_rag_detection script ignores logs that contain **ai_id** or **000_ai** in order to prevent a feedback loop in which the system repeatedly analyzes alerts that it previously generated itself. It also ignores certain noise logs related to the system's internal infrastructure, ensuring that the analysis remains focused on security events with meaningful value.

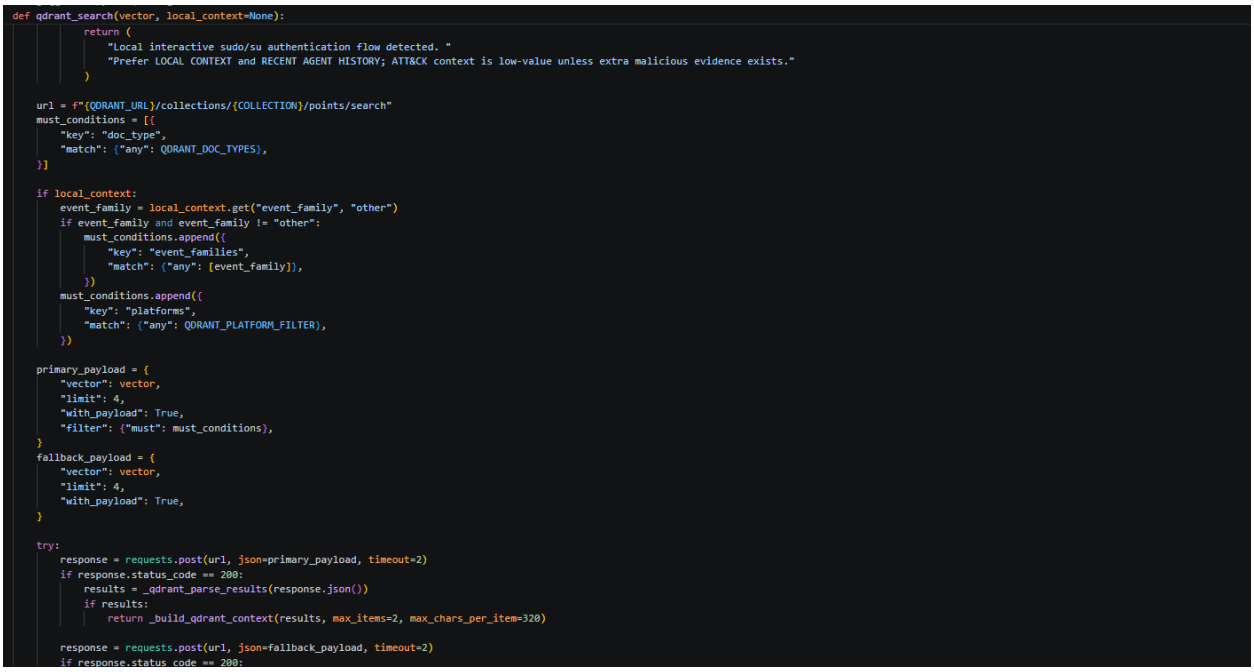
4.4.4 In–Depth Analysis Through RAG Detection

When a specific log is selected for analysis, the ai_rag_detection script begins by constructing a local context around that log. The analysis is not limited to the raw log text alone; instead, a set of supporting information is extracted, such as the event type, username, IP address if available, hostname, agent ID, path associated with the event, the program or service that generated the log, as well as temporal indicators such as the number of repeated occurrences within a short period. (as shown in **Figure 4.9**) This step is highly important because it transforms the log from an isolated text entry into a context-rich security event. For example, a single failed authentication attempt does not necessarily indicate an attack. However, repeated failures within a short time window followed by a successful login may represent a stronger indicator of a compromise attempt. Similarly, the significance of a suspicious command in a local log differs depending on whether it is associated with a known user, an unusual process, or an external source.

```
[AI ANALYZING] Mode: HUNTING | Agent: i77-server | Event: Enriched Log Analysis
  [RAW LOG] Apr 22 21:21:51 i77-server snapd[850]: storehelpers.go:914: cannot refresh: snap has no updates available: "bare", "c
d"
  [LOCAL CONTEXT]
    - event_family: other
    - source_ip: No_IP
    - username: No_User
    - auth_actor: No_Actor
    - auth_actor_uid: No_UID
    - auth_principal: No_Principal
    - tty: No_TTY
    - auth_outcome: other
    - path: No_Path
    - rule_id: unknown_rule
    - program_name: unknown_program
    - normalized_template: apr <num> <num>:<num>:<num> i77-server snapd[<num>]: storehelpers.go:<num>: cannot refresh: snap has n
-<num>", "gtk-common-themes", "snapd"
    - local_interactive_auth: False
    - known_benign_policy_noise: False
    - burst_count_60s: 1
    - unique_templates_60s: 1
    - unique_source_ips_60s: 0
    - unique_usernames_60s: 0
    - unique_paths_60s: 0
    - same_template_ratio: 1.0
    - high_frequency_context: False
    - moderate_repetition_context: False
    - auth_failures_60s: 0
    - failed_then_success_same_user: False
    - failed_then_success_same_tty: False
  [STEP 1] Generating Embedding...
```

Figure 4.9 : Building the Local Context in Preparation for Analysis.

After constructing the local context, the log, or the text extracted from it, is converted into a vector using the BAAI/bge-small-en-v1.5 embedding model. This model produces a numerical representation of the log, allowing it to be compared with documents previously stored in the Qdrant database. The script then sends a query to Qdrant to search for the most semantically similar results. The search does not rely solely on general textual similarity; it can also be constrained by specific properties such as document type, platform, and event family, ensuring that the retrieved results are more relevant to the current log. as shown in **Figure 4.10**



```
def qdrant_search(vector, local_context=None):
            return (
                "Local interactive sudo/su authentication flow detected. "
                "Prefer LOCAL CONTEXT and RECENT AGENT HISTORY; ATT&CK context is low-value unless extra malicious evidence exists."
            )

    url = f"{QDRANT_URL}/collections/{COLLECTION}/points/search"
    must_conditions = [{
        "key": "doc_type",
        "match": {"any": QDRANT_DOC_TYPES},
    }]

    if local_context:
        event_family = local_context.get("event_family", "other")
        if event_family and event_family != "other":
            must_conditions.append({
                "key": "event_families",
                "match": {"any": [event_family]},
            })
        must_conditions.append({
            "key": "platforms",
            "match": {"any": QDRANT_PLATFORM_FILTER},
        })

    primary_payload = {
        "vector": vector,
        "limit": 4,
        "with_payload": True,
        "filter": {"must": must_conditions},
    }
    fallback_payload = {
        "vector": vector,
        "limit": 4,
        "with_payload": True,
    }

    try:
        response = requests.post(url, json=primary_payload, timeout=2)
        if response.status_code == 200:
            results = _qdrant_parse_results(response.json())
            if results:
                return _build_qdrant_context(results, max_items=2, max_chars_per_item=320)

        response = requests.post(url, json=fallback_payload, timeout=2)
        if response.status_code == 200:
```

Figure 4.10 : Qdrant Search Function.

Qdrant returns a set of the most semantically relevant results, which serve as supporting knowledge context for the analysis. These results may be related to MITRE ATT&CK techniques, detection strategies, data components, or similar log patterns. The retrieved results are then combined with the original log, the local context, and the recent event history into a single prompt, which is sent to the Llama 3.2 model through Ollama.

4.4.5 Building the Detection Prompt and Generating a RAG Detection Alert

The RAG Detection Prompt represents one of the most important components of the proposed solution, as it defines the model's reasoning process and the boundaries of its decision-making. This

prompt was designed to instruct the model to act as a strict SOC analyst and to rely first on the current log, then on the local context, followed by the recent history, and finally on the knowledge retrieved from Qdrant. This ordering is important because it prevents the model from exaggerating the event based on general knowledge that is not directly related to the log.

The prompt also includes clear instructions to prevent hallucination, such as not assuming the existence of a remote attacker when no IP address is present, not increasing the severity level without evidence, and not using the RAG context if the retrieved results are not actually relevant to the event. It also includes a guide for assigning severity levels from 1 to 15, where lower levels are assigned to routine or low-significance events, medium levels to suspicious events, and high levels to events indicating a strong intrusion attempt or a potential compromise. Levels 14 and 15 are reserved for highly critical cases. as shown in **Figure 4.11**

```
You are a strict SOC analyst.
Return valid JSON only.
Do not use markdown.
Do not explain outside JSON.
Keep ai_analysis under 220 characters.
Keep description under 110 characters.

MODE:
{mode}

CURRENT_LOG:
{full_log}

LOCAL_CONTEXT:
{local_context_text}

RECENT_HISTORY:
{history_text}

KB_CONTEXT:
{context}

RULES:
- Use CURRENT_LOG first, then LOCAL_CONTEXT, then RECENT_HISTORY, then KB_CONTEXT.
- If KB_CONTEXT is not directly relevant, set rag_context_used="False".
- If source_ip=No_IP, do not invent a remote attacker.
- If burst_count_60s=1, treat as isolated unless history clearly proves repetition.
- auth_failures_60s is the count of password failures within 60s.
- failed_then_success_same_user=True means the same local principal had failures then success in 60s.
- failed_then_success_same_tty=True means the same local tty had failures then success in 60s.
- auth_actor_uid is the local numeric uid of the actor when username is missing, for example "by (uid=1000)".
- If auth_actor_uid matches earlier local auth failures, treat that as same-user auth correlation even when actor username is missing.
- Local sudo/su on tty/pts with no remote source IP is usually benign or low severity.
- Local auth_failures_60s 1-2 usually means level 2-3 unless extra evidence exists.
- Local auth_failures_60s 3-4 usually means level 3-4 unless extra evidence exists.
- Local auth_failures_60s >=5 without success usually means level 4-6.
- Password mistakes followed by local sudo/su success are usually level 4-6 unless extra malicious evidence exists.
- Do not rate local sudo/su authentication-only activity above 7 without extra malicious evidence.
- known_benign_policy_noise=True usually means level 1-2.
- Explicit web shell or RCE wording in /var/www/ is usually level 9-12.
- For local tty auth repetition, prefer wording like repeated local password failures or possible local password guessing, not remote brute force.
- If auth_failures_60s >= 3, auth_correlation_used should usually be "True".
- If failed_then_success_same_user=True or failed_then_success_same_tty=True, auth_correlation_used must be "True".
- Set recent_history_used="True" only if RECENT_HISTORY changed severity or changed the conclusion.
```

Figure 4.11: RAG Detection Prompt.

The model is required to return the result in JSON format only, without any external text or Markdown. This JSON contains a fixed identifier, ai_id, with the value 000_ai, in order to distinguish alerts generated by Artificial Intelligence. It also contains ai_rule, which includes the severity level, description, and groups; ai_agent_info, which contains information about the original

endpoint; full_log, which contains the original log; and ai_evaluation, which includes a concise textual analysis and indicates whether the RAG context was used or not. See **Figure 4.12**.

```
Return exactly this JSON:
{{
  "ai_id": "000_ai",
  "ai_rule": {{
    "level": <int 1-15>,
    "description": "<short summary>",
    "groups": ["ai_analyzed", "rag_detection", "correlated_event"]
  }},
  "ai_agent_info": {{
    "id": "{escaped_agent_id}",
    "name": "{escaped_agent_name}",
    "platform": "linux/windows/MacOS/ios/android/(try to guess what is the platform , if not then type other)"
  }},
  "manager": {{ "name": "wazuh-manager" }},
  "full_log": "{escaped_full_log}",
  "ai_evaluation": {{
    "ai_analysis": "<very short evidence-based reasoning>",
    "local_context_used": "True/False",
    "recent_history_used": "True/False",
    "auth_correlation_used": "True/False",
    "rag_context_used": "True/False"
  }}
}}
""".strip()
```

Figure 4.12 : Detection Alert Structure.

After receiving the model's output, the script attempts to extract a valid JSON object. If the output is incomplete or contains additional text, it is cleaned or repaired as much as possible. The value of ai_id is then fixed as 000_ai to ensure proper distinction. After that, the system decides whether to send the alert to Wazuh based on the resulting severity level.

4.4.6 Sending the RAG Detection Alert to Wazuh

After the language model completes the event analysis and generates the RAG Detection Alert, the ai_rag_detection script injects this alert into the Wazuh Manager. The injection process is performed over the TCP protocol using a local socket that connects to the address **127.0.0.1** on port **5555**. (illustrates in **Figure 4.13**) This port was configured in ossec.conf to receive locally generated events in **Syslog/JSON** format.

```
def write_to_wazuh(alert_data):
    try:
        message = json_dumps_text(alert_data) + "\n"

        print("  [STEP 6] Opening TCP Socket to Wazuh (127.0.0.1:5555)...")
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(2)
        sock.connect(("127.0.0.1", 5555))
        sock.sendall(message.encode("utf-8"))
        sock.close()

        level = alert_data.get("ai_rule", {}).get("level", "N/A")
        print(f"[SUCCESS] Alert injected into Wazuh Dashboard via TCP (Level {level})")
        return True

    except ConnectionRefusedError:
        print("[ERROR] Wazuh is NOT listening on port 5555. Check ossec.conf!")
        return False
    except Exception as exc:
        print(f"[ERROR] TCP Injection Failed: {exc}")
        return False
```

Figure 4.13 : Sending the Alert to Wazuh.

This step is essential for integrating Artificial Intelligence outputs into Wazuh. Instead of keeping the model's analysis in an external file or a separate interface, it is inserted into Wazuh as a new security event. The custom Wazuh rules then identify, classify, and display it within the Wazuh Dashboard. Since the alert contains the field ai_id with the value **000_ai**, it can be easily distinguished from traditional alerts. Using a local port on 127.0.0.1 provides a higher level of security in the design, because Wazuh receives these alerts only from local scripts rather than from external machines. In addition, the presence of a unique field within the JSON structure allows precise rules to be created that capture only RAG Detection alerts.

4.4.7 Response Planning Through RAG Response

After the RAG Detection Alert is injected into Wazuh and appears within **alerts.json**, the second stage of the proposed solution begins, namely the RAG Response stage. This stage operates through the **ai_rag_response** script, which monitors Wazuh alerts and specifically searches for alerts that contain the **ai_id** field with the value **000_ai**. In other words, this stage does not process all Wazuh alerts; rather, it handles only the alerts that have already been analyzed by RAG Detection.

The RAG Response script extracts the essential information from the detection alert, such as the severity level, alert description, analysis generated by the model, target device information, and the original log. It then extracts important indicators from the text, such as an IP address, file path, username, service name, or process ID. These indicators are important because they determine the type of action that can be executed. For example, **block_ip** cannot be performed if there is no clear IP address, and **quarantine_file** cannot be performed if there is no clear file path.

After that, the alert or its description is converted into a vector, and a search is performed within the **response knowledge** base in Qdrant. This knowledge base differs from the detection knowledge base, as it contains playbooks, mitigations, evidence collection guidelines, and links between detection patterns and response options. The script then constructs a dedicated response prompt and sends it to Llama 3.2 through Ollama.

The response prompt (illustrates in **Figure 4.14**) includes a predefined list of allowed actions, such as **monitor_only**, **collect_triage**, **block_ip**, **kill_process**, **quarantine_file**, **disable_user**, and **isolate_host**. The model is required to select only one action from this list, while providing a summary, the rationale behind the choice, manual steps, evidence that should be collected, and the execution parameters of the selected action. It is also required to determine whether the recommendation is safe for automatic execution through the **safe_for_auto_execute** field.

```
    return f"""
You are a strict SOC response planner.
Return valid JSON only.
Do not use markdown.
Do not explain outside JSON.
Keep summary under 160 characters.
Keep reasoning under 220 characters.

ALLOWED_ACTIONS:
{allowed_actions_text}

TARGET_ENDPOINT:
- agent_id: {_safe_prompt_text(target_id)}
- agent_name: {_safe_prompt_text(target_name)}
- platform: {_safe_prompt_text(target_platform)}

SOURCE_AI_ALERT:
- level: {level}
- description: {_safe_prompt_text(description)}
- ai_analysis: {_safe_prompt_text(ai_analysis)}

CURRENT_LOG:
{full_log}

IOC_HINTS:
- ip: {_safe_prompt_text(iocs.get("ip", ""))}
- path: {_safe_prompt_text(iocs.get("linux_path") or iocs.get("windows_path") or "")}
- user: {_safe_prompt_text(iocs.get("user", ""))}
- tty: {_safe_prompt_text(iocs.get("tty", ""))}
- service_name: {_safe_prompt_text(iocs.get("service_name", ""))}

KB_CONTEXT:
{kb_context}

RULES:
- Choose exactly one value from ALLOWED_ACTIONS as recommended_action.
- Use monitor_only for benign or low-confidence activity.
- Use collect_triage when more evidence is needed before containment.
- Use block_ip only when an IP is relevant.
- Use quarantine_file only when a file path is relevant.
- Use disable_user only when a specific user account is relevant.
- Use kill_process only when process/service-style containment is appropriate.
- Use isolate_host only for severe compromise scenarios.
- Do not invent remote attackers when no IP exists.
- Prefer conservative recommendations for routine local sudo/su activity.
- For explicit web shell / RCE / malicious service / AMSI bypass patterns, stronger response is acceptable.
- safe_for_auto_execute should be True only when the action is targeted, low-risk, and operationally safe.
```

Figure 4.14: RAG Response Prompt.

4.4.8 Decision Policy and Automated Execution

The action proposed by the model is not executed directly, because automated response in security systems may be risky if performed without proper controls. Therefore, the **ai_rag_response** script includes a **Policy Gate** mechanism, which serves as an intermediate validation layer separating the model's recommendation from the execution decision. The automated execution policy depends on several conditions. The first condition is that the original detection alert must have a **severity level of 15**, meaning that the event has reached the highest severity level in the system. The second condition is that the proposed action must be included in the list of actions allowed for automatic execution. The third condition is that the **safe_for_auto_execute** field must be set to True. If these

conditions are satisfied, the action is considered eligible for automated execution, and **auto_execute** is set to **True**. Otherwise, the action remains a manual recommendation displayed to the analyst without automatic execution.

After determining the policy decision, the RAG Response script generates a new alert (illustrates in **Figure 4.15**) containing **response_id** with the value **000_response_ai**. This alert includes the proposed action, the policy reason, the execution mode, endpoint information, response summary, confidence level, suggested evidence, and action parameters. It is then injected again into Wazuh through port 5555, where it is handled as an intelligent response alert.

```
"response_id": RESPONSE_ID,
"response_origin": RESPONSE_ORIGIN,
"response_version": RESPONSE_VERSION,
"response_rule": {
    "level": response_level,
    "description": (
        "AI response requires immediate execution"
        if policy["auto_execute"]
        else "AI response recommendation generated"
    ),
    "groups": groups,
},
"ai_agent_info": {
    "id": target_agent_id,
    "name": target_agent_name,
    "platform": target_platform,
},
"manager": {"name": "wazuh-manager"},
"full_log": source_alert.get("full_log", ""),
"source_ai_alert": {
    "level": source_level,
    "description": ai_rule.get("description", ""),
    "ai_analysis": ai_eval.get("ai_analysis", ""),
},
"target_endpoint": {
    "agent_id": target_agent_id,
    "agent_name": target_agent_name,
    "platform": target_platform,
},
"ai_response": {
    "recommended_action": plan["recommended_action"],
    "response_command": policy["response_command"],
    "auto_execute": "True" if policy["auto_execute"] else "False",
    "execution_mode": policy["execution_mode"],
    "policy_name": policy["policy_name"],
    "policy_reason": policy["policy_reason"],
    "summary": plan["summary"],
    "confidence": plan["confidence"],
    "reasoning": plan["reasoning"],
    "manual_steps": plan["manual_steps"],
    "evidence_to_collect": plan["evidence_to_collect"],
    "action_parameters": plan["action_parameters"],
    "rag_context_used": plan["rag_context_used"],
    "target_agent_id": target_agent_id,
    "target_agent_name": target_agent_name,
    "target_platform": target_platform,
```

Figure 4.15 : Response Alert Structure .

4.4.9 Active Response Execution and Tracking

If the response alert contains **auto_execute=True**, the custom Wazuh rules capture it and trigger the Active Response Dispatcher. The **ai_response_dispatcher** script acts as a bridge between the RAG Response decision and the Wazuh Active Response mechanism. It does not decide the response type again; rather, it reads the decision issued by RAG Response and translates it into an

appropriate executable command according to the operating system and the targeted endpoint. illustrates in **Figure 4.16**

```python
def handle_add(argv, wrapper: Dict[str, Any]) -> int:
    manual_ctx = extract_manual_request(wrapper)
    if manual_ctx:
        write_debug_file(
            argv[0],
            f"Direct manual execute request received | target_agent={manual_ctx['target_agent_id']} "
            f"| action={manual_ctx['recommended_action']}"
        )
        return dispatch_context(argv, manual_ctx)

    alert = get_nested(wrapper, "parameters", "alert", default={}) or {}
    ctx = extract_response_context(alert)

    if not ctx:
        write_debug_file(argv[0], "Ignored non-response alert")
        return OS_SUCCESS

    if not ctx["auto_execute"]:
        write_debug_file(
            argv[0],
            f"Manual-only response ignored | target_agent={ctx['target_agent_id']} "
            f"| action={ctx['recommended_action']} | reason={ctx['policy_reason']}"
        )
        return OS_SUCCESS

    if not ctx["response_command"]:
        write_debug_file(argv[0], "auto_execute=True but response_command is empty")
        return OS_INVALID

    action_key = build_action_key(alert)
    decision = send_keys_and_check_message(argv, [action_key])

    if decision == ABORT_COMMAND:
        write_debug_file(argv[0], f"Aborted duplicated action | key={action_key}")
        return OS_SUCCESS

    if decision != CONTINUE_COMMAND:
        write_debug_file(argv[0], f"Invalid check_keys decision | key={action_key}")
        return OS_INVALID
```

Figure 4.16 : Main Dispatch Function.

The design supports several response actions, such as blocking an IP address (illustrates in **Figure 4.17**) , killing a process, quarantining a file, disabling a user, or isolating a host. Dedicated versions of the Active Response scripts were prepared for both Linux and Windows, since the execution method differs between the two operating systems. For example, blocking an IP address in Linux may be performed differently from Windows, and the same applies to killing processes, quarantining files, or disabling users.

```python
def delete_rules(argv, ip):
    in_name, out_name = rule_names(ip)
    run_netsh(argv, 'netsh advfirewall firewall delete rule name="%s"' % in_name)
    run_netsh(argv, 'netsh advfirewall firewall delete rule name="%s"' % out_name)


def add_rules(argv, ip):
    in_name, out_name = rule_names(ip)

    delete_rules(argv, ip)

    add_in = (
        'netsh advfirewall firewall add rule '
        'name="%s" dir=in action=block remoteip=%s enable=yes profile=domain,private,public'
        % (in_name, ip)
    )
    add_out = (
        'netsh advfirewall firewall add rule '
        'name="%s" dir=out action=block remoteip=%s enable=yes profile=domain,private,public'
        % (out_name, ip)
    )

    result_in = run_netsh(argv, add_in)
    result_out = run_netsh(argv, add_out)

    if result_in.returncode != 0 or result_out.returncode != 0:
        raise RuntimeError(
            "Failed adding firewall rules | in_rc=%s out_rc=%s" % (
                result_in.returncode, result_out.returncode
            )
        )

    verify_in = show_rule(argv, in_name)
    verify_out = show_rule(argv, out_name)
```

Figure 4.17 : Main Function in Block-IP Active-Response for Windows

After the action is executed, the Active Response scripts write the execution result into dedicated JSONL files. These files are then monitored through the agent.conf settings assigned to the linux-ar and windows-ar groups. As a result, the execution outcome is returned to Wazuh as a log that can be displayed and analyzed. This point is important because it makes the response process traceable; the system does not merely issue a command, but also records whether the execution succeeded, failed, or required rollback.

The role of **ai_action_ledger** appears here in building a centralized record of action states.(illustrates in **Figure 4.18**) It links the response alert, the execution attempt, the execution result, and the possibility of rollback. Accordingly, the analyst can determine the final status of each action: whether it remained as a recommendation, was executed, failed, or was rolled back. This layer adds important transparency to the system and supports later review of response decisions.

```
return {
    "recommendation_alert_id": recommendation_alert_id,
    "recommendation_rule_id": str(ensure_dict(alert.get("rule")).get("id", "")).strip(),
    "recommendation_timestamp": recommendation_timestamp,
    "response_id": str(data.get("response_id", self.response_id)).strip(),
    "target_agent_id": target_agent_id,
    "target_agent_name": target_agent_name,
    "target_platform": target_platform,
    "recommended_action": recommended_action,
    "response_command": response_command,
    "parameters": params,
    "dedup_key": compute_dedup_key(
        str(data.get("response_id", self.response_id)).strip(),
        target_agent_id,
        recommended_action,
        response_command,
        params,
    ),
    "auto_execute": str(ai_response.get("auto_execute", "False")).strip().lower() == "true",
    "execution_mode": str(ai_response.get("execution_mode", "")).strip(),
    "summary": str(ai_response.get("summary", "")).strip(),
    "confidence": str(ai_response.get("confidence", "")).strip(),
    "reasoning": str(ai_response.get("reasoning", "")).strip(),
    "policy_name": str(ai_response.get("policy_name", "")).strip(),
    "policy_reason": str(ai_response.get("policy_reason", "")).strip(),
    "response_rule_level": int(ensure_dict(data.get("response_rule")).get("level", 0) or 0),
    "source_level": int(source_ai_alert.get("level", 0) or 0),
    "source_description": str(source_ai_alert.get("description", "")).strip(),
    "source_ai_analysis": str(source_ai_alert.get("ai_analysis", "")).strip(),
    "manual_steps": [str(x).strip() for x in (ai_response.get("manual_steps", []) or []) if str(x).strip()],
    "evidence_to_collect": [str(x).strip() for x in (ai_response.get("evidence_to_collect", []) or []) if str(x).strip()],
    "delivery_status": "not_sent",
    "delivery_timestamp": "",
    "delivery_alert_id": "",
    "execution_status": "not_executed",
    "execution_timestamp": "",
    "execution_alert_id": "",
    "rollback_status": "not_requested",
    "rollback_timestamp": "",
    "rollback_alert_id": "",
    "latest_status": "planned",
    "latest_status_timestamp": recommendation_timestamp,
    "related_dispatch_events": [],
    "related_execution_events": [],
    "related_rollback_events": [],
```

Figure 4.18 : Record Structure in the Action Ledger for Action States.

4.5 Alert Identification in Wazuh and Activation of Custom Rules

After the RAG Detection and RAG Response alerts are injected into the Wazuh Manager, it becomes necessary to define custom rules capable of identifying these alerts. Therefore, the **rules.xml** file was modified to include specific rules for intelligent alerts. These rules do not rely only on general text patterns; rather, they depend on distinctive fields within the JSON structure, such as **ai_id** and **response_id**, which makes the detection process more accurate.

4.5.1 Rules for Capturing RAG Detection Alerts

RAG Detection rules rely on the presence of the ai_id field with the value 000_ai. When an alert containing this field is received, the base rule identifies it as an event generated by the intelligent analysis system. It is then classified according to the severity level specified in **ai_rule.level**. The severity levels were divided into categories. Low levels are treated as informational or low-severity events, medium levels as suspicious activity, high levels as significant security incidents, and **levels 14 and 15** as critical events. illustrates in **Figure 4.19**

This classification allows the model's evaluation to be incorporated into Wazuh's rule logic, enabling the intelligent alert to appear in the dashboard with an appropriate severity level. These rules also allow the model's description and analysis to be displayed within the alert details. As a result, the classification allows the model's evaluation to be incorporated into **Wazuh's rule logic**, enabling the intelligent alert to appear in the dashboard with an appropriate severity level.

```
  <rule id="200009" level="9">
    <if_sid>200001</if_sid>
    <field name="ai_rule.level">9</field>
    <description>  Strong attack attempt or partial compromise indicators: $(ai_rule.description)</description>
  </rule>

  <rule id="200010" level="10">
    <if_sid>200001</if_sid>
    <field name="ai_rule.level">10</field>
    <description>High risk - successful foothold indicators: $(ai_rule.description)</description>
  </rule>

  <rule id="200013" level="13">
    <if_sid>200001</if_sid>
    <field name="ai_rule.level">11|12|13</field>
    <description>Very high  AI THREAT: $(ai_rule.description)</description>
  </rule>

  <rule id="200015" level="15">
    <if_sid>200001</if_sid>
    <field name="ai_rule.level">14|15</field>
    <description>CRITICAL AI THREAT: $(ai_rule.description)</description>
  </rule>
```

Figure 4.19 : A Set of Rules for Determining the Alert Level Based on RAG Detection.

4.5.2 Rules for Capturing RAG Response Alerts

RAG Response alerts, on the other hand, are distinguished using the **response_id** field with the value **000_response_ai**. When this type of alert is received, the intelligent response rules identify it and determine whether it represents only a **manual recommendation** or an action that requires **automated execution**. If **auto_execute=False**, the alert is displayed as a response recommendation that the analyst can review and manually execute when needed. However, if **auto_execute=True** and a clear response_command is present, a dedicated rule triggers the **Active Response Dispatcher**. In this case, the response alert becomes not merely an informational message, but an actual execution trigger within Wazuh. . illustrates in **Figure 4.20**

```
<group name="ai_response,rag">
  <rule id="200019" level="2">
    <if_sid>200001</if_sid>
    <field name="response_id">000_response_ai</field>
    <field name="response_origin" type="pcre2">^(ai_response_engine|ai_control_api)$</field>
    <field name="ai_response.recommended_action" type="pcre2">.+</field>
    <description>AI response base rule</description>
  </rule>

  <rule id="200020" level="12">
    <if_sid>200019</if_sid>
    <field name="ai_response.auto_execute">True</field>
    <field name="ai_response.response_command" type="pcre2">.+</field>
    <description>AI response requires automatic execution</description>
    <group>ai_response,rag,auto_execute,</group>
  </rule>

  <rule id="200021" level="5">
    <if_sid>200019</if_sid>
    <field name="ai_response.auto_execute">False</field>
    <description>AI response recommendation</description>
    <group>ai_response,rag,manual_recommendation,</group>
  </rule>
</group>
```

Figure 4.20 : Custom Rule Set for RAG Response .

This separation between detection alerts and response alerts is highly important. The field **ai_id=000_ai** represents the result of event analysis, whereas **response_id=000_response_ai** represents the proposed or executable response decision.

4.5.3 Active Response Lifecycle Rules

In addition to the detection and response rules, dedicated rules were added to track the lifecycle of Active Response actions. These rules capture success, failure, and rollback logs generated by the execution scripts. When an action such as **block_ip**, **kill_process**, or **quarantine_file** is **successfully executed**, it is recorded as a success event. If **execution fails** for any reason, a failure event is generated. Similarly, if a **rollback operation succeeds or fails**, this is also recorded. illustrates in **Figure 4.21**

```
    <if_sid>200030</if_sid>
    <field name="event" type="pcre2">^(unblock_ip_success|enable_user_success|unisolate_host_success|restore_success)$</field>
    <description>AI action rollback executed successfully</description>
    <group>ai_action_rollback_success,</group>
  </rule>

  <rule id="200034" level="7">
    <if_sid>200030</if_sid>
    <field name="event" type="pcre2">^(unblock_ip_failed|enable_user_failed|unisolate_host_failed|restore_failed)$</field>
    <description>AI action rollback failed</description>
    <group>ai_action_rollback_failed,</group>
  </rule>

  <rule id="200035" level="3">
    <if_sid>200030</if_sid>
    <field name="event">quarantine_delete_ignored</field>
    <description>AI quarantine rollback requires manual restore</description>
    <group>ai_action_manual_restore,</group>
  </rule>

  <rule id="200031" level="5">
    <if_sid>200030</if_sid>
    <field name="event" type="pcre2">^(quarantine_success|block_ip_success|kill_process_success|disable_user_success|isolate_host_success)$</field>
    <description>AI action executed successfully on endpoint</description>
    <group>ai_action_success,</group>
  </rule>

  <rule id="200032" level="8">
    <if_sid>200030</if_sid>
    <field name="event" type="pcre2">^(quarantine_failed|block_ip_failed|kill_process_failed|disable_user_failed|isolate_host_failed)$</field>
```

Figure 4 21: Active Response Lifecycle Rules.

These rules enable the construction of a complete view of the response lifecycle. Instead of seeing only a recommendation, the analyst can track whether the recommendation was converted into an executed action, whether the execution was **actually successful**, and whether it was later rolled back. This enhances auditability and reviewability within the system.

4.5.4 Modifications of ossec.conf and Agent Groups

To ensure that this system operates in an integrated manner, several modifications were made to the ossec.conf file. The most important of these modifications included enabling log storage in JSON format, **opening port 5555** to receive Artificial Intelligence alerts **locally through TCP** (illustrates

in **Figure 4.22**), adding command definitions for Active Response, and linking some of these commands to specific rules. illustrates in **Figure 4.23**

```
<remote>
  <connection>syslog</connection>
  <port>5555</port>
  <protocol>tcp</protocol>
  <allowed-ips>127.0.0.1</allowed-ips>
  <local_ip>127.0.0.1</local_ip>
</remote>
```

Figure 4.22 : Opening Port 5555 Through the TCP Protocol.

```
  <!---------------------------------------------------------- dispatcher -->
  <command>
    <name>ai-response-dispatcher</name>
    <executable>ai_response_dispatcher.py</executable>
    <timeout_allowed>no</timeout_allowed>
  </command>

  <active-response>
    <disabled>no</disabled>
    <command>ai-response-dispatcher</command>
    <location>server</location>
    <rules_id>200020</rules_id>
  </active-response>
  <!---------------------------------------------------------- command active response linux -->
  <command>
    <name>ai-quarantine-file-linux</name>
    <executable>ai-quarantine-file.py</executable>
    <timeout_allowed>no</timeout_allowed>
  </command>

<command>
  <name>ai-block-ip-linux</name>
  <executable>ai-block-ip.py</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<command>
  <name>ai-kill-process-linux</name>
  <executable>ai-kill-process.py</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<command>
  <name>ai-disable-user-linux</name>
  <executable>ai-disable-user.py</executable>
  <timeout_allowed>no</timeout_allowed>
</command>
```

Figure 4.23 : Some command definitions for Active Response.

Two agent groups were also created: **linux–ar** and **windows–ar**. Each group contains its own **agent.conf** file, allowing the monitoring settings for Active Response result files to be distributed to the appropriate agents. The Linux-based agent is assigned to the **linux–ar group**, while the Windows-based agent is assigned to the **windows–ar group**. In this way, each agent receives the configuration suitable for its operating system. illustrates in **Figure 4.24**

```
i77@i77-server:~/AS-Platform/services$
i77@i77-server:~/AS-Platform/services$ sudo mkdir -p /var/ossec/etc/shared/linux-ar
sudo mkdir -p /var/ossec/etc/shared/windows-ar

sudo chown -R wazuh:wazuh /var/ossec/etc/shared
sudo find /var/ossec/etc/shared -type d -exec chmod 750 {} \;
sudo find /var/ossec/etc/shared -type f -exec chmod 640 {} \;
[sudo] password for i77:
i77@i77-server:~/AS-Platform/services$ ^C
i77@i77-server:~/AS-Platform/services$ sudo ls /var/ossec/etc/shared/
agent-template.conf  ar.conf  default  linux-ar  windows-ar
i77@i77-server:~/AS-Platform/services$ sudo nano /var/ossec/etc/shared/linux-ar/agent.conf
i77@i77-server:~/AS-Platform/services$ sudo nano /var/ossec/etc/shared/windows-ar/agent.conf
i77@i77-server:~/AS-Platform/services$ sudo /var/ossec/bin/agent_groups -a -g linux-ar -q
sudo /var/ossec/bin/agent_groups -a -g windows-ar -q
Error 1711: The group already exists: linux-ar
Error 1711: The group already exists: windows-ar
i77@i77-server:~/AS-Platform/services$ sudo /var/ossec/bin/agent_groups -a -i 001 -g linux-ar -q
Group 'linux-ar' added to agent '001'.
i77@i77-server:~/AS-Platform/services$ sudo /var/ossec/bin/agent_groups -a -i 005 -g windows-ar -q
Group 'windows-ar' added to agent '005'.
i77@i77-server:~/AS-Platform/services$
```

Figure 4.24 : Creating Agent Groups and Assigning Agents to Them.

These configurations include monitoring the JSONL files written by the Active Response scripts after actions are executed. As a result, execution outcomes are returned to Wazuh and appear within alerts and logs, thereby closing the loop between detection, response, and result verification. For example, see **Figure 4.25**, which contains the configuration settings for the Windows agents.

```
  GNU nano 6.2                                    /var/ossec/etc/shared/windows-ar/agent.conf
<agent_config>
  <localfile>
    <location>C:\Program Files\ossec-agent\active-response\ai-quarantine-file.jsonl</location>
    <log_format>json</log_format>
  </localfile>

  <localfile>
    <location>C:\Program Files\ossec-agent\active-response\ai-block-ip.jsonl</location>
    <log_format>json</log_format>
  </localfile>

  <localfile>
    <location>C:\Program Files\ossec-agent\active-response\ai-kill-process.jsonl</location>
    <log_format>json</log_format>
  </localfile>

  <localfile>
    <location>C:\Program Files\ossec-agent\active-response\ai-disable-user.jsonl</location>
    <log_format>json</log_format>
  </localfile>

  <localfile>
    <location>C:\Program Files\ossec-agent\active-response\ai-isolate-host.jsonl</location>
    <log_format>json</log_format>
  </localfile>
</agent_config>
```

Figure 4.25 : configuration settings for the Windows agents.

4.6 Displaying Alerts and Control Within the Wazuh Dashboard

After the RAG Detection and RAG Response alerts became part of Wazuh, it was necessary to create a clear display interface that helps the analyst distinguish these alerts from traditional ones. Therefore, Wazuh Dashboard was used to create customized dashboards for each type of alert, in addition to developing a floating button that enables direct interaction with the system from within the interface.

4.6.1 Creating a Dedicated Dashboard for RAG Detection Alerts

1. In the Discover section of the Wazuh interface, a filter was added to display only events containing the field **data.ai_id** with the value "**000_ai**". This filter effectively isolates all intelligent alerts generated by the AI system from the traditional alerts. Once the filter is applied, the analyst is presented with a list containing only these specific events, as shown in **Figure 4.26**.

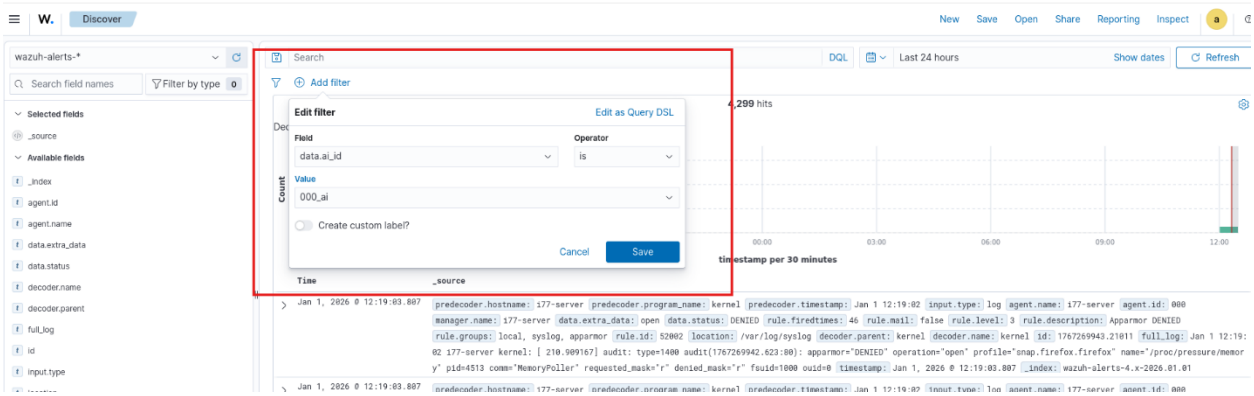


Figure 4.26 : Adding a Filter to Refine Alerts.

2. This filtered search was saved as a predefined query named "AI", so that it could be used as a reference in the dashboard components. Saving the search makes it easier to reuse the filtering criteria within visualizations.
3. We then moved to the Dashboards section and created a new dashboard named "**RAG**". Within this dashboard, several visual elements were added to provide a comprehensive and quick overview of **RAG Detection alerts**:

   Metric Element:This element displays the total number of generated alerts, for example: "Number of AI Alerts: 5." It was linked to the data from the saved query "AI Alerts", so that it counts only the documents or events that match the previously defined filter.

   To add this element, after accessing the interface for creating a new dashboard, we select Create New Object, as shown in **Figure 4.27**.

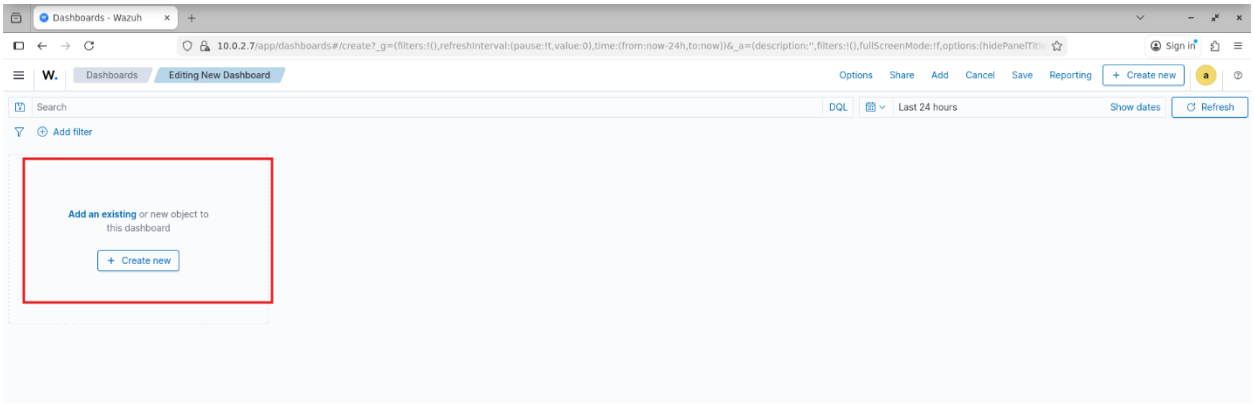


Figure 4.27 : Adding a New Object.

After that, we select the type of object to be added. In our case, Metric was selected, as shown in **Figure 4.28**.

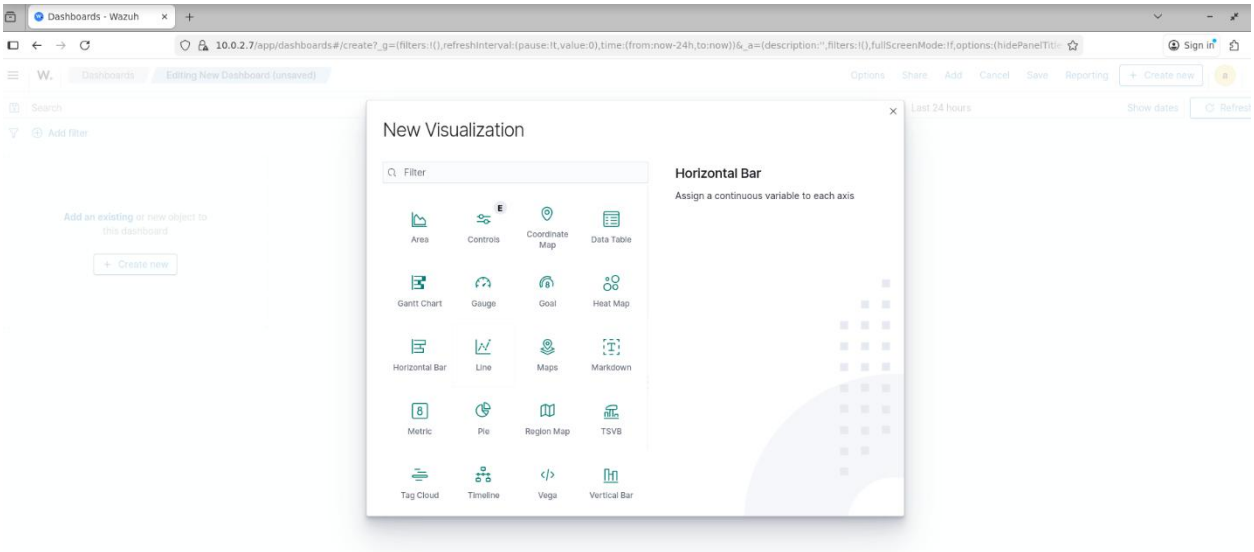


Figure 4.28 : Selecting the Type of Object to Be Added.

We then select the alert data source to be used. In our case, **wazuh–alerts–*** is selected to facilitate the display of the targeted alerts, as shown in **Figure 4.29**.

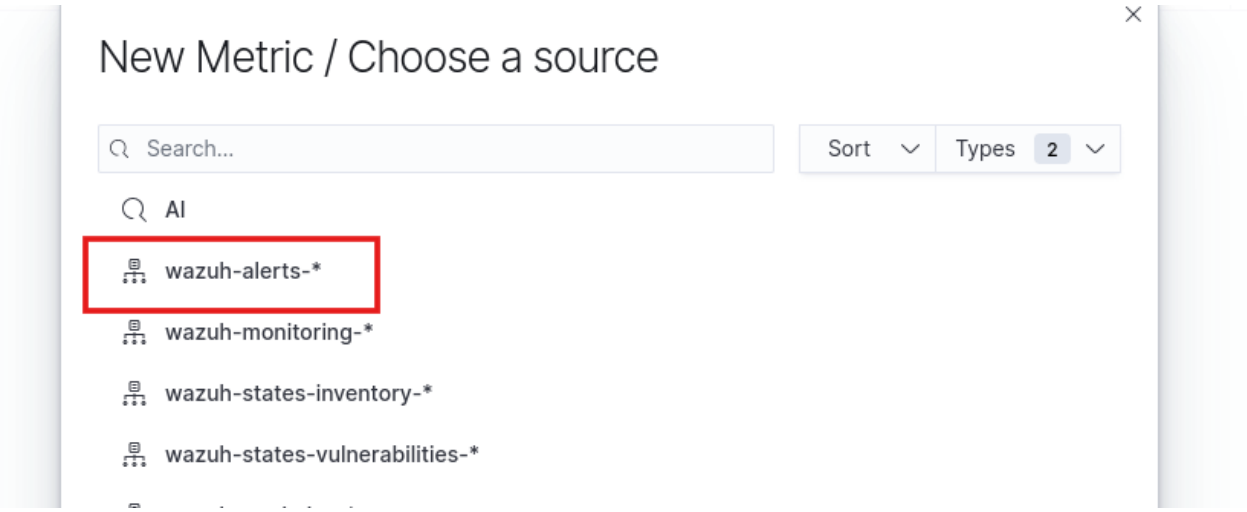


Figure 4.29 : Selecting the Alert Data Source.

After that, in the final step, we add a new Bucket from the side menu, then select Split Group and choose the Aggregation type. In our case, Terms was selected. Next, the Field is set to **data.ai_id**, and the visualization is saved, as shown in **Figure 4.30**.

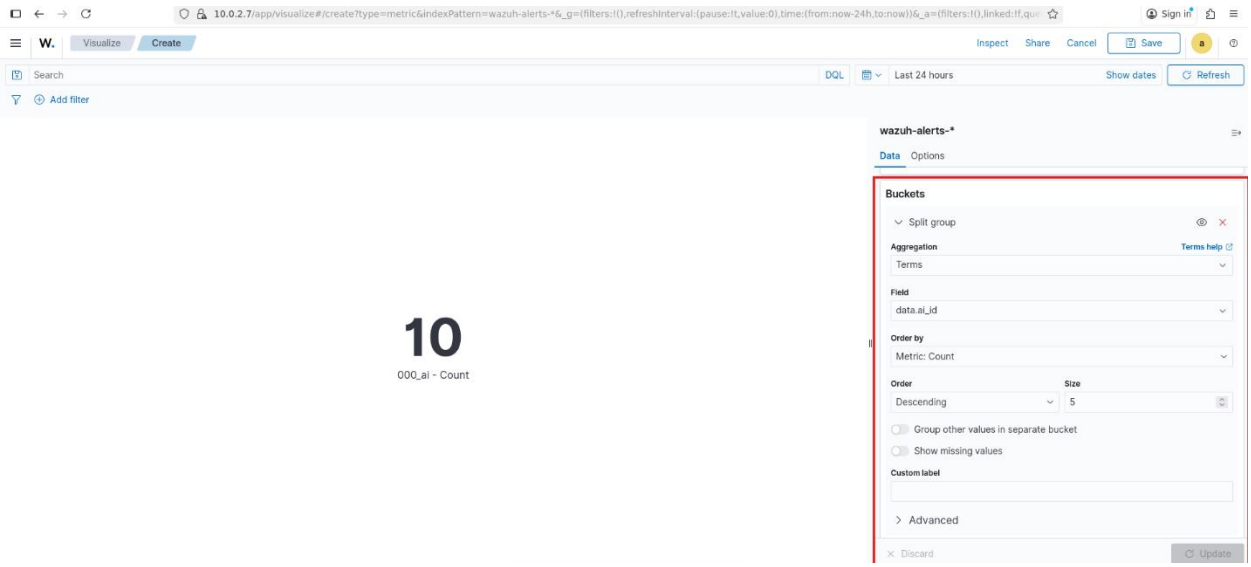


Figure 4.30 : Adding a Bucket and Specifying the Fields.

Pie Chart: A pie chart was used to present a statistical distribution of the intelligent alerts according to a specific classification. For example, the chart can be configured to divide the AI-generated alerts by severity level, allowing the analyst to observe the proportion of critical alerts versus medium-level alerts, and so forth, or alternatively by description type or group. In our implementation, this chart was configured to use the field **data.ai_rule.level** for data aggregation ,so that each segment

represents one severity level present in the intelligent alerts, with the size of the segment corresponding to the number of alerts at that level. This provides a rapid visual overview of the overall severity profile of the incidents most frequently reported by the AI system.

Data Table: A Data Table was used to display a detailed list of the most recent intelligent alerts. In this table, the alert description was included using the data.full_log field. The table updates automatically to display any new alert generated by the intelligent system, thereby providing analysts with a unified screen for monitoring the latest AI-generated alerts.

Each of the above elements was configured, in addition to incorporating the saved search "**AI Alerts**" as the data reference, which contains the same filtering condition: **ai_id = 000_ai**. Accordingly, we ensure that the dashboard displays only data related to AI-generated alerts, without being mixed with other alerts, as shown in **Figure 4.31**.

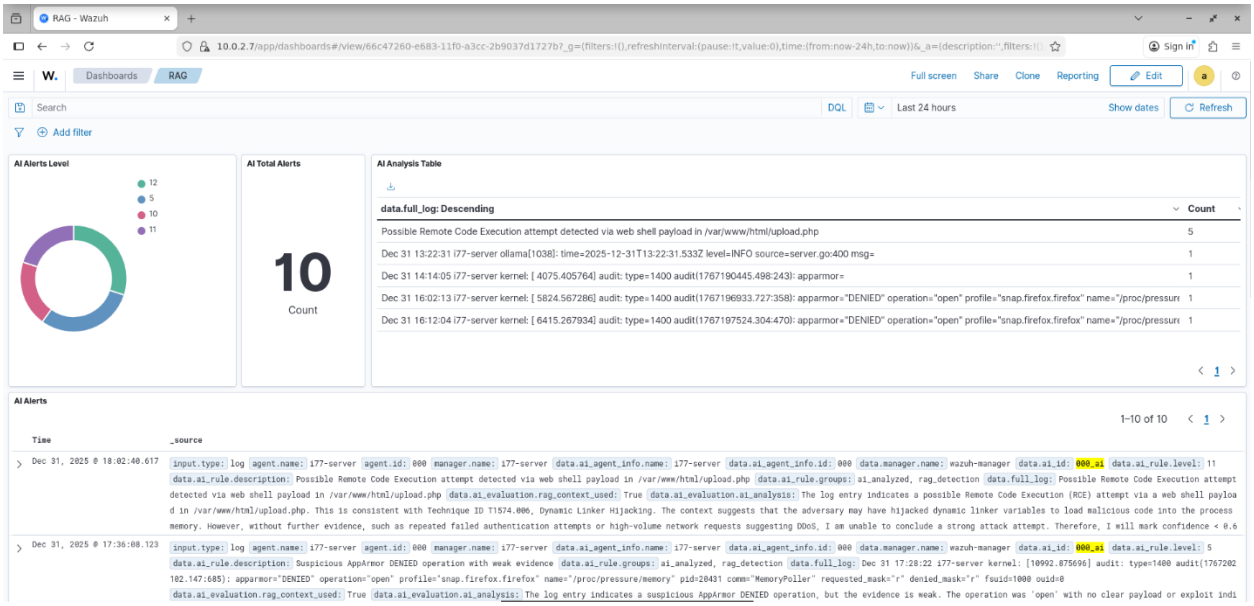


Figure 4.31: RAG Detection Dashboard.

Finally, the dashboard was saved and made available for monitoring. Security analysts can now open this dashboard at any time to observe the overall status of alerts generated by **RAG Detection**. They can determine whether there is an increase in the number of alerts through the metric counter, identify the dominant severity levels through the pie chart, and review the details of each incident through the data table. Analysts can also click on any record in the table to navigate to its full details

within the standard Wazuh interface, where the detailed analysis fields, such as those stored under **data.ai_evaluation**, are displayed.

4.6.2 Creating a Dedicated Dashboard for RAG Response Alerts

Similarly, another dashboard was created specifically for RAG Response alerts. The difference here is that the filter does not rely on **data.ai_id;** instead, it relies on the field **data.response_id** or **response_id**, depending on how it appears in Wazuh, with the value set to **000_response_ai**. After applying this filter, only the response alerts.

Within this dashboard, it is possible to display the number of response recommendations, the type of proposed action, whether the execution was automated or manual, the confidence level, the name of the targeted endpoint, and the policy reason that caused the system either to execute or not execute the action. A table can also be added to display important fields such as **recommended_action**, auto_execute, **execution_mode**, and **policy_reason**. as shown in **Figure 4.32**.

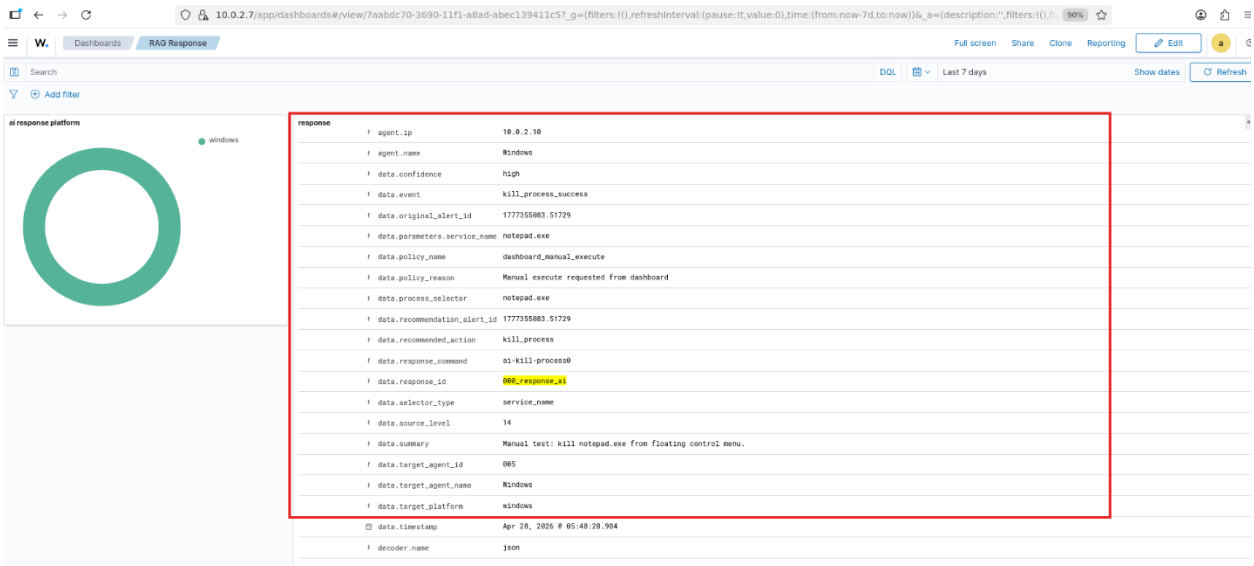


Figure 4.32 : RAG Response Dashboard.

This dashboard helps answer a question that differs from the one addressed by the Detection dashboard. The RAG Detection dashboard explains what the system has detected, whereas the RAG Response dashboard explains what the system has decided to do in response to that detection. As a

result, the analyst obtains a dual perspective: one view of the threat itself and another view of the proposed or executed response.

4.6.3 Flask Control API

The **ai_control_api.py** represents the software control layer within the system. It was built using Flask to act as an intermediary between the Wazuh Dashboard interface and the Artificial Intelligence and response components. This API provides several functions, such as checking the system status, retrieving response recommendations from the Action Ledger, executing a manual action, performing a rollback, and starting or stopping the RAG Detection and RAG Response services. The API also includes a dedicated route for chatting with the intelligent assistant, where the user can ask questions about an alert, an agent, or a response action. The system collects context from the alerts, the Ledger, and the Qdrant database, then sends the question to Llama 3.2 through Ollama. The model then returns a concise and structured answer. as shown in **Figure 4.33**.

This makes the API not only a control layer, but also an interpretation and assistance layer that supports the security analyst during investigation and response activities.

```
i77@i77-server:~/AS-Platform/services$ curl -sS -X POST "http://127.0.0.1:8777/api/ai/chat"   -H "Content-Type: application/json
"   --data-raw '{"question":"Give me a quick summary about the environment"}' | jq

{
  "agent_id": null,
  "alert_ref": null,
  "answer": {
    "answer": "The environment currently holds 59,744 alerts in the Wazuh indexer, with 3,160 of those loaded in the local archi
ve (covering the period from 2026-05-01T00:00:21.908Z to 2026-05-01T19:43:44.704Z). Alert severity distribution in the archive i
s dominated by \"unknown\" (2,945 alerts), with smaller numbers of level 7 (128), level 3 (51), level 2 (28), level 5 (6), level
 1 (2) and level 7 (128) events. All archived alerts come from a single agent (ID 000). The most frequent rule IDs are unknown (
2,945), 2904 (66 – “Dpkg (Debian Package) half configured.”), 2902 (44 – “New dpkg (Debian Package) installed.”), 1002 (28 – “Un
known problem somewhere in the system.”), 87101 (12 – “VirusTotal: Error: Public API request rate limit reached”), 5501 (12 – “P
AM: Login session opened.”), 550 (10 – “Integrity checksum changed.”), 5402 (10 – “Successful sudo to ROOT executed.”), 5502 (9
– “PAM: Login session closed.”) and 533 (5 – “Listened ports status (netstat) changed (new port opened or closed).”).",
    "confidence": "high",
    "recommended_next_step": "Review the high-severity (level 7) and unknown alerts in the archive to identify any critical issu
es, and consider expanding the archive coverage to include older alerts for a more complete historical view."
  },
  "ok": true,
  "response_id": null
}
```

Figure 4.33 : Requesting a Summary of the System .

The existence of this layer eliminates the need for direct manual interaction with scripts or log files, and allows interaction with the system to take place through a unified and monitored interface.

4.6.4 Floating Assistant Button Within the Wazuh Dashboard

A **custom plugin** named **wazuh_ai_assistant** was developed within **OpenSearch Dashboards**. Its purpose is to add a **floating button** inside the Wazuh interface. This button appears above the dashboard, and when clicked, it opens a side panel that contains functions related to Artificial Intelligence and incident response.

Build the Setup environment :

```
i77@i77-server:~/AS-Platform/services$ sudo apt-get update

i77@i77-server:~/AS-Platform/services$ sudo apt-get install -y git curl unzip
build-essential

i77@i77-server:~/AS-Platform/services$ curl -fsSL
https://deb.nodesource.com/setup_18.x | sudo -E bash -

i77@i77-server:~/AS-Platform/services$ sudo apt-get install -y nodejs

i77@i77-server:~/AS-Platform/services$ sudo npm install -g yarn
```

Then :

```
i77@i77-server:~/AS-Platform$ git clone --branch 2.19.3 --depth 1
https://github.com/opensearch-project/OpenSearch-Dashboards.git osd-2.19.3

i77@i77-server:~/AS-Platform$ cd /osd-2.19.3

i77@i77-server:~/AS-Platform/osd-2.19.3$ yarn osd bootstrap
```

Creating Plugin Structure :

```
i77@i77-server:~/AS-Platform$  mkdir -p /osd-2.19.3/plugins/wazuh_ai_assistant/public

i77@i77-server:~/AS-Platform$ mkdir -p /osd-2.19.3/plugins/wazuh_ai_assistant/server
```

After that, we customize the plugin as we want and add it to Wazuh. The interface includes several main tabs, such as Ask AI, Control, and Settings. In the Ask AI tab, the analyst can ask a question about a specific alert (as shown in **Figure 4.34**), agent, or security condition. In the Control tab (as shown in **Figure 4.35**), the analyst can view the current response recommendations and determine whether they can be executed manually or rolled back. The Settings tab allows the user to start or

stop the RAG Detection and RAG Response services, and change the LLM model, the maximum number of threads, and the minimum auto-response level.. as shown in **Figure 4.36**.

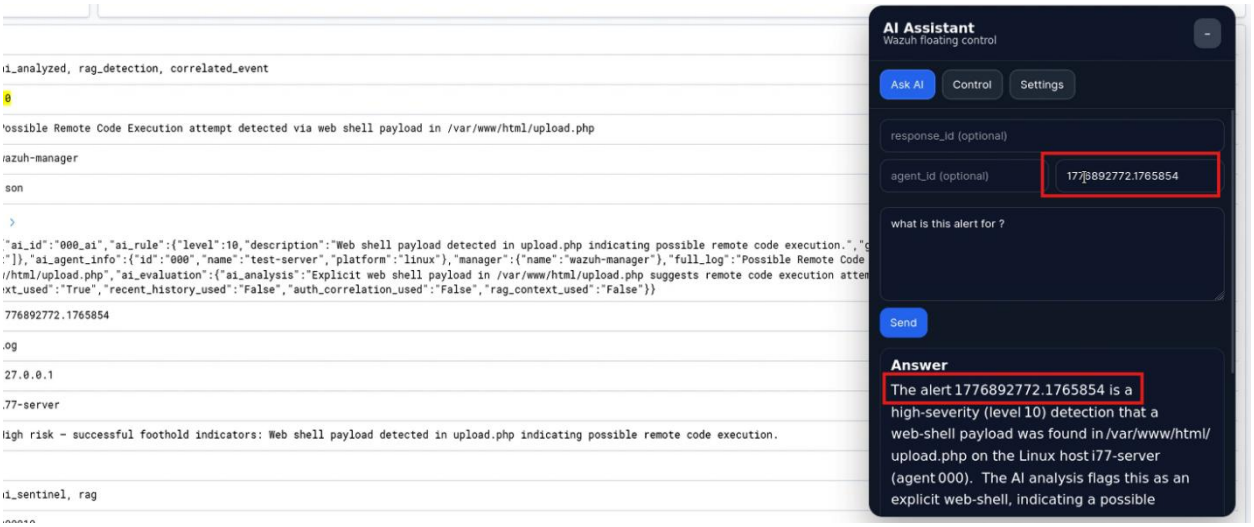


Figure 4.34 : ask a question about a specific alert .

Figure 4.35 : the Control tab .

Figure 4.36 : Settings tab.

This button relies on communication with the **Flask Control API**. It does not execute the security logic by itself; rather, it functions as a user interface that simplifies access to the system's functions. This design transforms the **Wazuh Dashboard** from merely a monitoring and visualization screen into an interactive control point for the intelligent security platform.

4.7 Complete Practical Workflow of the Proposed Solution

The complete practical workflow of the proposed solution can be summarized as follows: the attack begins from the **Parrot Security** machine and targets either a **Windows** or **Ubuntu** endpoint. The **Wazuh Agent** installed on the endpoint collects the logs generated by the attack and sends them to the **Wazuh Manager**. The **Wazuh Manager** then stores these logs and analyzes them according to traditional rule-based detection mechanisms. If a high-severity alert is generated, or if a suspicious raw log appears without a clear matching rule, it is forwarded to **RAG Detection**.

In the RAG Detection stage, the **ai_rag_detection** script builds a local context around the log, converts it into a vector using the BGE model, and searches **Qdrant** for semantically similar knowledge. The log and its associated context are then sent to **Llama 3.2** through **Ollama**, and the model returns a structured JSON alert containing **ai_id=000_ai**. This alert is injected into Wazuh

through **port 5555**, where it is captured by the custom **RAG Detection** rules and displayed in the dashboard.

After that, the **ai_rag_response** script monitors alerts containing **000_ai** and analyzes them from a response perspective. It extracts incident indicators, searches the **response knowledge** base in Qdrant, and then sends a prompt to the language model to generate a response plan. The generated plan passes through the **Policy Gate** to determine whether it should be executed automatically or remain as a manual recommendation. A response alert containing **response_id=000_response_ai** is then injected into Wazuh.

If the response alert requires automated execution, the Wazuh rules trigger the Active Response Dispatcher, which selects the appropriate command and executes it on the targeted endpoint. The Active Response scripts then record the execution result in **JSONL** files, and these results are returned to Wazuh. Finally, the Action Ledger consolidates the state of each action, while the analyst can monitor the entire process through the dashboard or the floating assistant button.

4.8 Security and Configuration Considerations

Although the proposed solution provides a high degree of automation, it was designed with several security considerations in mind. The first of these considerations is that the alert injection port, 5555, operates only on 127.0.0.1, which reduces the possibility of sending forged alerts from outside the server. In addition, AI-generated alerts are ignored during log monitoring to prevent the system from reanalyzing its previous outputs and entering an infinite loop. The detection and response prompts were also configured to prevent the model from inventing facts that are not present in the log. If no IP address exists, the model must not assume the presence of a remote attacker. If no file path is present, it must not recommend file quarantine. Similarly, if the retrieved RAG context is not relevant, it must not be used in the decision-making process. These rules reduce hallucination and make the model outputs more trustworthy.

Nevertheless, the system still requires continuous tuning. It is important to review the quality of the knowledge base, improve the prompts, monitor auto_execute decisions, and test response actions in an isolated environment before using them in a real production environment. In the future, the system can be further enhanced by adding live threat intelligence sources, using a larger model, or expanding the set of supported Active Response actions.

4.9 Conclusion

This chapter provided a detailed description of the proposed solution in the project, which is based on enhancing the Wazuh platform with two intelligent layers: RAG Detection and RAG Response. The system begins with the execution of attacks within a virtual environment, followed by log collection through the Wazuh Agent, transmission to the Wazuh Manager, and the selection of suitable events for intelligent analysis. After that, RAG Detection analyzes the log, enriches it with context retrieved from Qdrant, and uses Llama 3.2 to generate an intelligent alert and inject it into Wazuh. The chapter also explained how the RAG Response stage analyzes detection alerts, retrieves response-related knowledge, generates an appropriate response plan, and then determines whether the plan should be executed automatically or remain as a manual recommendation. The roles of the Active Response Dispatcher, the Linux and Windows scripts, the Action Ledger, the Flask Control API, and the floating button within the Wazuh Dashboard were also discussed, in addition to the custom rules and dashboards dedicated to alerts identified by 000_ai and 000_response_ai.

This solution represents a transition from a traditional SIEM system based on static rules to an intelligent security platform capable of contextual understanding, alert reassessment, false-positive reduction, and the generation of executable and traceable response recommendations. Accordingly, this chapter prepares the ground for the following chapter, in which the system will be tested and its practical results will be analyzed through various attack scenarios.

5. Chapter Four: System Testing and Results Analysis

5.1 Introduction

This chapter aims to evaluate the performance of the developed security platform, which is built on the integration of Wazuh with RAG technologies, Large Language Models (LLMs), and Active Response mechanisms. After explaining the architecture of the proposed solution and its software components in the previous chapter, it becomes necessary to test the system practically through real or semi-real attack scenarios in order to verify the platform's ability to detect security events, analyze them, classify them, generate appropriate response recommendations, and execute certain containment actions when the conditions for automated execution are satisfied. The tests were designed to cover several aspects of cybersecurity, particularly the CIA model, which focuses on Confidentiality, Integrity, and Availability. In addition, a Windows-specific compromise scenario using Meterpreter was included to simulate an actual or semi-actual control case over an endpoint device. This diversity of tests enables the system to be evaluated from multiple perspectives. Therefore, the evaluation is not limited to detection capability alone, but also includes the quality of analysis, the suitability of response recommendations, and the success of execution or rollback when required.

The testing methodology relied on documenting each scenario through a set of evidence types, namely: the Ground Truth of the attack executed within the lab, the Raw Evidence from the Wazuh environment or endpoint, the AI Evidence representing the Artificial Intelligence analysis, the Dashboard Evidence from the interface or dashboard, and the Verification Evidence confirming the success of the action. Thus, the evaluation does not depend on a single observation, but rather on an interconnected chain of evidence that begins with attack execution and ends with verifying the impact of the response.

5.2 Evaluation Methodology

The system evaluation process was based on the execution of four main test cases, each representing a different type of threat. The first case represents a threat to confidentiality through an attempt to gain unauthorized access to a sensitive file. The second case represents a threat to availability through

the execution of a Denial-of-Service (DoS) attack using the hping3 tool. The third case represents a Windows compromise scenario through the creation of a suspicious process that resembles Meterpreter-like behavior. The fourth case represents a threat to system integrity through the upload of a malicious Web Shell file within the web directory.

Each case was evaluated according to the following elements illustrates in **Table 5.1**.

Table 5.1: system evaluation elements.

Evaluation Element	Description
Ground Truth	What was actually executed within the test environment
Raw Evidence	The raw evidence before Artificial Intelligence analysis
AI Evidence	The output of RAG Detection and RAG Response
Dashboard Evidence	The appearance of the result within the Wazuh Dashboard or the custom interface
Verification Evidence	Evidence confirming the success of the response action or the success of the rollback
Result	The final outcome: Passed, Partially Passed, or Failed

This template makes the evaluation clearer and more systematic, as it links what was actually executed with what the system detected, what it recommended, and what was verified as successful. This approach also highlights the difference between merely generating an alert and producing a response that is measurable and verifiable.

5.3 Test One: Unauthorized Access Attempt to a Sensitive File

This test represents the Confidentiality aspect of the CIA model. The objective of this test is to verify the system's ability to detect an unauthorized access attempt to a sensitive file, as well as an attempt to modify that file on an Ubuntu machine running a Wazuh Agent. A sensitive file was created in the following path:

```
/opt/confidentiality_lab/customer_secrets.csv
```

It was then accessed through a user who did not have the required permission to read it, using the following command:

```
sudo -u confidentiality_test cat /opt/confidentiality_lab/customer_secrets.csv
```

Figure 5.1 illustrates the execution of the attempted access to the sensitive file from an unauthorized account. The result shows that the user was unable to read the file due to insufficient permissions. Although the operating system successfully prevented the access attempt, the event remains significant from a security perspective because it represents an attempt to access sensitive data from an unauthorized account.

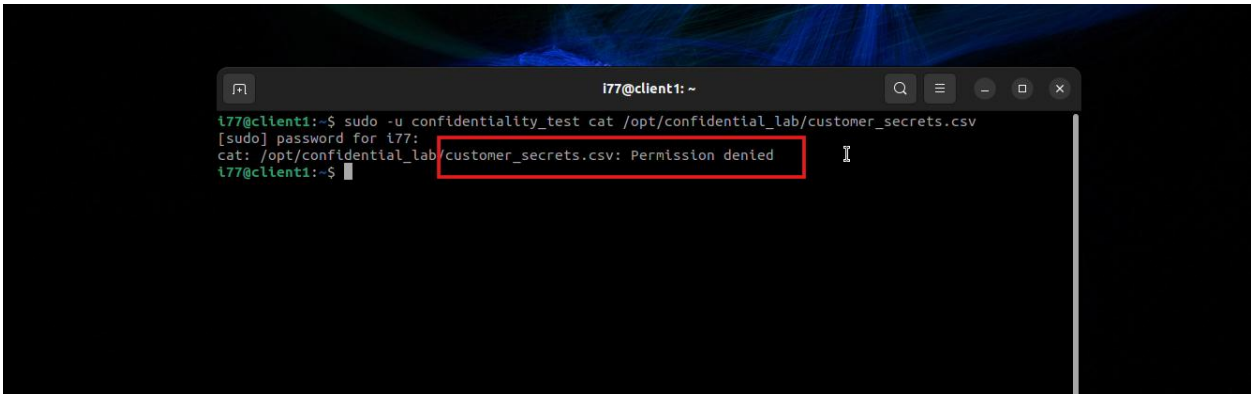


Figure 5.1 : Attempt to Read a Sensitive File by an Unauthorized User.

After the event was recorded by the Wazuh Agent and sent to the Wazuh Manager, the RAG Detection system analyzed the log and correlated it with the confidentiality context. **Figure 5.2** shows the result of the Artificial Intelligence analysis, where the system classified the event as an unauthorized access attempt to a sensitive file. The image also shows the Floating Button, which displays a proposed response action, such as disabling the user account or reviewing the account associated with the attempt.

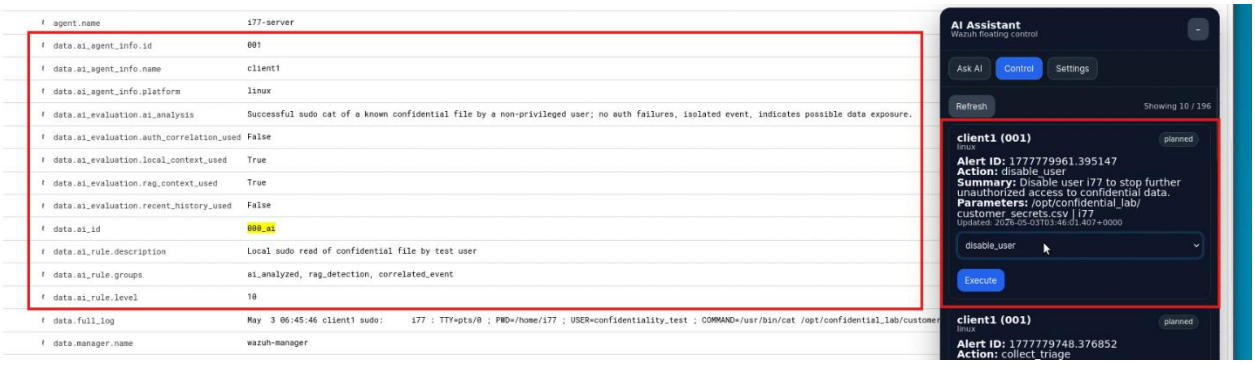


Figure 5.2: RAG Detection Analysis of the Event and the Appearance of a Proposed Manual Response Action.

After reviewing the recommendation, a manual action was executed by disabling the user account associated with the attempt. **Figure 5.3** shows that the account was disabled, as it no longer appeared among active accounts or was no longer able to log in. This result confirms that the manual response recommended by the system was executable and produced a clear operational effect.

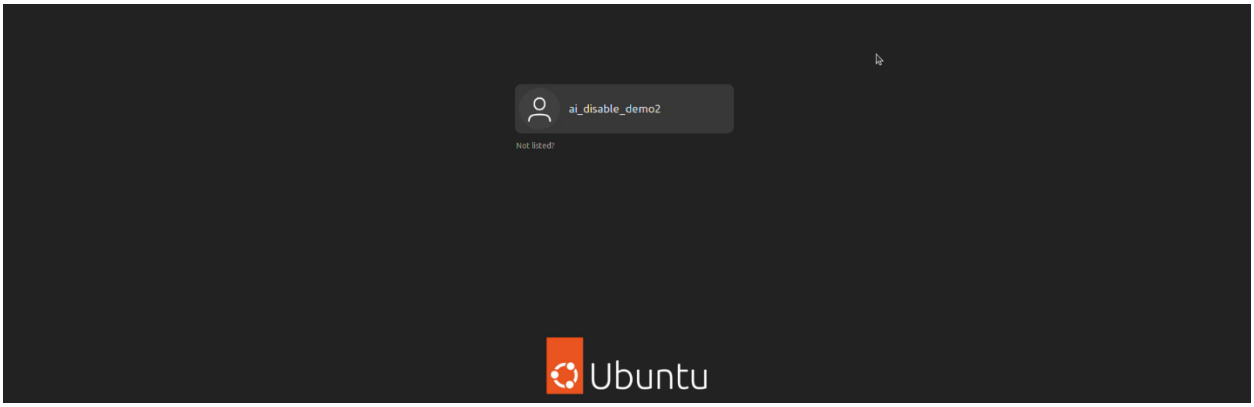


Figure 5.3: Verification of User Account Disabling After Executing the Response Action.

To verify that the system does not only execute the action, but also supports rollback when needed, a rollback operation was performed to restore the account to its previous state. **Figure 5.4** shows that the account became active again after the action was reverted, confirming that the control mechanism is not limited to response execution, but also includes the ability to restore the previous state when necessary.

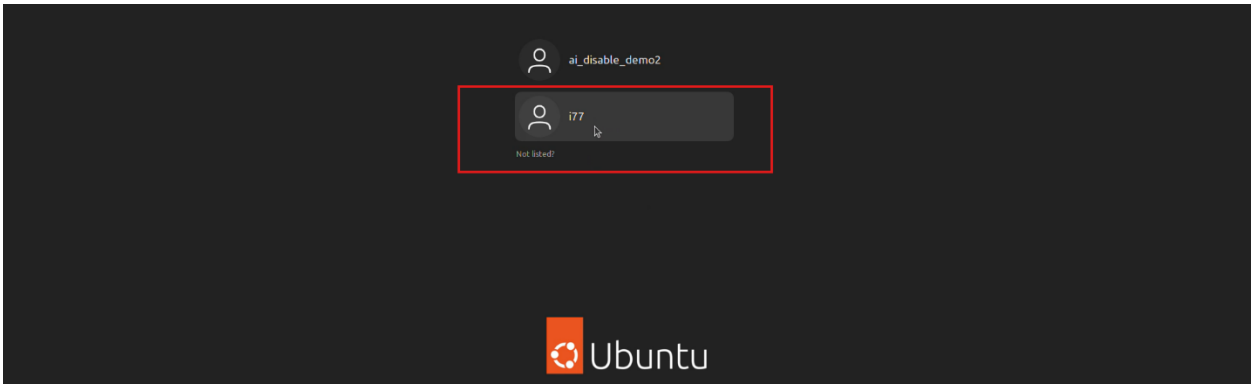


Figure 5.4: Successful Rollback Operation and Restoration of the Account After Reverting the Action.

5.3.1 Test One : Result Analysis

This test demonstrated that the system is capable of handling confidentiality-related events in a contextual manner. The system did not treat the event merely as a normal permission-denied

message; rather, it correlated it with an attempt to access a sensitive file, which increased the significance of the event from a simple permission error to a security indicator requiring review. The RAG Response recommendation to disable the user or investigate the account was also appropriate, as the user attempted to access a sensitive resource outside their authorized permissions.

It should be noted that the response in this scenario was manual rather than automatic, because the event does not represent a confirmed compromise by itself, but rather a rejected access attempt. This reflects the safety of the system's policy, as it did not execute an immediate automated action that could affect users without human review. Instead, it provided a clear recommendation to the security administrator, with the ability to execute it or roll it back if needed.

Based on the previous evidence, the test result can be considered Passed, as the system detected the event, correctly classified it under confidentiality impact, provided an appropriate recommendation, verified the effect of the action after execution, and successfully tested the rollback mechanism.

5.4 Test Two: Denial-of-Service Attack

This test represents the Availability aspect of the CIA model. Its objective is to verify the system's ability to handle an aggressive attack aimed at disrupting a service on one of the endpoint devices. The attack was executed from the attacker machine using the hping3 tool through the following command:

```
sudo hping3 --flood -R 10.0.2.15 -p 80
```

This command aims to send a large number of packets toward port 80 on the target machine, which may lead to service flooding or negatively affect its ability to respond. **Figure 5.5** illustrates the execution of the attack from the Parrot Security machine, where intensive traffic was sent toward the targeted endpoint.

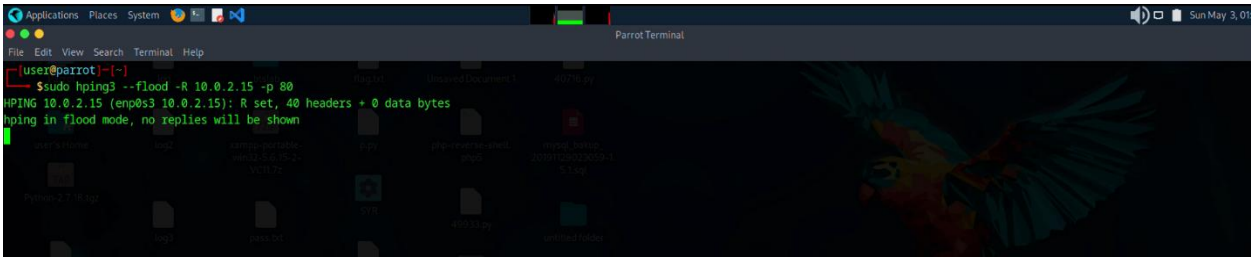


Figure 5.5: Executing a DoS Attack Using hping3 Against Port 80.

After the abnormal traffic was recorded, the **RAG Detection system** analyzed the event and classified it as an attack affecting service availability. Due to the severity of the attack and its clear impact on the service, the **RAG Response system** treated it as a critical case requiring automated response. **Figure 5.6** shows that the system did not merely display a recommendation; rather, it activated the automated response to block the attacker's IP address. The interface also displays a Rollback button, which allows the security administrator to reverse the AI decision if needed.

Figure 5.6: Classifying the Attack as an DoS Attack and Automatically Executing the Block Action with the Rollback Button Displayed.

To verify the success of the execution, the firewall rules on the agent machine were inspected. **Figure 5.7** shows that the response script added new rules to iptables to block the attacker's IP address. This provides direct evidence that the RAG Response decision did not remain merely an alert or recommendation, but was transformed into an actual containment action on the endpoint.

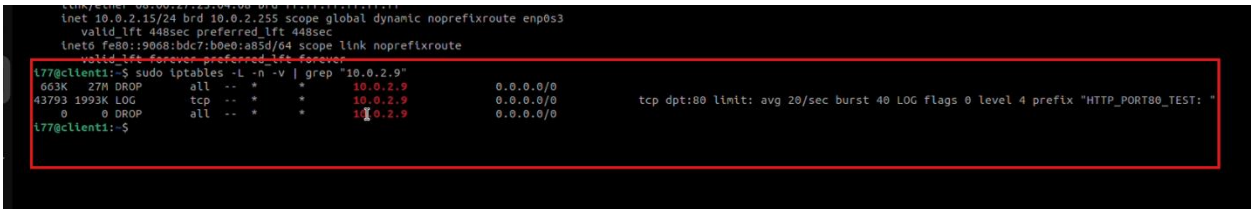


Figure 5.7: Appearance of IP-Blocking Rules in iptables After the Automated Response Execution.

After that, a rollback operation was executed to reverse the block action. **Figure 5.8** shows that the rules added to iptables were removed after the rollback, confirming that the response mechanism supports not only execution, but also restoration of the system to its previous state.

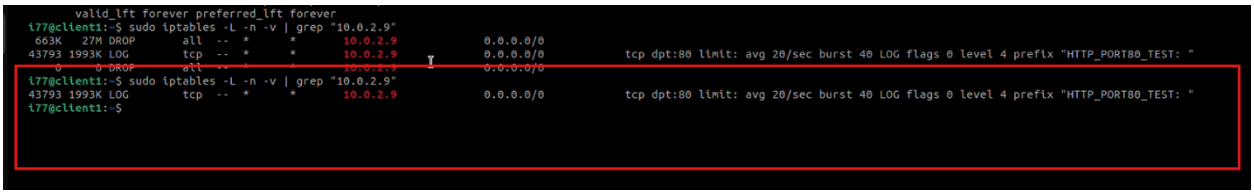


Figure 5 8:Removal of Blocking Rules After Executing the Rollback Operation.

5.4.1 Test Two : Result Analysis

This test demonstrated the system's ability to handle availability-related attacks more decisively than in the previous test. In this case, the event was not merely a rejected attempt, but an active and aggressive attack that affected service **availability**. Therefore, the system's decision to execute an automated response was more justified, as delays in such cases may allow the service disruption to continue. The test also highlights the importance of the **Policy Gate** within the system. Automated execution occurred only because the severity level was high and because the proposed action was clear and limited in impact, namely blocking the attacker's IP address. This type of action is suitable for automated execution in **DoS attacks**, particularly when the source is clear and the attack is ongoing. The test result shows that the system successfully moved from detection and analysis to actual containment. Since the blocking action was successfully executed, verified through iptables, and then successfully rolled back, the test result is considered Passed.

5.5 Test Three: Windows Compromise Using Meterpreter

This scenario represents a compromise or control case involving a Windows machine running a Wazuh Agent. The objective of this test is to verify the system's ability to detect suspicious activity associated with the creation of a new process on the targeted device. In this scenario, the **attacker** machine successfully **gained access** to the **Windows machine** through an offensive technique, and then created a new process cloned from **notepad.exe** and executed it in a suspicious manner to simulate **Meterpreter** behavior or behavior similar to it.

Figure 5.9 illustrates the attacker's activity on the Windows machine, where the created and executed process appears within the victim environment. This type of activity is significant from a security perspective because running an unusual process, or a process cloned from a legitimate one, may indicate an attempt to conceal malicious activity or to use a legitimate process as a cover.

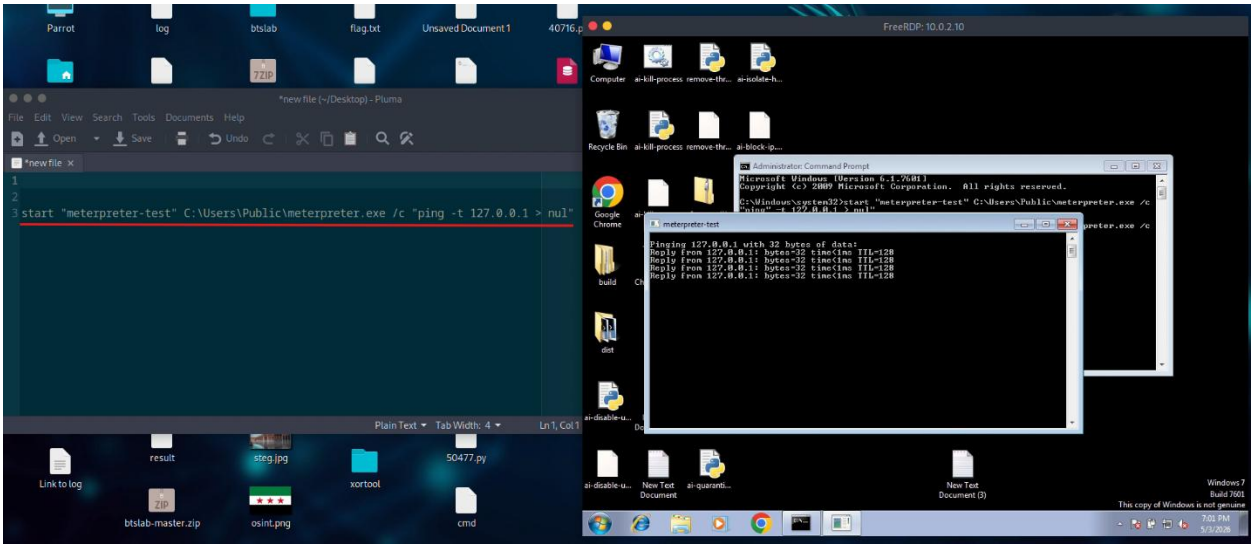


Figure 5.9 : Creating a Suspicious Process on the Windows Machine to Simulate Meterpreter Behavior.

After the event was recorded and analyzed, the RAG Detection system classified the activity as a potential threat on the Windows machine. The RAG Response system then generated an appropriate recommendation, suggesting the termination of the suspicious process through **kill_process**. It also proposed an additional action, namely isolating the host from the network through **isolate_host**, in case the severity of the incident required such containment. **Figure 5.10** shows the analysis and recommendation displayed within the interface.

Figure 5.10: Threat Detection and Recommendation to Terminate the Process or Isolate the Host.

After reviewing the recommendation, the process termination action was executed manually. **Figure 5.11** shows that the process created by the attacker was successfully stopped. This verifies that the system's recommendation was not merely theoretical, but was executable and produced a clear effect on the targeted machine.

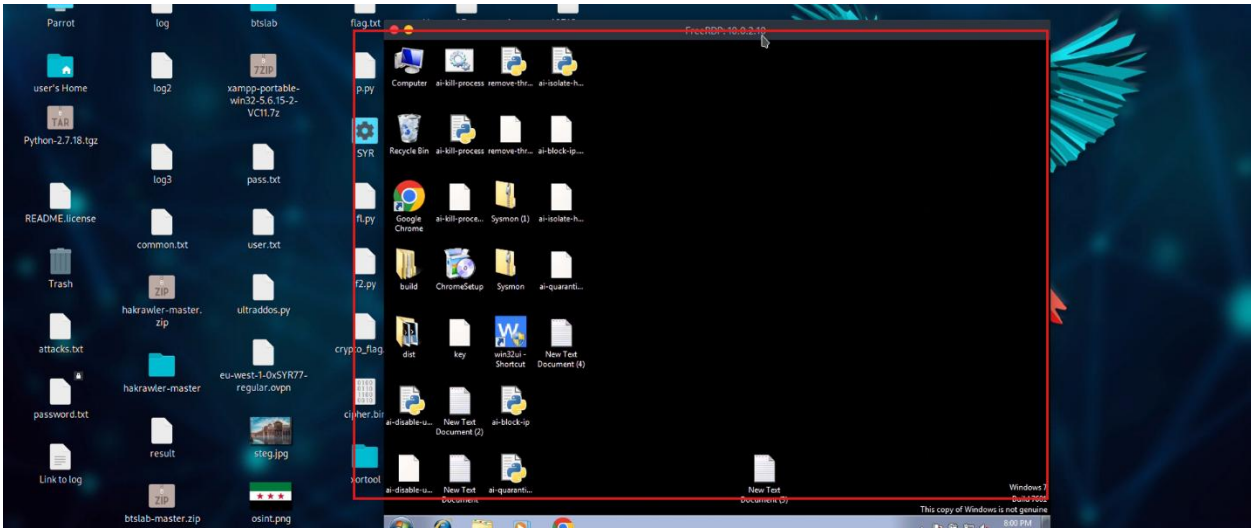


Figure 5.11: Successful Execution of kill_process and Termination of the Suspicious Process.

5.5.1 Test Three : Result Analysis

This scenario demonstrated the system's ability to handle indicators of compromise within a Windows environment. The system did not merely identify the creation of a new process; rather, it correlated the activity with a security context indicating the possibility of compromise or **malicious behavior**. The recommendation to terminate the process was also appropriate, as it targeted the direct impact of the attack without disabling the entire device. It is also important to note that the system suggested host isolation as an additional option, but did not execute it automatically. This reflects a suitable response logic, as isolating a host is a strong action that may affect business operations and should therefore be associated with a higher confidence level or an administrative decision. In contrast, terminating the process is a more specific action and can be executed once the process is confirmed to be suspicious.

Accordingly, the test can be considered Passed, as the system detected the activity, classified it as a threat, recommended appropriate response actions, and successfully executed and verified the kill_process action.

5.6 Test Four: Uploading a Malicious Web Shell

This test represents the Integrity aspect of the CIA model. Its objective is to verify the system's ability to detect an unauthorized modification within the web directory, specifically the upload of a malicious Web Shell file. Initially, **Figure 5.12** shows that the web directory on the target machine did not contain any suspicious files before the attack was executed, thereby providing a clear baseline for comparison after the attack.

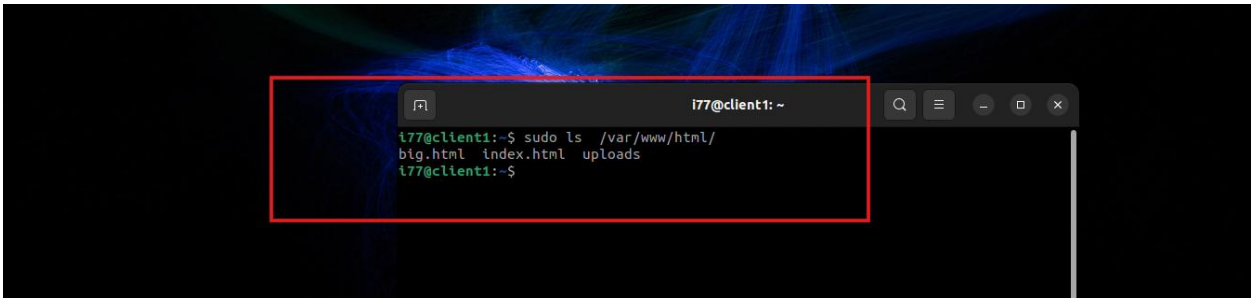


Figure 5.12: Web Directory Before Executing the Attack, Showing No Suspicious File Present.

After that, the attacker attempted to obtain unauthorized privileges by exploiting the SSH service, and then uploaded a malicious file containing a Web Shell or a reverse shell into the web directory. **Figure 5.13** illustrates the attacker's attempt to perform this step within the attack scenario.

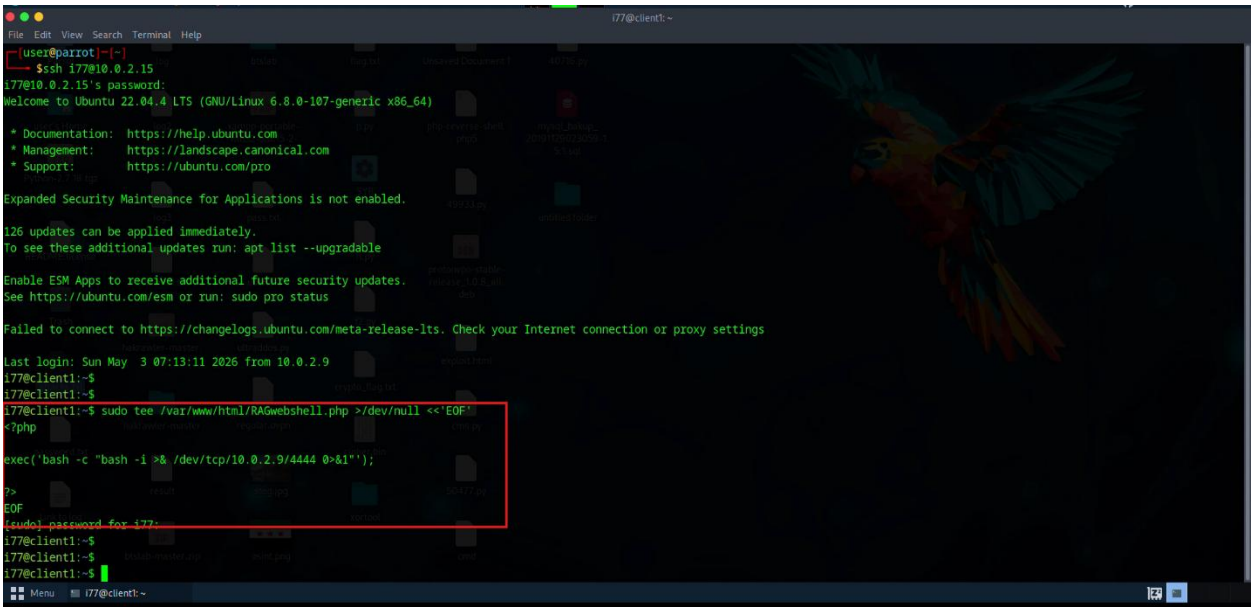


Figure 5.13: Attacker Attempt to Upload a Malicious Web Shell File by Exploiting Unauthorized Access.

After the attack was executed, the malicious file became present within the web directory, as shown in **Figure 5.14**. This indicates that an unauthorized modification occurred within the application environment, representing a direct threat to system integrity, because the presence of a Web Shell allows the attacker to execute commands or maintain persistent access.

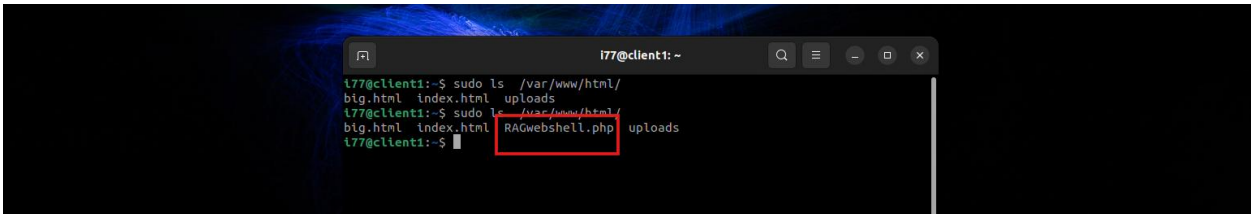


Figure 5.14: Appearance of the Malicious File Within the Web Directory After the Attack.

The Wazuh system detected the presence of the malicious file, after which RAG Detection analyzed the event and classified it as a threat related to a Web Shell. Subsequently, RAG Response proposed an appropriate action, namely **quarantine_file**, to isolate the malicious file. **Figure 5.15** illustrates the RAG Detection analysis and the appearance of the RAG Response recommendation.

Figure 5.15 : RAG Detection Analysis and Recommendation to Quarantine the Malicious File.

After the file quarantine action was executed, the web directory was examined again for verification. **Figure 5.16** shows that the malicious file was no longer present in the directory, thereby confirming the successful execution of the quarantine or isolation procedure.

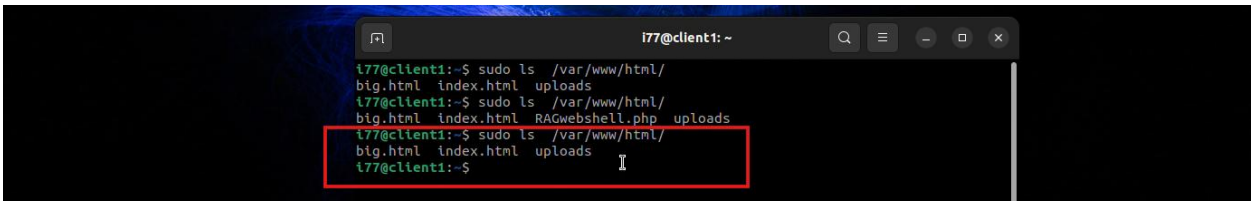


Figure 5.16 : Disappearance of the Malicious File from the Web Directory After Executing quarantine_file.

5.6.1 Test Four : Result Analysis

This test demonstrated the system's ability to detect a threat affecting the integrity of application files. The presence of a malicious file within the web directory does not merely represent a new file; rather, it may function as a **backdoor** that allows the attacker to execute commands or maintain a reverse connection. Therefore, the system's classification of the event as a **Web Shell** threat was appropriate, and the recommendation to quarantine the malicious file was logical and limited in impact. This scenario was distinguished by the fact that the response was clearly verifiable. Before the attack, the directory was clean; after the **attack**, the malicious file appeared; and after the response action was executed, the file disappeared from the web directory. This sequence provides strong evidence of the system's success, not only in detection, but also in removing the direct impact of the attack.

Accordingly, the test result is considered Passed, as the system detected the malicious file, classified it correctly, recommended file quarantine, executed the action, and verified the disappearance of the file from the directory.

5.7 Test Results Summary

Table 5.2 presents a general summary of the results of the four tests. The table shows that the system successfully detected all executed attacks, classified them correctly, and provided an appropriate response recommendation for each case. The success of the response actions, or their rollback when needed, was also verified.

Table 5.2: Summary of Practical Test Results.

Test Case	Security Impact	RAG Detection Analysis	RAG Response	Verification Evidence	Result
Unauthorized access attempt to a sensitive file	Confidentiality	Correct	Disable user / investigate account	User disabled, followed by successful rollback	Passed

DoS attack	Availability	Correct	Automatically block IP address	iptables rule added, then removed after rollback	Passed
Meterpreter on Windows	Multi-impact	Correct	Kill process / isolate host	Process successfully terminated	Passed
Web Shell upload	Integrity	Correct	Quarantine malicious file	File disappeared from the web directory	Passed

Based on the previous table, it can be observed that the system was not limited to generating alerts; rather, it provided an explanation, a recommendation, or a verifiable response action. The results also demonstrated the system's ability to handle multiple types of threats, whether related to confidentiality, integrity, availability, or compromise of a Windows endpoint

5.8 Time and Performance Evaluation

In addition to evaluating detection accuracy and response quality, the time required for analysis in both the **RAG Detection** and **RAG Response** stages was measured. Analysis time here refers specifically to the time associated with the **RAG/LLM** stage; that is, the time the system takes to send the request to the language model and receive the result. This time does not include human decision-making time in manual response cases, as such time depends on the security administrator rather than on the efficiency of the system itself.

Table 5.3 shows the analysis time for each test case.

Table 5.3: Analysis Time for RAG Detection and RAG Response.

Test Case	RAG Detection Time	RAG Response Time	Total AI Analysis Time
Confidentiality / Unauthorized File Access	27 seconds	23 seconds	50 seconds
DoS / Availability Attack	25 seconds	18 seconds	43 seconds
Meterpreter on Windows	26 seconds	24 seconds	50 seconds
Web Shell / Integrity Attack	20 seconds	15 seconds	35 seconds

These times represent the most significant part of the overall processing time, since the other stages—such as reading the log, querying Qdrant, injecting the alert into Wazuh, and executing the response script—typically take only fractions of a second and are negligible compared with the language model inference time. As for manual execution time, it depends on the speed of the security administrator's decision; therefore, it was separated from the system's processing time.

Table 5.4 presents the overall performance averages.

Table 5.4: Analysis Time Metrics.

Metric	Value
Average RAG Detection Time	24.5 seconds
Average RAG Response Time	20 seconds
Average Total AI Analysis Time	44.5 seconds
Highest Recorded Analysis Time	50 seconds
Lowest Recorded Analysis Time	35 seconds

The results indicate that the average total analysis time reached 44.5 seconds, which is considered acceptable in a test environment relying on a locally hosted Large Language Model. The highest recorded time was 50 seconds in the Confidentiality and Meterpreter scenarios, while the fastest scenario was the Web Shell test, with a total analysis time of 35 seconds.

5.8.1 Technical MTTR Evaluation

The approximate technical response time can be expressed using the following equation:

Technical MTTR = RAG Detection Time + RAG Response Time + Execution Time

Since the actual execution time for actions such as blocking an IP address, quarantining a file, or terminating a process was approximately less than one second, the dominant factor affecting the response time is the inference time of the language model. See **Table 5.5**.

Table 5.5: Estimated Technical MTTR.

Test Case	AI Analysis Time	Response Type	Approximate Execution Time	Approximate Technical MTTR
Confidentiality	50 seconds	Manual action	Depends on the administrator's decision + less than 5 seconds for execution	50 seconds + administrator decision time
DoS	43 seconds	Automated response	Less than 5 seconds	Approximately 48 seconds
Meterpreter	50 seconds	Manual action	Depends on the administrator's decision + less than 5 seconds for execution	50 seconds + administrator decision time
Web Shell	35 seconds	Manual action	Depends on the administrator's decision + less than 5 seconds for execution	35 seconds + administrator decision time

These results show that the **DoS** scenario achieved the best performance in terms of reducing **MTTR**, because the response was executed automatically without waiting for human decision-making. In the manual scenarios, the system provided analysis and recommendations, but the final processing time depended on how quickly the security administrator interacted with the recommendation.

5.9 Comparison Between the Proposed System and Traditional SIEM

The test results show that the fundamental difference between a traditional system and the proposed system (as show in **Table 5.6**) does not lie solely in event detection, but rather in understanding the event, correlating it with a security context, and providing an appropriate response. In a traditional SIEM system, an alert may appear for a denied access attempt, intensive network traffic, a suspicious process, or a new file within a web directory. However, the analyst still needs to manually read the logs, interpret them, and make the appropriate decision. In contrast, in the proposed system, RAG Detection analyzes the event and correlates it with a knowledge-based context, while RAG

Response generates an appropriate response recommendation. In certain critical cases, such as a DoS attack, the system can execute the action automatically while also providing a rollback capability.

Table 5.6 : Comparison Between Traditional SIEM and the Proposed System.

Scenario	Traditional SIEM	Wazuh + RAG/LLM + Active Response
Confidentiality	Displays the access attempt or permission denial	Understands it as an attempt to access a sensitive file and recommends disabling the user or investigating the account
DoS	Displays increased traffic or repeated alerts	Classifies it as an Availability attack and automatically blocks the IP address
Meterpreter	Displays process creation or suspicious activity	Correlates the process with a compromise context and recommends kill_process or isolate_host
Web Shell	Displays a suspicious file or modification	Classifies it as a Web Shell and recommends quarantine_file, then verifies the disappearance of the file

The general difference can be summarized as follows in **Table 5.7**:

Table 5.7: general difference between Models.

Model	Capability	Result
Traditional SIEM	Detection and alerting	Requires fully manual analysis and response
SIEM + LLM	Detection and general interpretation	May provide useful explanations, but may lack knowledge-grounded context
SIEM + RAG/LLM	Detection, interpretation, classification, and recommendation	More accurate analysis linked to a knowledge base
SIEM + RAG/LLM + Active Response	Detection, analysis, recommendation, and execution	Reduces MTTR and provides verifiable containment

5.10 Discussion of Results

The results show that integrating Wazuh with RAG, LLMs, and Active Response led to a clear improvement across the stages of handling security incidents. In all scenarios, the system was able to understand the nature of the event, classify it according to a clear security impact, and provide an appropriate response recommendation. The system also did not treat all events with the same severity level; rather, it differentiated between events that required manual intervention and those that justified automated execution. In the confidentiality test, the system behaved conservatively; it did

not automatically disable the user, but instead presented the action as a manual option. This was appropriate because the event represented a denied attempt rather than a confirmed compromise. In the DoS test, the attack was clear, aggressive, and impactful on service availability; therefore, the system automatically executed the blocking action. In the Meterpreter test, the system recommended terminating the process and isolating the host, but left the decision to the security administrator, since host isolation may have operational consequences. In the Web Shell test, the system recommended quarantining the malicious file, which was a precise and appropriate action because it targeted a specific file without disabling the entire service.

These results confirm that the system's execution policy is not arbitrary; rather, it considers the severity of the incident, the type of action, and its potential impact. Moreover, the presence of Rollback in several scenarios enhances confidence in automation, as the administrator can reverse an action if it is later found to be inappropriate.

5.11 Current Problems and Challenges

Despite the success of the tests, several challenges emerged and should be taken into consideration.

- **The first challenge** is the analysis time of the language model. The average analysis time reached 44.5 seconds, which is acceptable for a prototype, but may be relatively high in large production environments that contain a large number of alerts. The results indicate that the LLM inference stage is the most influential factor in overall processing time, while the other stages, such as Qdrant, Wazuh, Socket, and Active Response, take approximately fractions of a second.
- **The second challenge** is that manual responses still depend on the speed of the security administrator. The system can generate accurate recommendations, but their execution in manual cases depends on the administrator's decision. Therefore, improving the interface, clarifying the confidence level, and providing direct execution buttons can help reduce human decision-making time.

- **The third challenge** is the need to carefully tune the automated execution policy. Executing actions such as isolating a host or disabling a user may affect operations if performed incorrectly.
- **The fourth challenge** is that the test results were based on only four cases. Although these cases represent important attack patterns, a more comprehensive system evaluation would require executing a larger number of scenarios, repeating each scenario several times, and measuring the average values more accurately.

5.12 Technical Recommendations

Based on the test results, several technical recommendations can be proposed to improve the system.

- **First**, it is recommended to improve the performance of the language model by using a more powerful GPU, reducing the prompt size, reducing the number of retrieved chunks from Qdrant, or using a smaller model dedicated to the RAG Response stage. The results showed that the LLM processing time is the most significant factor affecting response time.
- **Second**, it is recommended to repeat each test scenario several times, such as 5 or 10 times, and then calculate the average, so that the results become more statistically accurate. The current tests demonstrate that the system functions correctly, but they are not sufficient on their own to provide a comprehensive statistical evaluation.
- **Third**, it is recommended to expand the response knowledge base to include more detailed playbooks for each type of attack. The higher the quality of the knowledge base, the more accurate the recommendations and the better the model's ability to select the appropriate action.
- **Fourth**, it is recommended to add more detailed levels for automated execution. For example, quarantine_file could be allowed automatically if the file is located within a web directory and contains clear Web Shell indicators, while isolate_host should remain manual except in cases of confirmed compromise.
- **Fifth**, it is recommended to develop a performance monitoring dashboard that displays the average RAG Detection time, average RAG Response time, number of automated responses, rollback success rate, and number of manually executed actions. This would help monitor the system's efficiency over time.

5.13 Future Prospects

This project can be developed in several future directions.

- **The first direction** is expanding Active Response capabilities to include more precise actions, such as applying rate limiting instead of complete blocking, isolating a specific service instead of isolating the entire host, or creating a snapshot before executing an action.
- **The second direction** is improving interaction with the security administrator by enhancing the Floating Button interface so that it displays the confidence score, reason for the recommendation, execution risks, and rollback possibility before the execution button is pressed. This would make human decision-making faster and safer.
- **The third direction** is connecting the system to live Threat Intelligence sources, allowing RAG to use up-to-date information about malicious IP addresses, indicators of compromise, or actively exploited vulnerabilities. This would make the analysis more realistic and more capable of handling emerging threats.
- **The fourth direction** is supporting broader evaluation in larger environments with a greater number of agents. In this study, the system was tested within a limited laboratory environment. However, in a production environment, the number of logs and alerts would be much larger, requiring improvements in filtering mechanisms and workload distribution.
- **The fifth direction** is improving the models used, either by using a larger model if sufficient computing resources are available, or by using a smaller cybersecurity-specialized model to reduce response time.

5.14 Conclusion

This chapter presented a practical evaluation of the developed security platform by executing four test cases covering confidentiality, availability, and integrity, in addition to a Windows compromise scenario using Meterpreter. The results showed that the system was able to detect all attacks, classify them correctly, provide appropriate response recommendations, and successfully execute some actions, while supporting Rollback in several cases.

The average total AI analysis time reached 44.5 seconds, indicating that the main performance factor was the language model inference stage, while actual execution actions were very fast and took approximately fractions of a second. The DoS test also demonstrated that the system is capable of executing a complete automated response when severity and safety conditions are met, while keeping other responses under the control of the security administrator when the operational impact of the action is higher.

It can be concluded that the proposed system succeeded in transforming Wazuh from a traditional detection and alerting platform into a more intelligent platform capable of contextual analysis, response recommendation generation, and verifiable containment action execution. Although the system still requires broader testing and performance improvements, the current results confirm the feasibility of integrating RAG/LLM with Wazuh and Active Response to build a more automated cybersecurity platform capable of reducing response time and improving the quality of security decision-making.

References :

[1] Virk S. G., Iqbal J., Ali A., Mahmud A. R., Rashid I., and Hanif T., "Advancing Security Operations Centers: Modern Use Cases, MITRE ATT&CK Integration, and Coverage Optimization " February 2025. https://jcbi.org/index.php/Main/article/view/1101

[2] Ismail K., Widyatama F., Wibawa I. M., Brata Z. A., Ukasyah, Nelistiani G. A., and Kim H., "Enhancing Security Operations Center: Wazuh Security Event Response with Retrieval-Augmented-Generation-Driven Copilot" February 2025. https://www.mdpi.com/1424-8220/25/3/870#B75-sensors-25-00870

[3] Lin X., Zhang J., Deng G., Liu T., Liu X., Yang C., Zhang T., Guo Q., and Chen R., "IRCopilot: Automated Incident Response with Large Language Models" May 2025. https://arxiv.org/abs/2505.20945

[4] Veluru S.P. & Manchala M.K., "Using LLMs as Incident Prevention Copilots in Cloud Infrastructure" December 2024. https://ijaibdcms.org/index.php/ijaibdcms/article/view/149

[5] Hays S. & White J., "Employing LLMs for Incident Response Planning and Review" March 2024. https://arxiv.org/abs/2403.01271

[6] Tellache A., Amara Korba A., Mokhtari A., Moldovan H., & Ghamri-Doudane Y., "Advancing Autonomous Incident Response: Leveraging LLMs and Cyber Threat Intelligence" August 2025. https://arxiv.org/abs/2508.10677

[7] Kim M., Wang J., Moore K., Goel D., "CyberAlly: Leveraging LLMs and Knowledge Graphs to Empower Cyber Defenders" May 2025. https://dl.acm.org/doi/abs/10.1145/3701716.3715171

[8] Stefanov M., et al., "Autonomous Agentic AI Architectures for Optimizing Security Operations Centers (SOC) KPIs" September 2025. https://sciendo.com/pdf/10.2478/raft-2025-0046

[9] Hammar K., Alpcan T., Lupu E. C., "Incident Response Planning Using a Lightweight Large Language Model with Reduced Hallucination" August 2025. https://arxiv.org/abs/2508.05188

[10] Salek M. S., Chowdhury M., Bin Munir M., Cai Y., Hasan M. I., Tine J.-M., Khan L., Rahman M., "A Large Language Model-Supported Threat Modeling Framework for Transportation Cyber-Physical Systems" June 2025. https://arxiv.org/abs/2506.00831

[11] Naraduhita P. P., Bumbungan M. T. E., "Leveraging LLM Integration with Wazuh and Jira for Automated Cyberattack Detection and Incident Response" June 2025. https://www.researchgate.net/publication/392734102_Leveraging_LLM_Integration_with_Wazuh_and_Jira_for_Automated_Cyberattack_Detection_and_Incident_Response

[12] Olabiyi W., "Combating Alert Fatigue: AI-Enhanced SIEM Framework for Optimized Incident Response" April 2023. https://www.researchgate.net/publication/386696111_Combating_Alert_Fatigue_AI-Enhanced_SIEM_Framework_for_Optimized_Incident_Response

[13] Scarfone K., Mell P., "Guide to Intrusion Detection and Prevention Systems (IDPS)" February 2007. https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-94.pdf

[14] Denning D.E., "An Intrusion-Detection Model" February 1987. https://www.cs.colostate.edu/~cs656/reading/ieee-se-13-2.pdf

[15] Chouhan P.K., Beard A., Chen L., "Intrusion Response Systems: Past, Present and Future" March 2023. https://arxiv.org/abs/2303.03070

[16] Miloslavskaya N., "Analysis of SIEM Systems and Their Usage in Security Operations and Security Intelligence Centers" August 2018. https://www.researchgate.net/publication/318708872_Analysis_of_SIEM_Systems_and_Their_Usage_in_Security_Operations_and_Security_Intelligence_Centers

[17] Xu Z., Wu Y., Wang S., Gao J., Qiu T., Wang Z., Wan H., Zhao X., "Deep Learning-based Intrusion Detection Systems: A Survey" April 2025. https://arxiv.org/abs/2504.07839

[18] Yang S., Zheng X., Zhang X., Xu J., Li J., Xie D., Long W., Ngai E.C.H., "Large Language Models for Network Intrusion Detection Systems: Foundations, Implementations, and Future Directions" July 2025. https://arxiv.org/abs/2507.04752

[19] Rai D.H., "Artificial Intelligence Through Time: A Comprehensive Historical Review" October 2024. https://www.researchgate.net/publication/385939923_Artificial_Intelligence_Through_Time_A_Comprehensive_Historical_Review

[20] Jaffal N.O., Alkhanafseh M., Mohaisen D., "Large Language Models in Cybersecurity: A Survey of Applications, Vulnerabilities, and Defense Techniques" September 2025. https://www.mdpi.com/2673-2688/6/9/216

[21] Lewis P., Perez E., Piktus A., Petroni F., Karpukhin V., Goyal N., Küttler H., Lewis M., Yih W., Rocktäschel T., Riedel S., Kiela D., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" May 2020. https://arxiv.org/abs/2005.11401

[22] Tian S., Zhang T., Liu J., Wang J., Wu X., Zhu X., Zhang R., Zhang W., Yuan Z., Mao S., Kim D.I., "Exploring the Role of Large Language Models in Cybersecurity: A Systematic Survey" April 2025. https://arxiv.org/abs/2504.15622

[23] Singh R., Tariq S., Jalalvand F., Baruwal Chhetri M., Nepal S., Paris C., Lochner M., "LLMs in the SOC: An Empirical Study of Human-AI Collaboration in Security Operations Centres" August 2025. https://arxiv.org/abs/2508.18947

[24] Rajapaksha S., Rani R., Karafili E., "A RAG-Based Question-Answering Solution for Cyber-Attack Investigation and Attribution" August 2024. https://arxiv.org/abs/2408.06272

[25] Al-kfairy M., Mustafa D., Kshetri N., Insiew M., Alfandi O., "Ethical Challenges and Solutions of Generative AI: An Interdisciplinary Perspective" August 2024. https://www.mdpi.com/2227-9709/11/3/58

[26] Arnob A.K.B., Roy Chowdhury R., Chaiti N.A., Saha S., Roy A., "A comprehensive systematic review of intrusion detection systems: emerging techniques, challenges, and future research directions" May 2025. https://acnsci.org/journal/index.php/jec/article/view/885